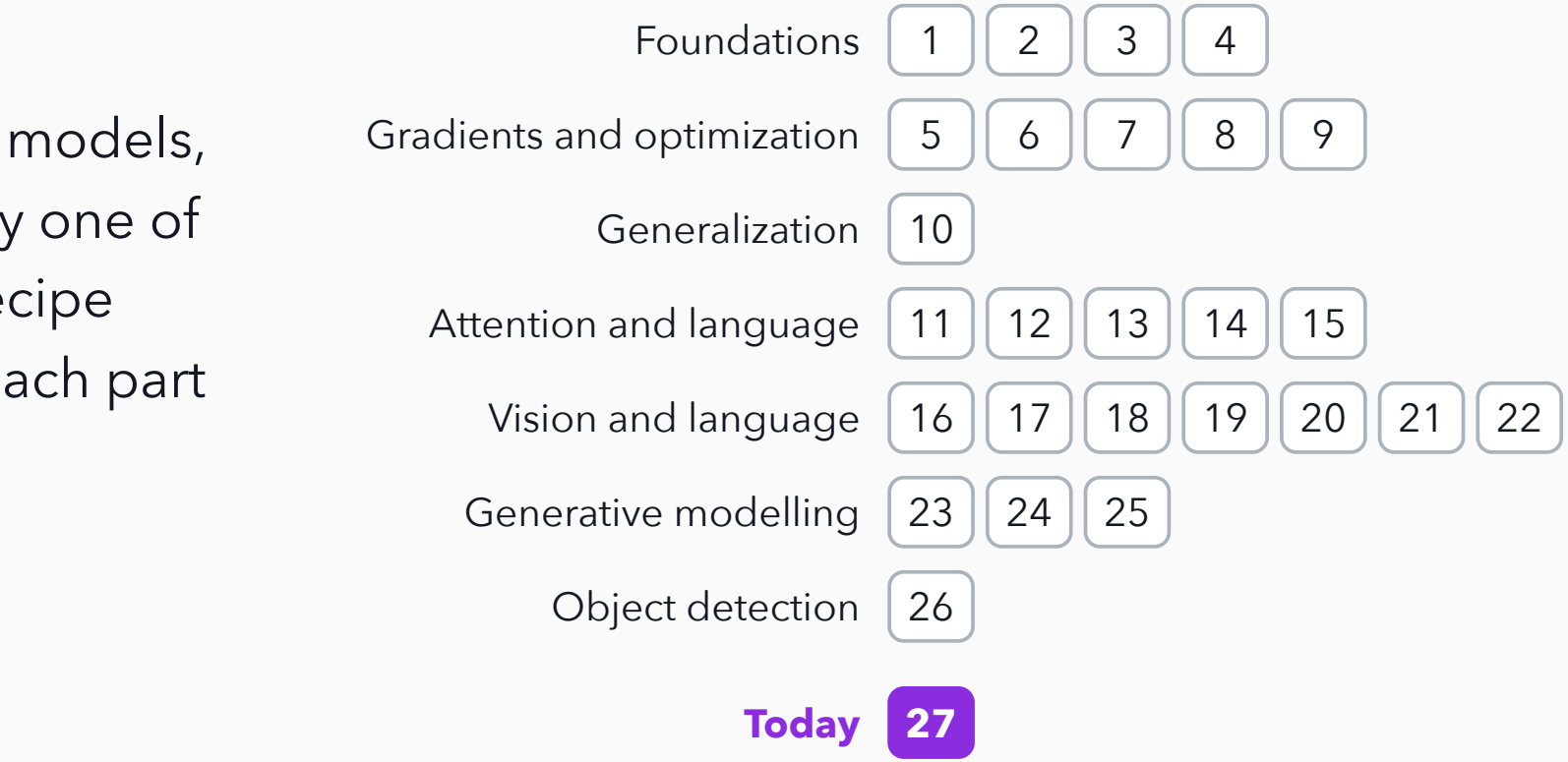
The course in one lecture

Twenty-six lectures built ten models, and one recipe trained every one of them. Today we write that recipe down once, then see what each part of the course changed in it.

ES 667 Deep Learning · Lecture 27
Nipun Batra · IIT Gandhinagar



Every tile is one lecture and opens its slides. The seven rows are the seven parts of today.

Ten models, built in twenty-six lectures

MLP classifier **a class**	Character language model **the next character**	Transformer language model **the next token**	Vision Transformer **the class of a photo**	CLIP **which caption fits which photo**
Vision-language model **words about a photo**	Concept bottleneck model **concepts, then a class**	Bayesian neural network **a prediction and its uncertainty**	Variational autoencoder **new images**	Object detector **a list of boxes**

> The models we built, in order, each with what it returns. What do all ten have in common?

Ten models, built in twenty-six lectures

MLP classifier **a class**	Character language model **the next character**	Transformer language model **the next token**	Vision Transformer **the class of a photo**	CLIP **which caption fits which photo**
Vision-language model **words about a photo**	Concept bottleneck model **concepts, then a class**	Bayesian neural network **a prediction and its uncertainty**	Variational autoencoder **new images**	Object detector **a list of boxes**

Each one outputs a probability distribution. Its loss is the negative log of the probability given to what was observed.

So one recipe trained them all. The next slides run it once, on the smallest network we built.

The recipe, on one example

Training: one step, on one example

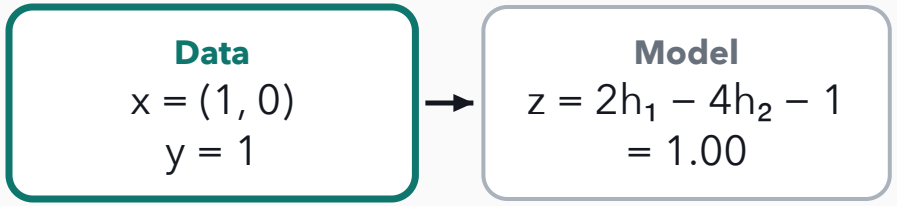
Data
x = (1, 0)
y = 1

> One training example from Lecture 4's XOR problem: the input x = (1, 0) and its **target y = 1**.

The network is the one Lecture 4 built by hand for XOR. Numbers: scripts/sci/p0.py.

The recipe, on one example

Training: one step, on one example

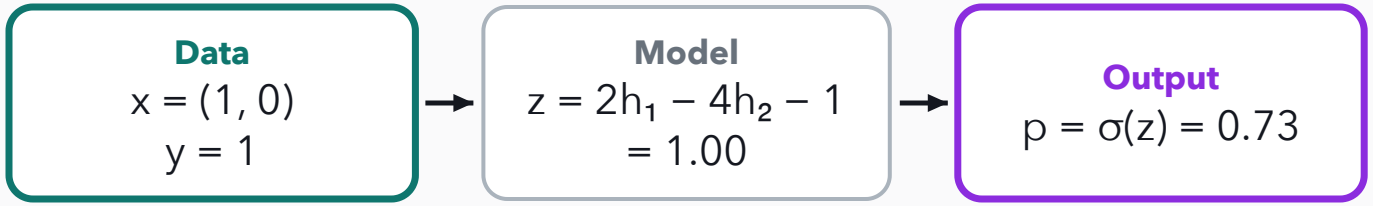


> The model is a function with 9 parameters θ. Its two ReLU units give h = (1, 0), and the last layer turns them into a score, z = 1.00.

The network is the one Lecture 4 built by hand for XOR. Numbers: scripts/sci/p0.py.

The recipe, on one example

Training: one step, on one example

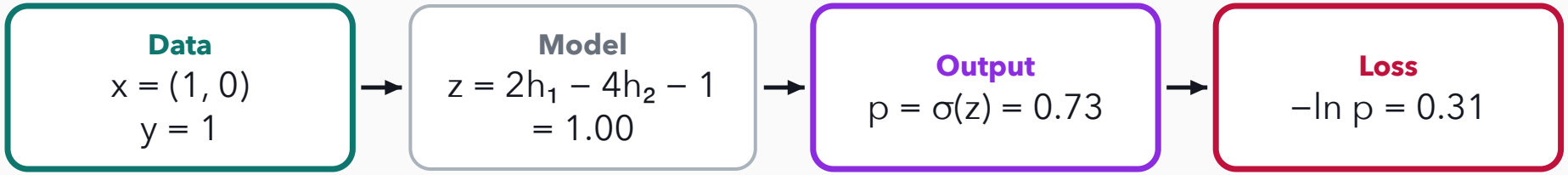


A sigmoid turns the score into a probability. The network says y = 1 with **p = 0.73**.

The network is the one Lecture 4 built by hand for XOR. Numbers: scripts/sci/p0.py.

The recipe, on one example

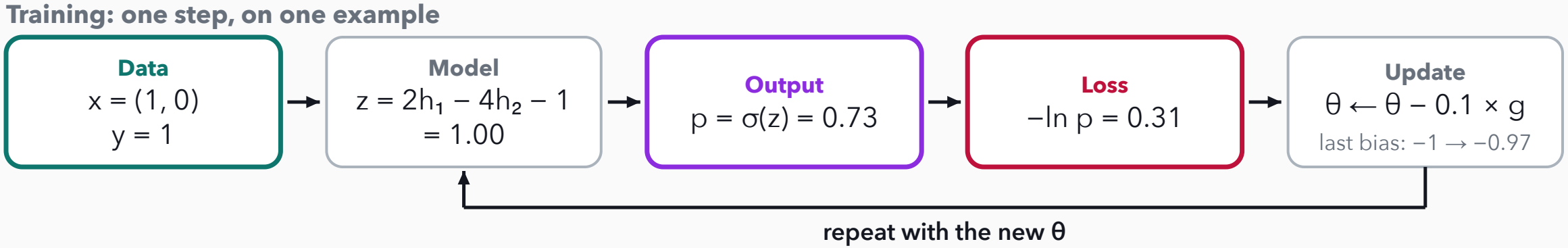
Training: one step, on one example



> The loss is the negative log of the probability the network gave to the right answer: **−ln 0.73 = 0.31**.

The network is the one Lecture 4 built by hand for XOR. Numbers: scripts/sci/p0.py.

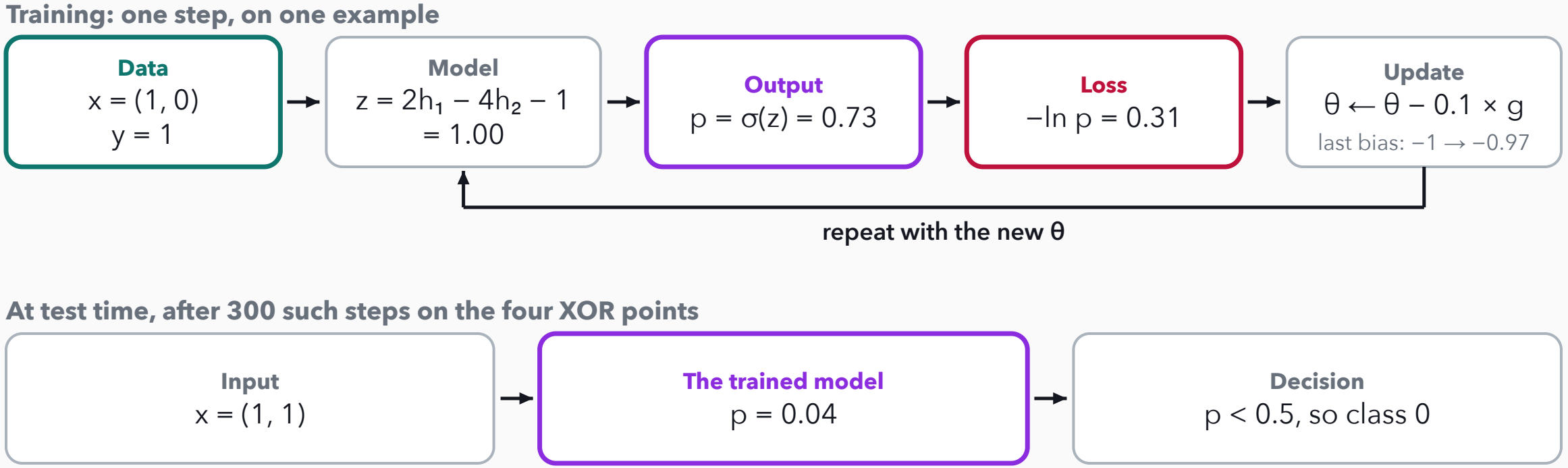
The recipe, on one example



> Backprop gives the gradient g of the loss for all 9 parameters. One step of size 0.1 against it moves p from 0.73 to 0.78 and the loss to 0.25.

The network is the one Lecture 4 built by hand for XOR. Numbers: scripts/sci/p0.py.

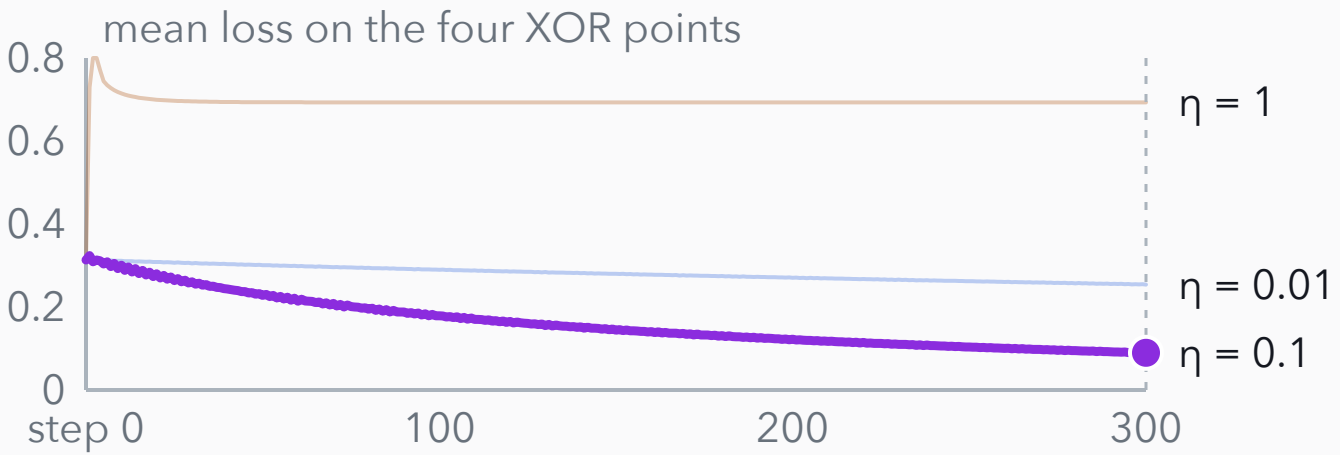
The recipe, on one example



> Training repeats that step. At test time the loop is gone: the trained network takes an input and returns a probability.

The network is the one Lecture 4 built by hand for XOR. Numbers: scripts/sci/p0.py.

The recipe, repeated

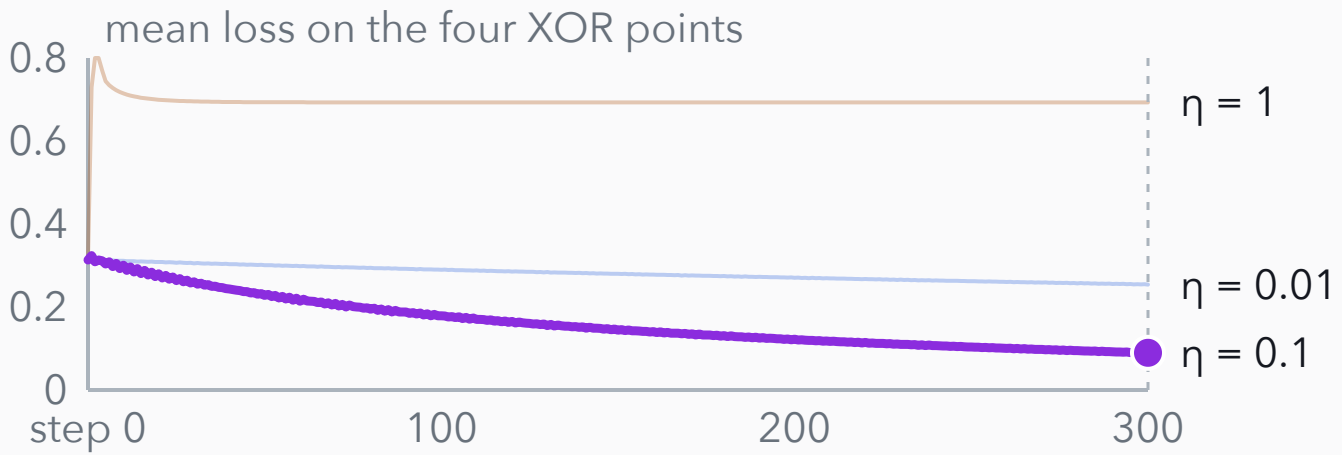


input x	target y	prediction p
(0, 0)	**0**	**0.18**
(0, 1)	**1**	**0.94**
(1, 0)	**1**	**0.94**
(1, 1)	**0**	**0.04**
mean loss after 300 steps		**0.09**

> The same step, now on all four XOR points, 300 times. Drag the number of steps and switch the step size η. Which of the three learns XOR?

The network is the one Lecture 4 built by hand for XOR. Numbers: scripts/sci/p0.py.

The recipe, repeated



input x	target y	prediction p
(0, 0)	**0**	**0.18**
(0, 1)	**1**	**0.94**
(1, 0)	**1**	**0.94**
(1, 1)	**0**	**0.04**
mean loss after 300 steps		**0.09**

Only η = 0.1 learns XOR: its loss falls from 0.31 to 0.09. At 0.01 it only reaches 0.25. At 1 it jumps to 2.15, then stays at 0.69 with every prediction 0.5.

> The loop is the same in all three runs; only the step size differs. Part 2 comes back to that choice.

The network is the one Lecture 4 built by hand for XOR. Numbers: scripts/sci/p0.py.

The recipe as a card, and as code

The recipe, for a classifier

Data	pairs (x, y)
Model	scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

```
for x, y in loader:
```

The card has one row per step of the flowchart. Beside it is the loop that ran in every lecture. The loader hands over a batch of pairs (x, y).

The recipe as a card, and as code

The recipe, for a classifier

Data	pairs (x, y)
Model	scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

```
for x, y in loader:
    logits = model(x)
```

> The model turns the inputs into scores, one per class.

The recipe as a card, and as code

The recipe, for a classifier

Data	pairs (x, y)
Model	scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

```
for x, y in loader:
    logits = model(x)
    loss = F.cross_entropy(logits, y)
```

cross_entropy applies the softmax and takes −ln of the probability of the right class, so this one line is two rows of the card.

The recipe as a card, and as code

The recipe, for a classifier

Data	pairs (x, y)
Model	scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

```
for x, y in loader:
    logits = model(x)
    loss = F.cross_entropy(logits, y)
    opt.zero_grad()
```

PyTorch adds new gradients to the old ones, so the loop clears them first.

The recipe as a card, and as code

The recipe, for a classifier

Data	pairs (x, y)
Model	scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

```
for x, y in loader:
    logits = model(x)
    loss = F.cross_entropy(logits, y)
    opt.zero_grad()
    loss.backward()
```

backward() is backprop. It fills in the gradient of the loss for every parameter.

The recipe as a card, and as code

The recipe, for a classifier

Data	pairs (x, y)
Model	scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

```
for x, y in loader:
    logits = model(x)
    loss = F.cross_entropy(logits, y)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

step() applies the update. With SGD or with Adam, this line stays the same.

The route for today

Part	What it changed in the recipe
1 · Foundations	the loss, read off a likelihood
2 · Gradients and optimization	how the update is computed
3 · Generalization	how the fit does on new data
4 · Attention and language	a model for sequences of tokens
5 · Vision and language	images as tokens, captions as labels
6 · Generative modelling	distributions over weights and codes
7 · Object detection	the output becomes a list of boxes

> Each part changed some rows of the card. We take the parts in order and ask of each: which rows, and how?

Part 1 · Where the loss comes from

Every loss in this course was read off a probability model. This part recalls how, and builds the first network.

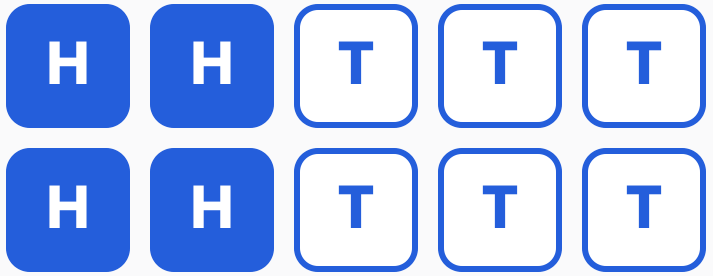
1. Pick a distribution for the output and write the likelihood of the data.
2. Take the negative log: that is the loss.
3. A prior on the weights adds a penalty.
4. Stack linear layers and ReLUs to get a network.

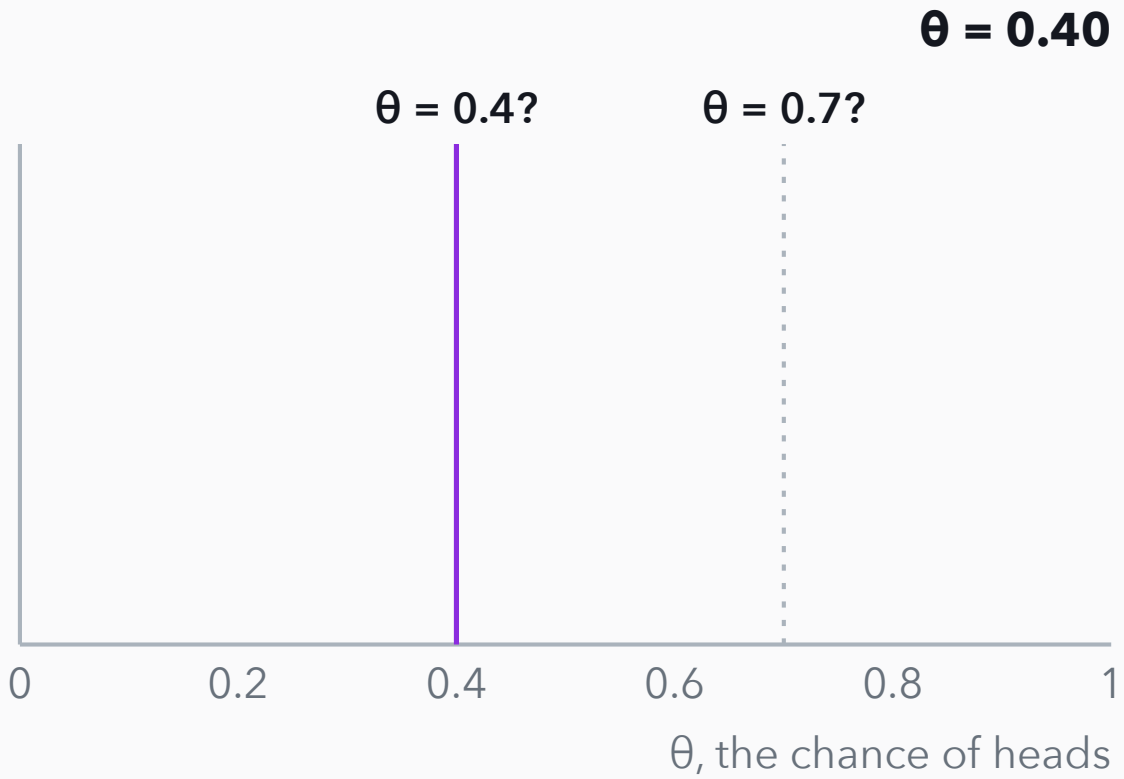
Lectures 1 to 4

> By the end of this part you can say where cross-entropy and squared error come from.

From Lecture 1

The loss is a negative log-likelihood





Lecture 1's ten tosses: four heads, six tails. Drag θ, the coin's chance of heads. Which θ would you bet on: 0.4, 0.7, or another?

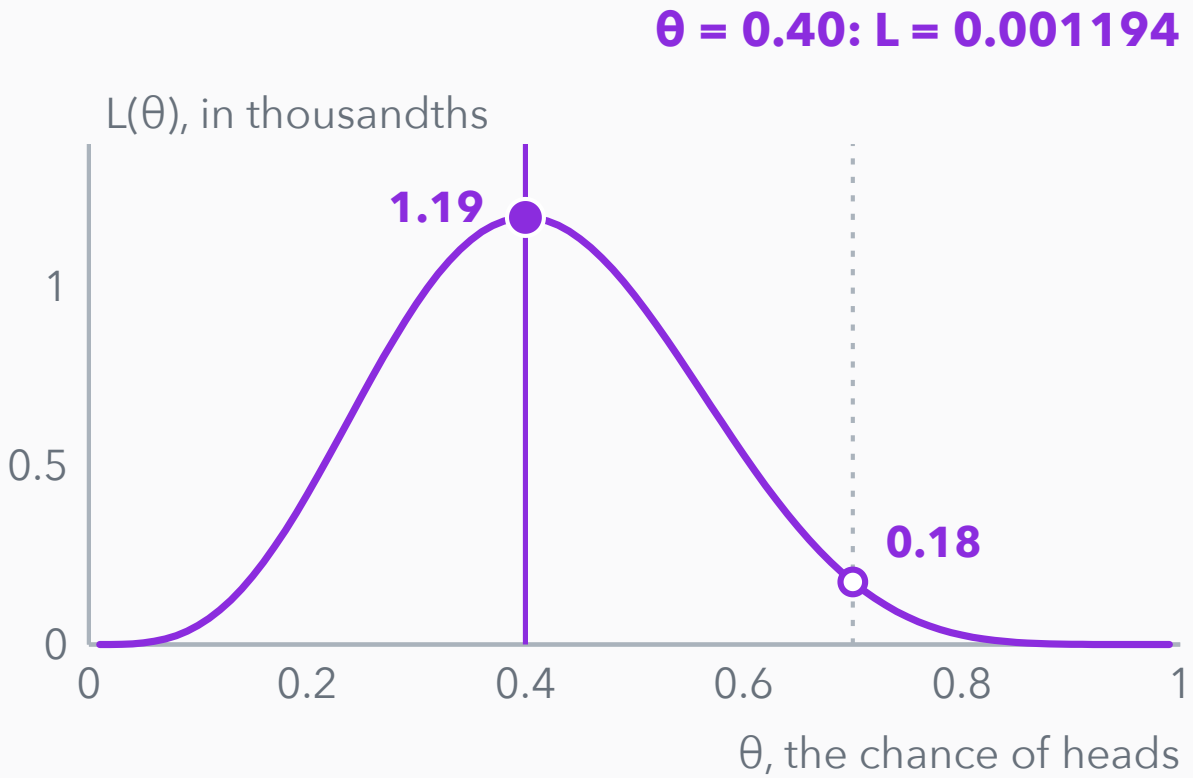
The tosses HHTTTHHTTT are Lecture 1's. Numbers: scripts/sci/p1.py.

From Lecture 1

The loss is a negative log-likelihood

H	H	T	T	T
H	H	T	T	T

$$L(\theta) = \theta^4 (1 - \theta)^6$$

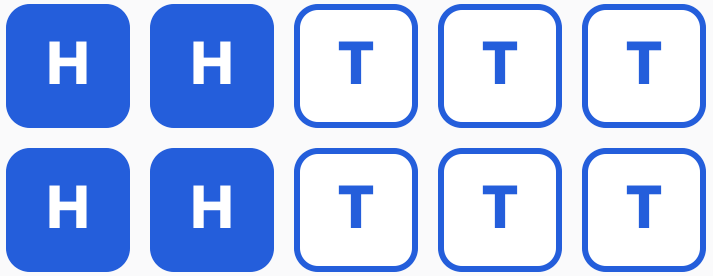


> The likelihood scores each θ by how probable it makes these tosses. At 0.4 they are 6.82 times as likely as at 0.7.

The tosses HHTTTHHTTT are Lecture 1's. Numbers: scripts/sci/p1.py.

From Lecture 1

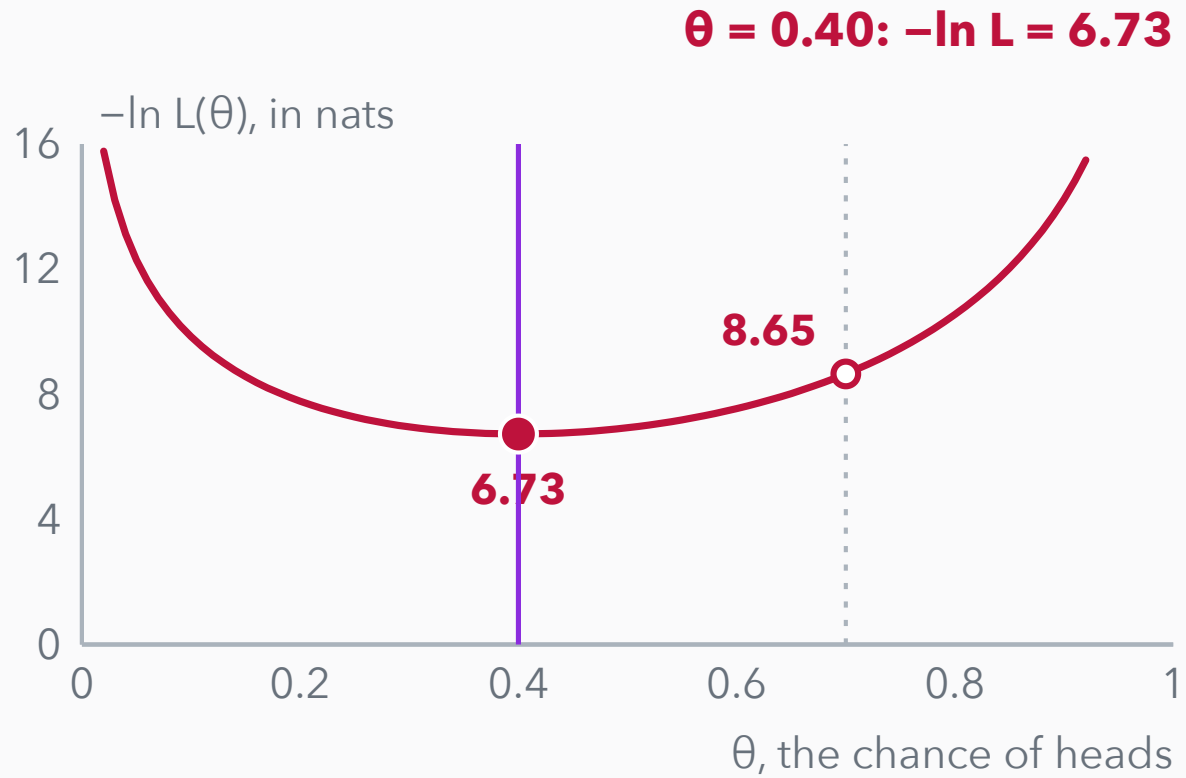
The loss is a negative log-likelihood



$$L(\theta) = \theta^4 (1-\theta)^6$$

$$-\ln L(\theta) = -4\ln\theta - 6\ln(1-\theta)$$

$$\text{lowest at } \hat{\theta} = \tfrac{4}{10} = 0.4$$



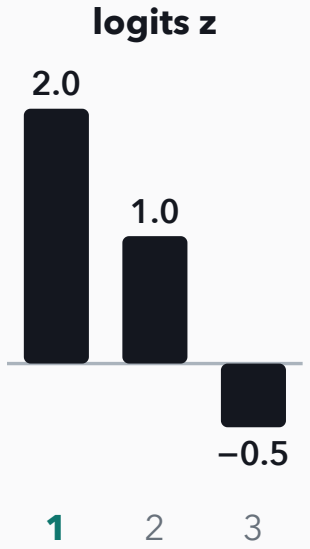
> Take the negative log. The product becomes a sum and the best θ becomes the lowest point: 6.73 nats at θ = 0.4.

The tosses HHTTTHHTTT are Lecture 1's. Numbers: scripts/sci/p1.py.

From Lecture 2

Cross-entropy is the negative log of the right class

$$-\ln p(y \mid x) = -y \ln p - (1 - y) \ln(1 - p)$$



> Two networks. One says **p = 0.8** for a label **y = 1**. The other gives the logits shown, and class 1 is correct. What loss does each pay?

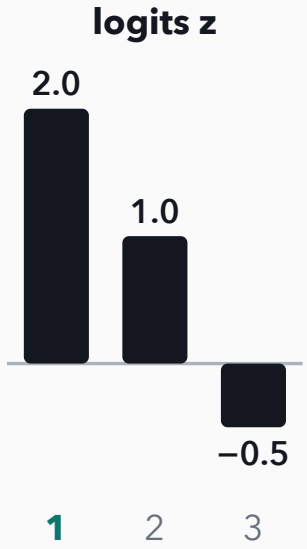
The logits (2, 1, −0.5) are from Lecture 2's softmax chart. Numbers: scripts/sci/p1.py.

From Lecture 2

Cross-entropy is the negative log of the right class

$$-\ln p(y \mid x) = -y \ln p - (1 - y) \ln(1 - p)$$

$y = 1,\ p = 0.8 :\quad -\ln 0.8 = 0.22$



> The Bernoulli's negative log is binary cross-entropy: **−ln 0.8 = 0.22**. Had it said 0.2, it would pay 1.61.

The logits (2, 1, −0.5) are from Lecture 2's softmax chart. Numbers: scripts/sci/p1.py.

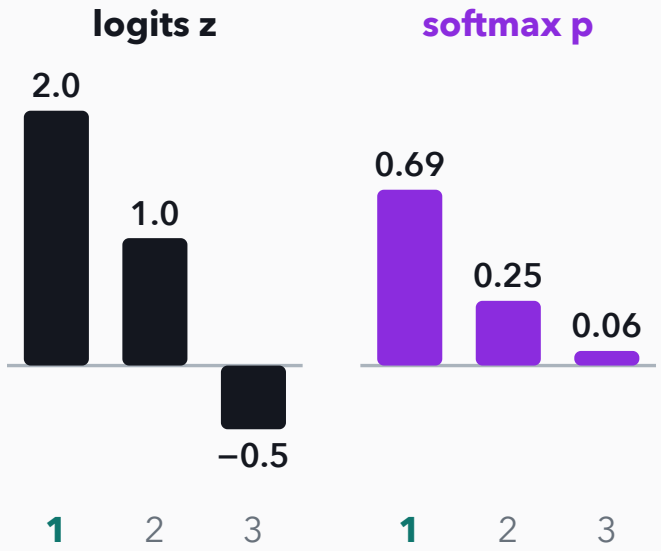
From Lecture 2

Cross-entropy is the negative log of the right class

$$-\ln p(y \mid x) = -y \ln p - (1-y)\ln(1-p)$$

$$y = 1,\ p = 0.8: \quad -\ln 0.8 = 0.22$$

$$-\ln p_1 = -z_1 + \ln \sum_j e^{z_j} = -2 + 2.37 = 0.37$$



> Softmax turns the logits into probabilities. Class 1 gets 0.69, so the loss is **−ln 0.69 = 0.37**. That is cross-entropy.

The logits (2, 1, −0.5) are from Lecture 2's softmax chart. Numbers: scripts/sci/p1.py.

From Lecture 2

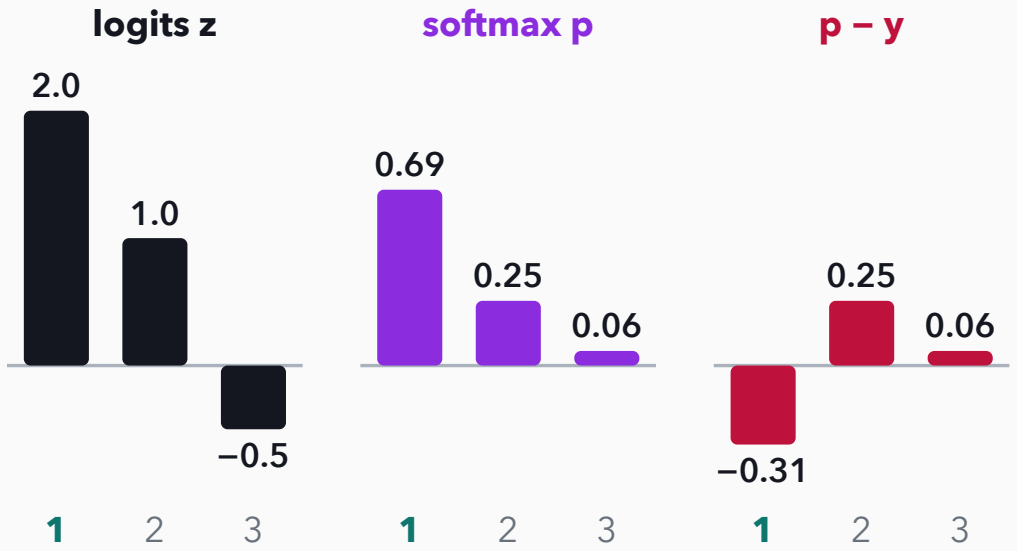
Cross-entropy is the negative log of the right class

$$-\ln p(y \mid x) = -y \ln p - (1-y)\ln(1-p)$$

$$y = 1,\ p = 0.8: \quad -\ln 0.8 = 0.22$$

$$-\ln p_1 = -z_1 + \ln \sum_j e^{z_j} = -2 + 2.37 = 0.37$$

$$\frac{\partial\, \text{loss}}{\partial z} = p - y = (-0.31,\ 0.25,\ 0.06)$$



> Its gradient at the logits is p − y. A step against it raises the right class's logit and lowers the others by what they took.

The logits (2, 1, −0.5) are from Lecture 2's softmax chart. Numbers: scripts/sci/p1.py.

From Lecture 3

Each kind of target picks a distribution, and a loss

the target	distribution	output layer	loss	one example
a label, 0 or 1	Bernoulli	sigmoid	binary cross-entropy	$-\ln 0.8 = 0.22$
one of K classes	categorical	softmax	cross-entropy	$-\ln 0.69 = 0.37$
a real number				

> The last two slides in one table: each kind of target gets a distribution, and its negative log is the loss. What goes in the last row?

Rows follow the retrieval summary of Lectures 2 and 3; the Gaussian example is from the S03 cheat sheet. Numbers: scripts/sci/p1.py.

From Lecture 3

Each kind of target picks a distribution, and a loss

the target	distribution	output layer	loss	one example
a label, 0 or 1	Bernoulli	sigmoid	binary cross-entropy	$-\ln 0.8 = 0.22$
one of K classes	categorical	softmax	cross-entropy	$-\ln 0.69 = 0.37$
a real number	Gaussian, fixed σ	none (the mean)	squared error	

> A real number gets a Gaussian with a fixed σ. The network outputs its mean, with nothing to squash it.

Rows follow the retrieval summary of Lectures 2 and 3; the Gaussian example is from the S03 cheat sheet. Numbers: scripts/sci/p1.py.

From Lecture 3

Each kind of target picks a distribution, and a loss

the target	distribution	output layer	loss	one example
a label, 0 or 1	Bernoulli	sigmoid	binary cross-entropy	$-\ln 0.8 = 0.22$
one of K classes	categorical	softmax	cross-entropy	$-\ln 0.69 = 0.37$
a real number	Gaussian, fixed σ	none (the mean)	squared error	$r = 2:\ \ 2 + 0.92$

$$-\ln \mathcal{N}(y;\ \mu, \sigma^2) = \frac{(y-\mu)^2}{2\sigma^2} + \tfrac{1}{2}\ln(2\pi\sigma^2)$$

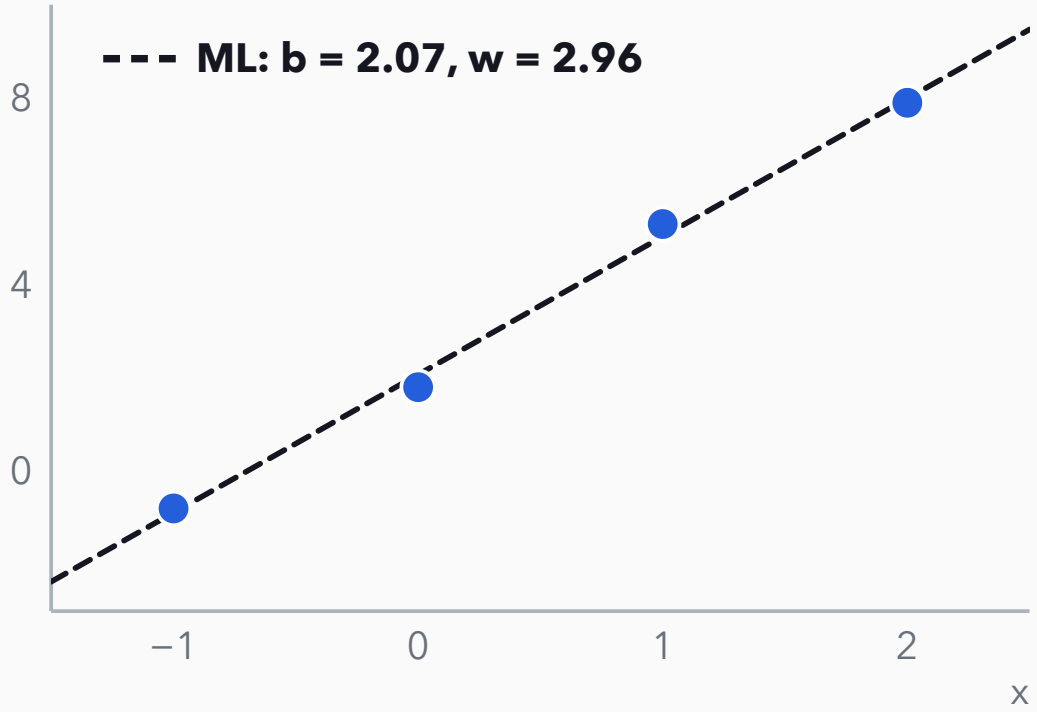
> Its negative log is the squared error over 2σ², plus a constant that never moves the fit. With r = 2 and σ = 1: 2 plus 0.92.

Rows follow the retrieval summary of Lectures 2 and 3; the Gaussian example is from the S03 cheat sheet. Numbers: scripts/sci/p1.py.

From Lecture 3

A Gaussian prior is an L2 penalty

$$\text{ML: } \min_{\theta} \frac{1}{2\sigma^2} \sum_i \left(y_i - (b + w x_i)\right)^2$$



> Four points from 2 + 3x, noise σ = 0.5. Maximum likelihood gives the dashed line. With a prior θ ~ N(0, 0.5²), which way does it move?

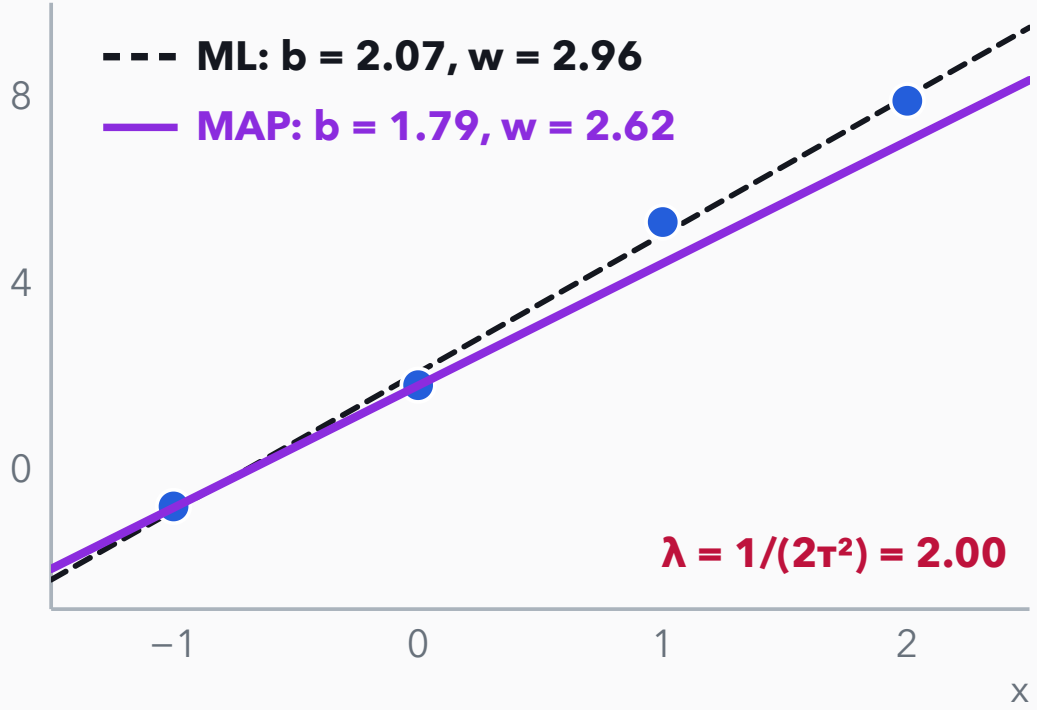
The four points and σ = 0.5 are Lecture 3's MAP example. Numbers: scripts/sci/p1.py.

From Lecture 3

A Gaussian prior is an L2 penalty

$$\text{ML: } \min_{\theta} \frac{1}{2\sigma^2} \sum_i \left(y_i - (b + w x_i)\right)^2$$

$$\text{MAP: add } -\ln p(\theta) = \frac{1}{2\tau^2} \|\theta\|^2 + C$$



Toward zero. The prior adds its negative log to the loss, and at τ = 0.5 the fit is (1.79, 2.62). Drag τ.

The four points and σ = 0.5 are Lecture 3's MAP example. Numbers: scripts/sci/p1.py.

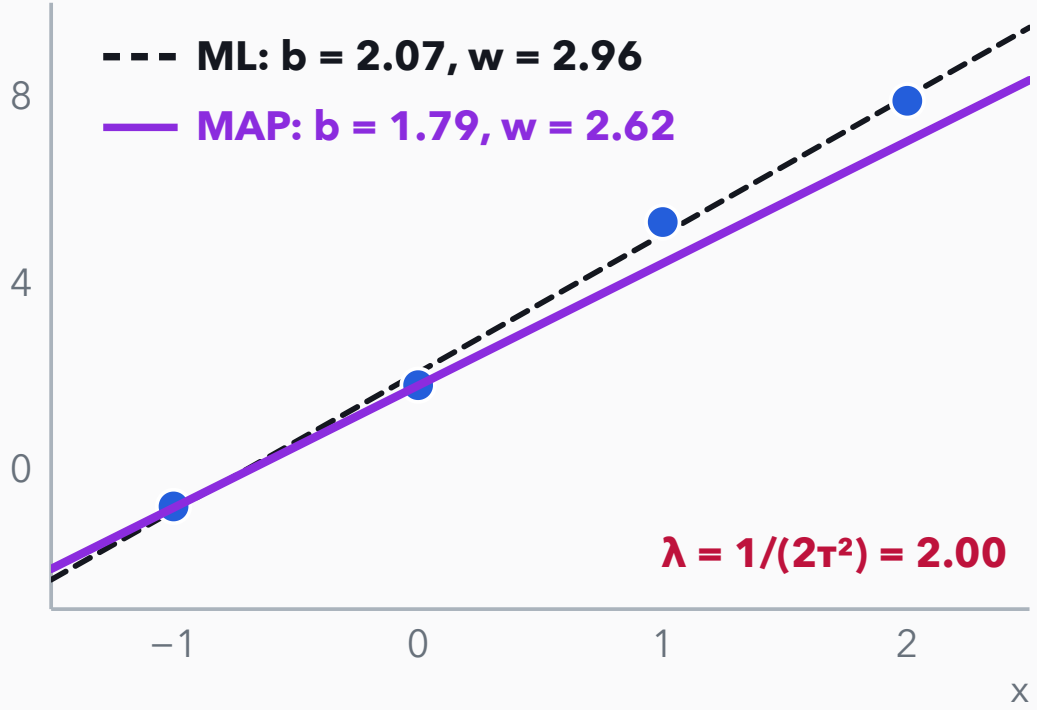
From Lecture 3

A Gaussian prior is an L2 penalty

$$\text{ML: } \min_{\theta} \frac{1}{2\sigma^2} \sum_i \left(y_i - (b + w x_i)\right)^2$$

$$\text{MAP: add } -\ln p(\theta) = \frac{1}{2\tau^2} \|\theta\|^2 + C$$

> A Gaussian prior on the weights is the L2 penalty, with $\lambda = 1/(2\tau^2)$. In PyTorch it is weight decay.



> A narrow prior, τ = 0.1, pulls to (0.44, 0.68); a wide one gives maximum likelihood back.

The four points and σ = 0.5 are Lecture 3's MAP example. Numbers: scripts/sci/p1.py.

From Lecture 4

Two ReLU units solve XOR



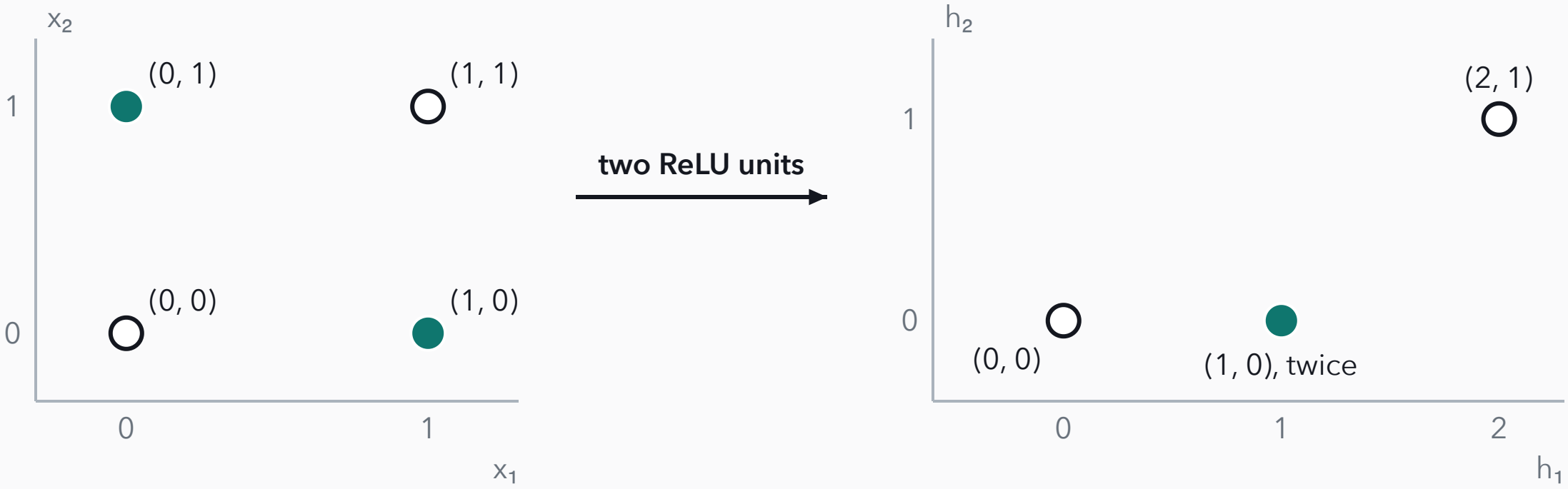
Filled: y = 1. Hollow: y = 0.

> Lecture 4's XOR. The two filled points have y = 1. Can one straight line put them on one side and the hollow points on the other?

The XOR network is the one Lecture 4 built by hand. Numbers: scripts/sci/p1.py.

From Lecture 4

Two ReLU units solve XOR



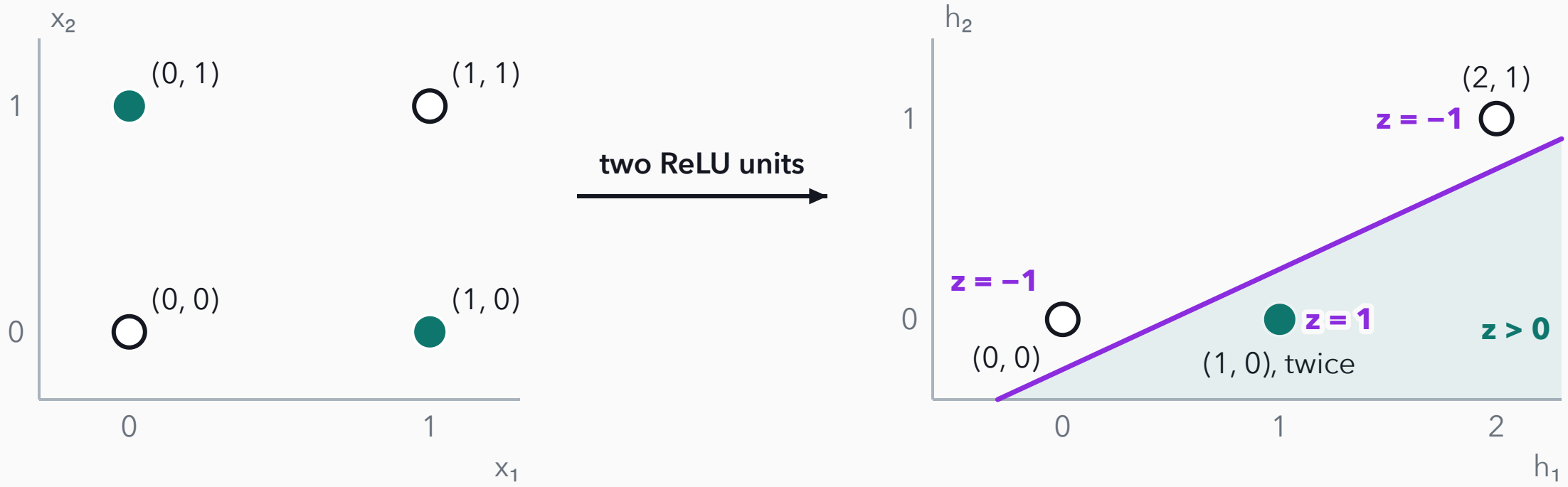
$$h_1 = \mathrm{ReLU}(x_1 + x_2), \quad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$$

No line can. Two ReLU units move the points: (1, 0) and (0, 1) land on the same spot, h = (1, 0).

The XOR network is the one Lecture 4 built by hand. Numbers: scripts/sci/p1.py.

From Lecture 4

Two ReLU units solve XOR



$$h_1 = \mathrm{ReLU}(x_1 + x_2), \quad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$$

$$z = 2h_1 - 4h_2 - 1$$

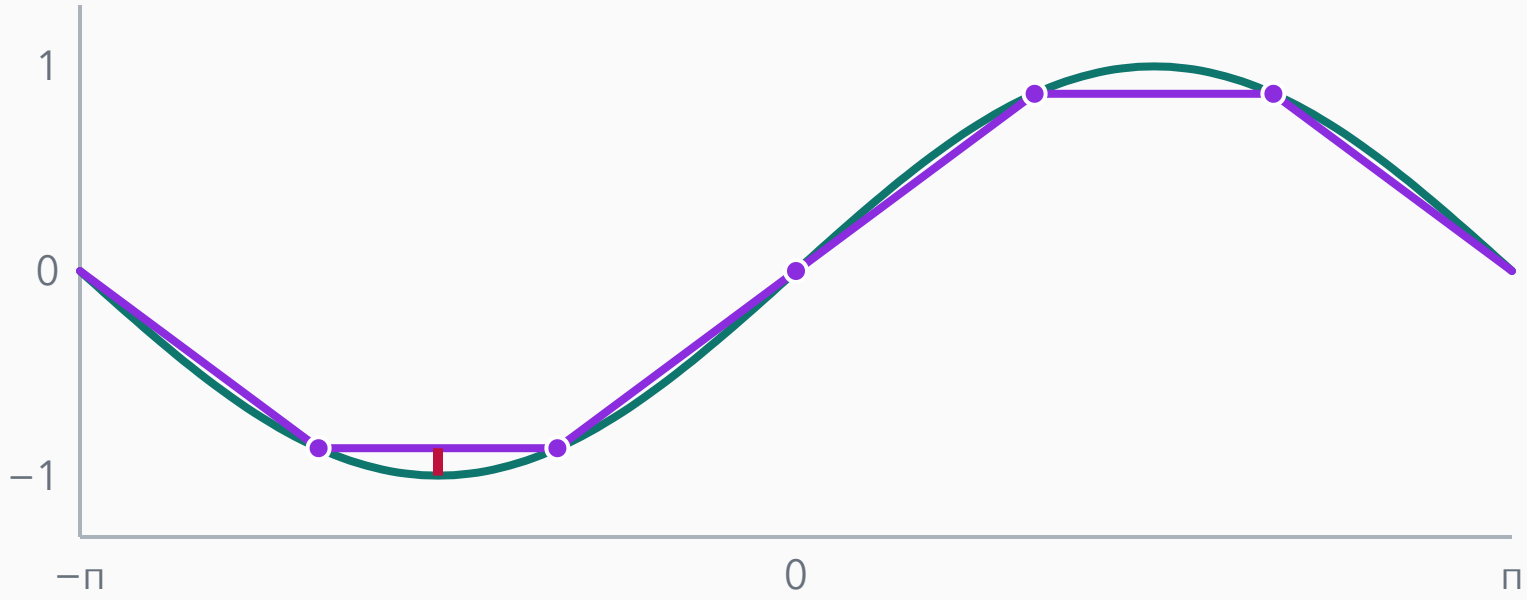
In h, one line does it: z is positive only for the filled points. This is the network the opening trained.

The XOR network is the one Lecture 4 built by hand. Numbers: scripts/sci/p1.py.

From Lecture 4

Enough hinges fit any curve

$$g(x) = c + \sum_j v_j \ \mathrm{ReLU}(x - k_j)$$



sin x, the target

a sum of hinges

hinges

5

largest gap

0.134

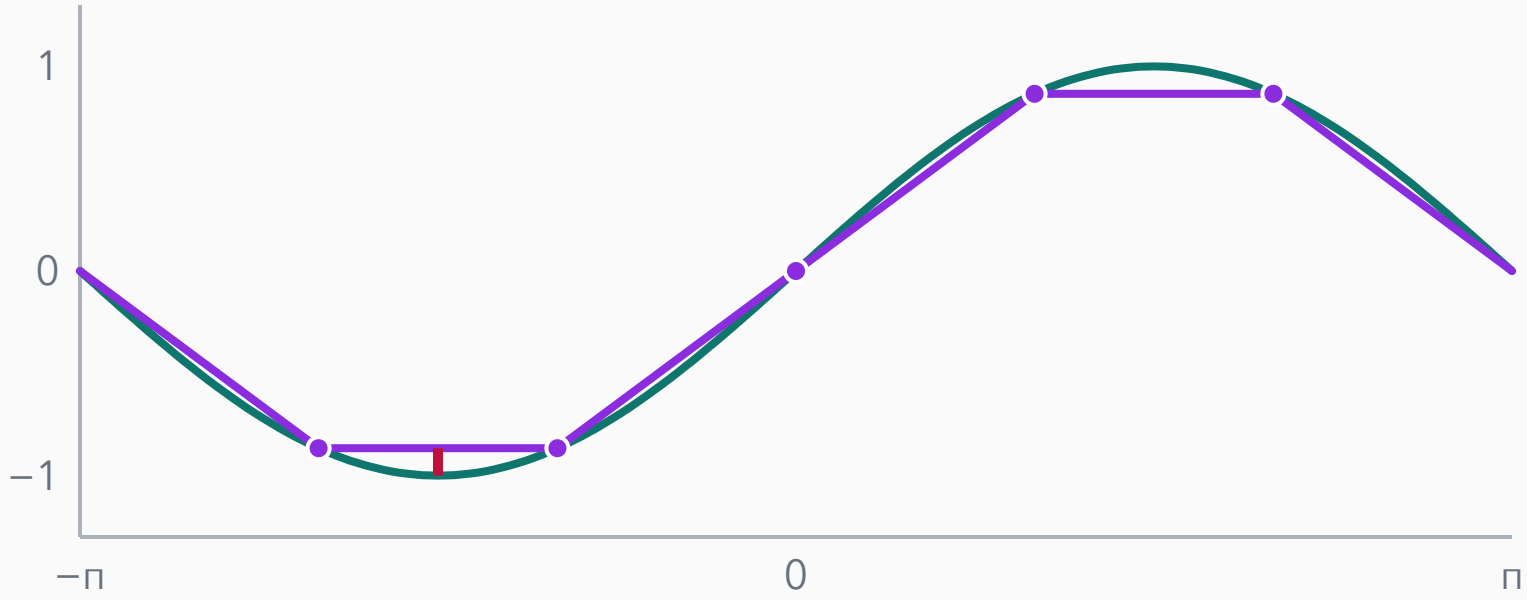
Each ReLU is a hinge, and a sum of hinges is a bent line. Here it is fitted to sin x. Add hinges: what happens to the largest gap?

Fits of sin x on [−π, π] with equally spaced hinges, as in Lecture 4. Numbers: scripts/sci/p1.py.

From Lecture 4

Enough hinges fit any curve

$$g(x) = c + \sum_j v_j \ \mathrm{ReLU}(x - k_j)$$



sin x, the target

a sum of hinges

hinges

5

largest gap

0.134

It shrinks: 0.437, 0.134, 0.019 for 2, 5, 15 hinges. Enough hinges fit any continuous curve on a bounded range; training still has to find them.

Fits of sin x on [−π, π] with equally spaced hinges, as in Lecture 4. Numbers: scripts/sci/p1.py.

Check: a loss of 2.30 at the first step

```
step 1   loss 2.3026
```

A 10-class classifier, trained with cross-entropy, reports this loss at its first step. Is something broken?

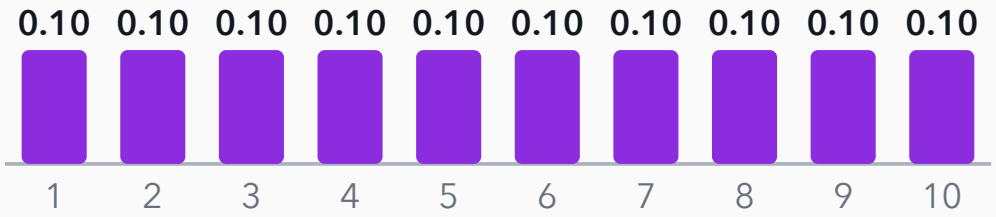
> Two ideas of this part meet here. Take a minute with your neighbour.

Check: a loss of 2.30 at the first step

```
step 1   loss 2.3026
```

A 10-class classifier, trained with cross-entropy, reports this loss at its first step. Is something broken?

No. A fresh network gives the ten classes about equal probability, 0.10 each, and $-\ln 0.1 = \ln 10 = 2.30$. It starts where it should.



> ln 10 is the loss of ten equal guesses. A first loss far above it would mean the starting logits are too large.

Part 1 in one slide

1. A loss is $-\ln$ of the probability the model gave the data. Cross-entropy and squared error both come from there.
2. A prior on the weights adds $-\ln p(\theta)$. A Gaussian prior is the L2 penalty.
3. ReLUs between linear layers let the model bend: two units solve XOR.

MLP classifier

Data	pairs (x, y): an input and its class
Model	an MLP gives scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

Next: how the gradient of that loss is computed, for every weight at once.

Part 2 · How the weights move

The Update row needs a gradient and a step size. These lectures were about getting both right.

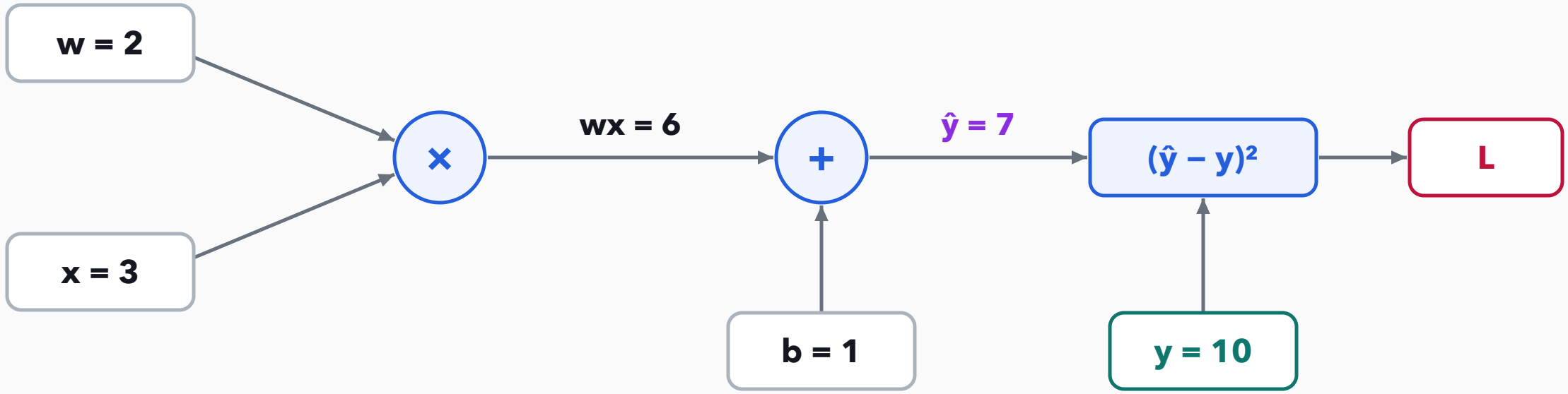
1. Backpropagation computes the gradient, one node at a time.
2. A mini-batch gives a cheap, noisy estimate of it.
3. Momentum and Adam shape each step.
4. A schedule shrinks the step size over time.

Lectures 5 to 9

> By the end you can run backprop by hand and say what each optimizer adds to the plain step.

From Lecture 6

Backprop, one node at a time



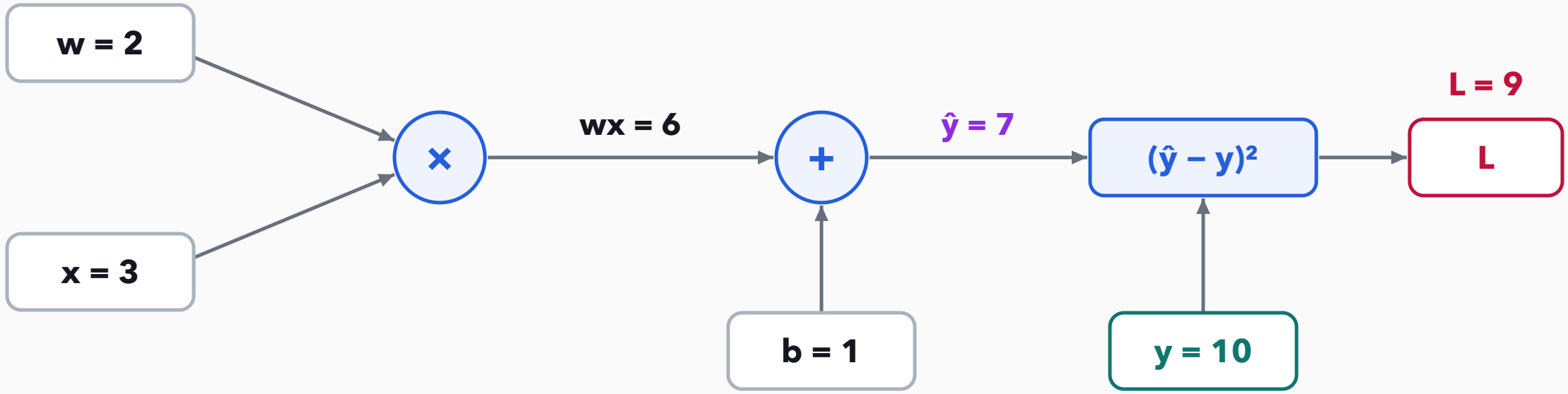
$$\hat{y} = wx + b = 2 \cdot 3 + 1 = 7$$

> Lecture 6's graph for one prediction, ŷ = wx + b. The forward pass works left to right and stores every value on the way.

Example: Lecture 6, the central graph. Numbers: scripts/sci/p2.py.

From Lecture 6

Backprop, one node at a time



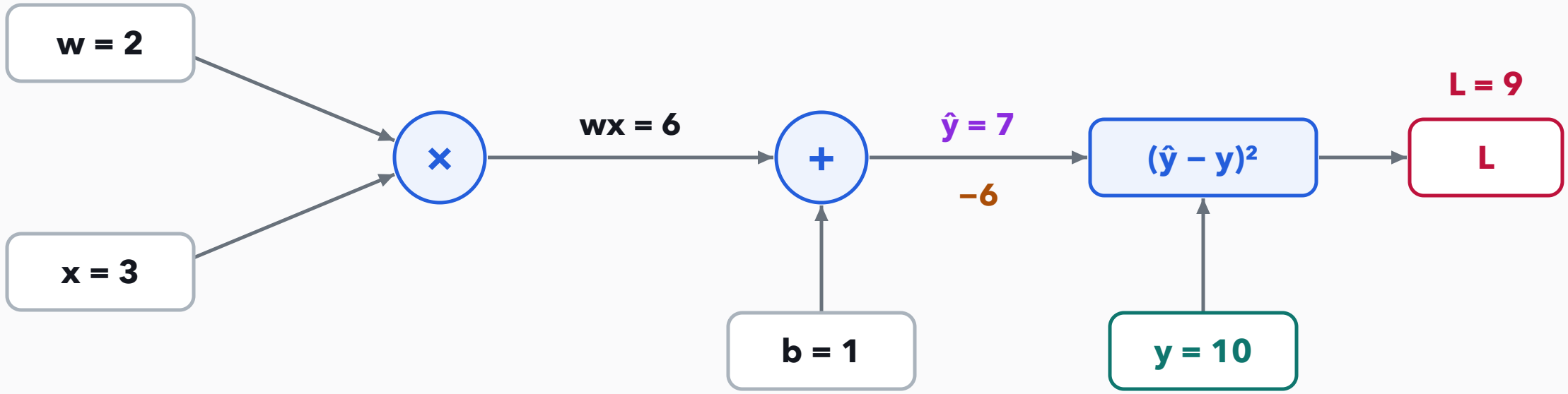
$$L = (\hat{y} - y)^2 = (7 - 10)^2 = 9$$

> The target is **y = 10**. Should w go up or down to lower the loss?

Example: Lecture 6, the central graph. Numbers: scripts/sci/p2.py.

From Lecture 6

Backprop, one node at a time



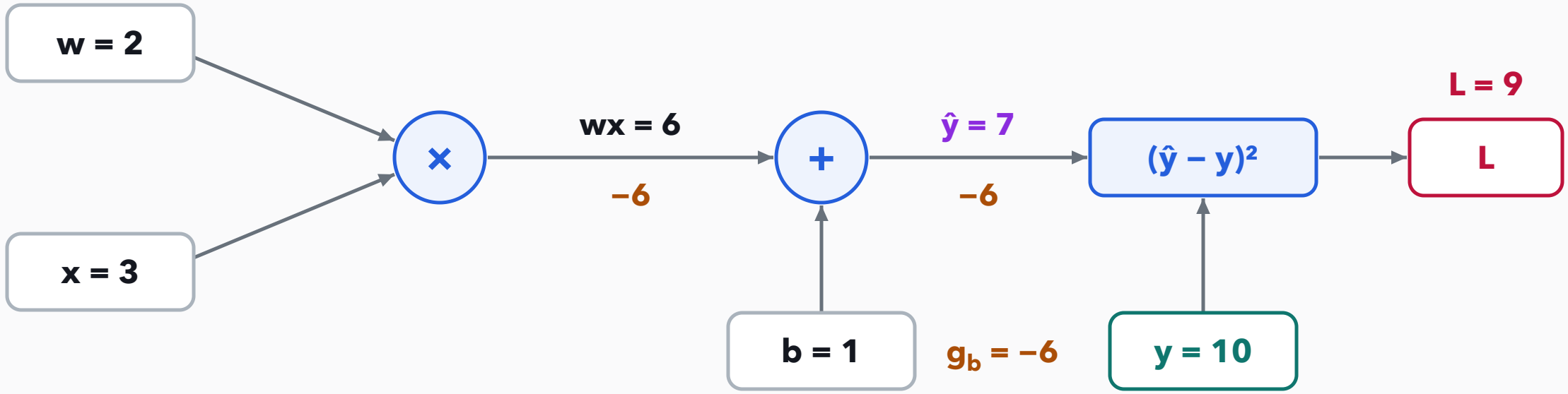
$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = 2(7 - 10) = -6$$

> Backward starts at the loss and walks right to left. The square node passes back its local slope.

Example: Lecture 6, the central graph. Numbers: scripts/sci/p2.py.

From Lecture 6

Backprop, one node at a time



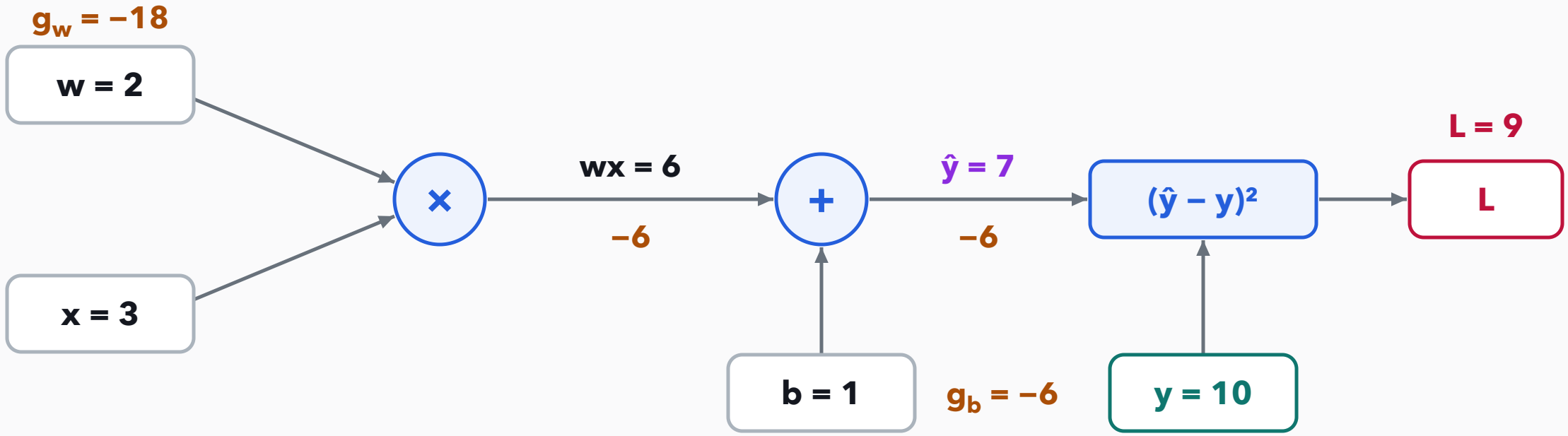
$$g_b = \underbrace{-6}_{\text{arrives}} \times \underbrace{1}_{\text{local}} = -6$$

> An add node copies the gradient that arrives to both of its inputs.

Example: Lecture 6, the central graph. Numbers: scripts/sci/p2.py.

From Lecture 6

Backprop, one node at a time



$$g_w = \underbrace{-6}_{\text{arrives}} \times \underbrace{x}_{\text{local}} = -6 \times 3 = -18$$

A multiply node scales it by the other input. The gradient on w is negative, so w should go up.

Example: Lecture 6, the central graph. Numbers: scripts/sci/p2.py.

From Lecture 6

One step, in PyTorch

```
w = torch.tensor(2., requires_grad=True)
b = torch.tensor(1., requires_grad=True)
loss = (w * 3 + b - 10) ** 2
```

	before
w	2
b	1
ŷ	7
loss	9

> The same graph in PyTorch. The forward line builds the graph and stores its values; the **target is 10**.

Example: Lecture 6. Numbers: scripts/sci/p2.py.

From Lecture 6

One step, in PyTorch

```
w = torch.tensor(2., requires_grad=True)
b = torch.tensor(1., requires_grad=True)
loss = (w * 3 + b - 10) ** 2
loss.backward()
```

	before	gradient
w	2	−18
b	1	−6
ŷ	7	
loss	9	

> backward() runs the sweep from the last slide and leaves each gradient in .grad.

Example: Lecture 6. Numbers: scripts/sci/p2.py.

From Lecture 6

One step, in PyTorch

```
w = torch.tensor(2., requires_grad=True)
b = torch.tensor(1., requires_grad=True)
loss = (w * 3 + b - 10) ** 2
loss.backward()
with torch.no_grad():
    w -= 0.01 * w.grad
    b -= 0.01 * b.grad
```

	before	gradient	after one step
w	2	−18	2.18
b	1	−6	1.06
ŷ	7		
loss	9		

> The step moves each parameter against its gradient, scaled by the step size η = 0.01.

Example: Lecture 6. Numbers: scripts/sci/p2.py.

From Lecture 6

One step, in PyTorch

```
w = torch.tensor(2., requires_grad=True)
b = torch.tensor(1., requires_grad=True)
loss = (w * 3 + b - 10) ** 2
loss.backward()
with torch.no_grad():
    w -= 0.01 * w.grad
    b -= 0.01 * b.grad
loss = (w * 3 + b - 10) ** 2
```

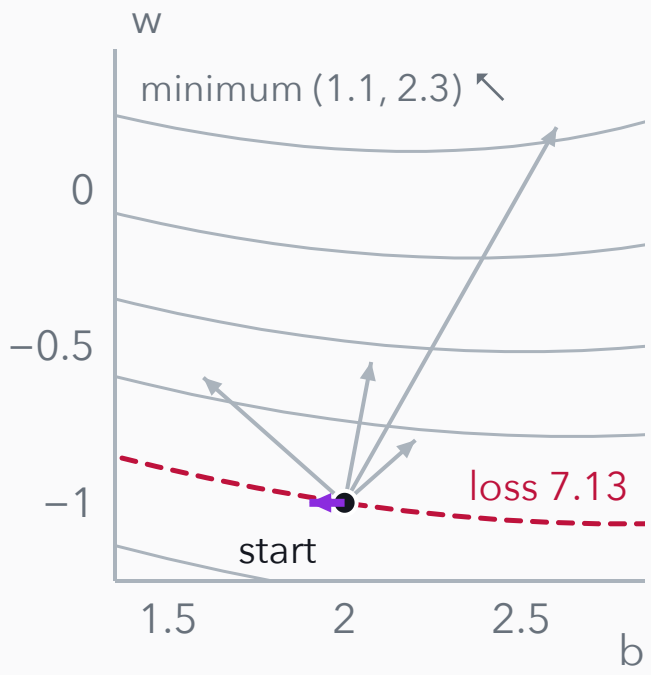
	before	gradient	after one step
w	2	−18	2.18
b	1	−6	1.06
ŷ	7		7.60
loss	9		5.76

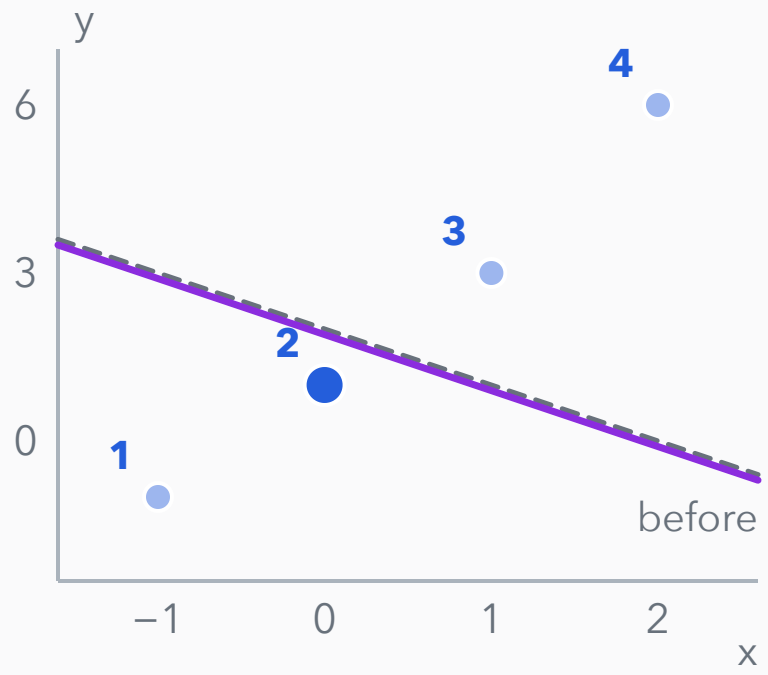
> Run the forward pass again with the new values. The prediction moved toward the target.

Example: Lecture 6. Numbers: scripts/sci/p2.py.

From Lecture 5

A gradient from part of the data





the step used
point 2 only

gradient
(1.00, 0.00)

loss on all four points
7.13 → 7.21

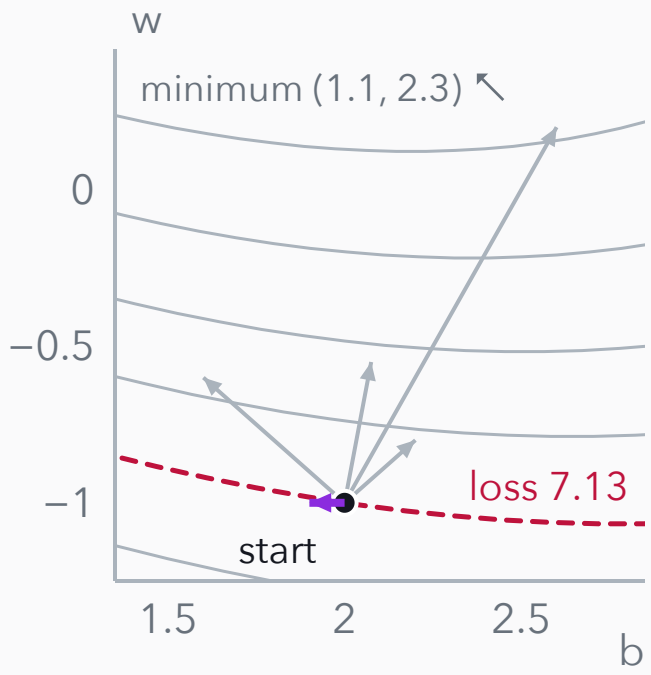
loss on point 2
0.50 → 0.41

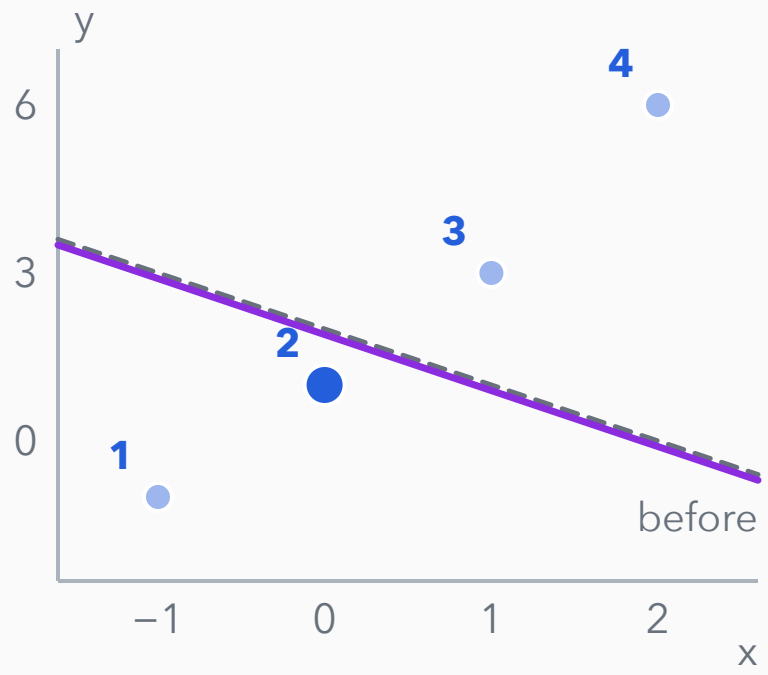
> Lecture 5's line fit, one step of size 0.1 from (b, w) = (2, −1). With all four points the loss falls. Which single point would you rather not draw?

Example A of Lecture 5. Numbers: scripts/sci/p2.py.

From Lecture 5

A gradient from part of the data





the step used
point 2 only

gradient
(1.00, 0.00)

loss on all four points
7.13 → 7.21

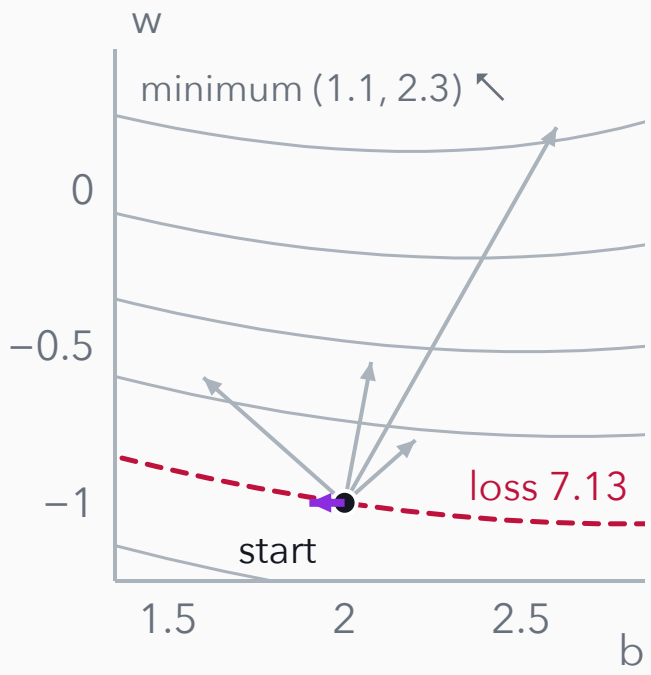
loss on point 2
0.50 → 0.41

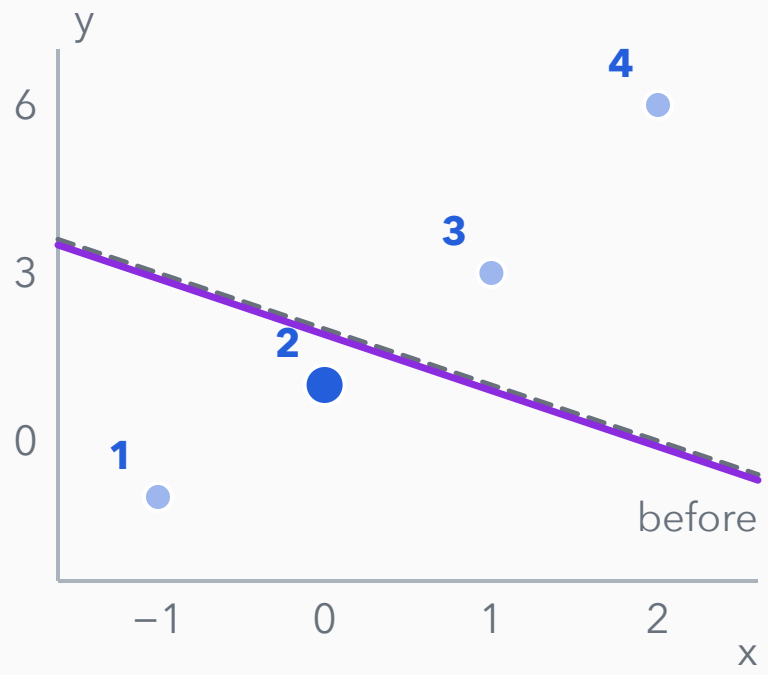
> Click point 2: its own loss falls, but the loss on all four rises. The grey arrows are the other one-point steps. Where is their average?

Example A of Lecture 5. Numbers: scripts/sci/p2.py.

From Lecture 5

A gradient from part of the data





the step used
point 2 only

gradient
(1.00, 0.00)

loss on all four points
7.13 → 7.21

loss on point 2
0.50 → 0.41

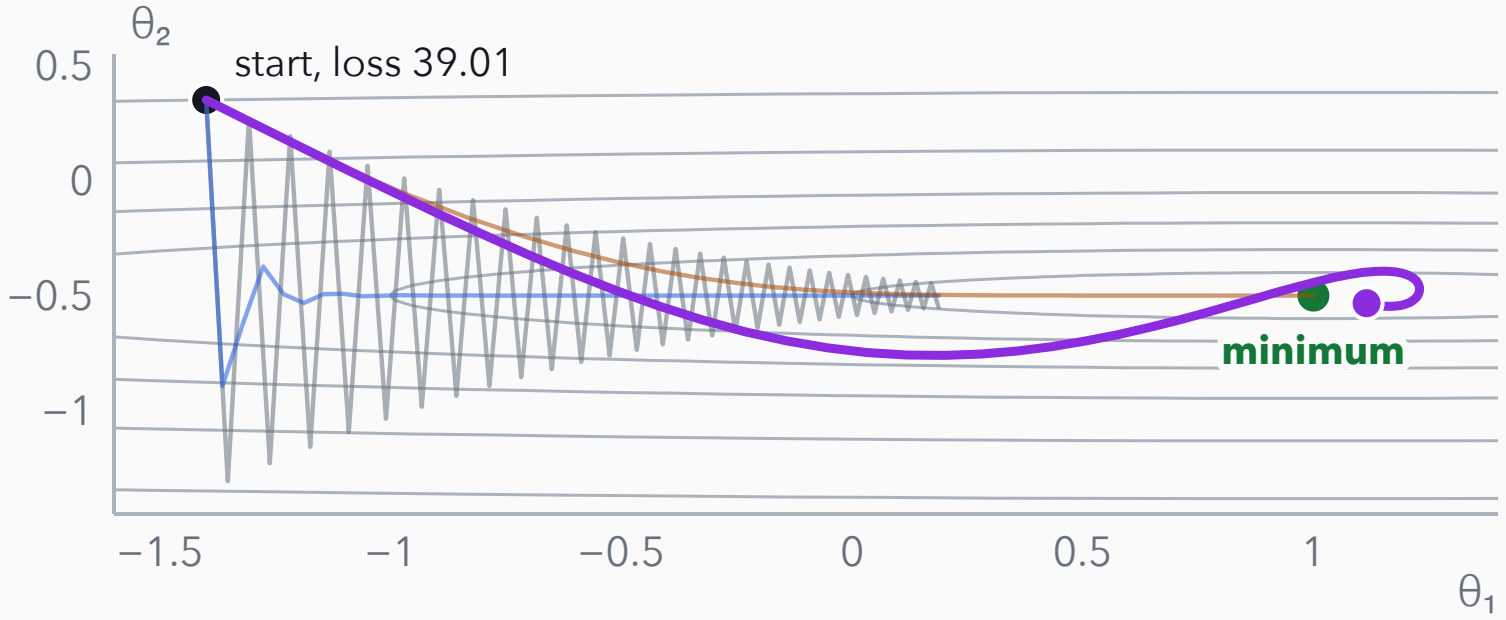
Their average is exactly the full step, (0.075, 0.450): right on average, noisy on each draw.

One draw can go uphill while the average goes downhill. A larger batch shrinks the spread.

Example A of Lecture 5. Numbers: scripts/sci/p2.py.

From Lecture 7

Same ravine, four optimizers



loss after **55 steps**

gradient descent	0.46
momentum	0.33
RMSProp	0.00
Adam	**0.06**

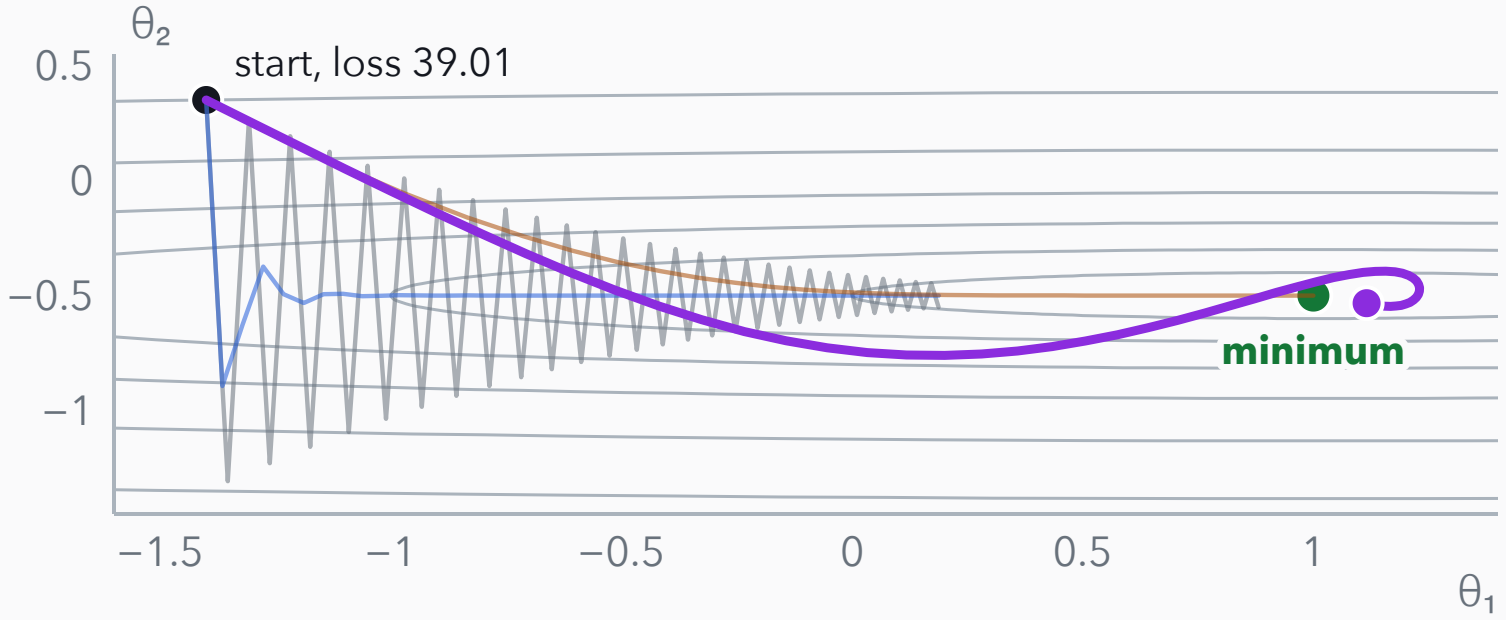
$\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$

> Lecture 7's ravine is 100 times steeper across θ_2 than along θ_1. Each method took 55 steps from the same start. Why does gradient descent zigzag?

Example B of Lectures 5 and 7, with each method's step size from Lecture 7. Numbers: scripts/sci/p2.py.

From Lecture 7

Same ravine, four optimizers



loss after **55 steps**

gradient descent	0.46
momentum	0.33
RMSProp	0.00
Adam	**0.06**

$\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$

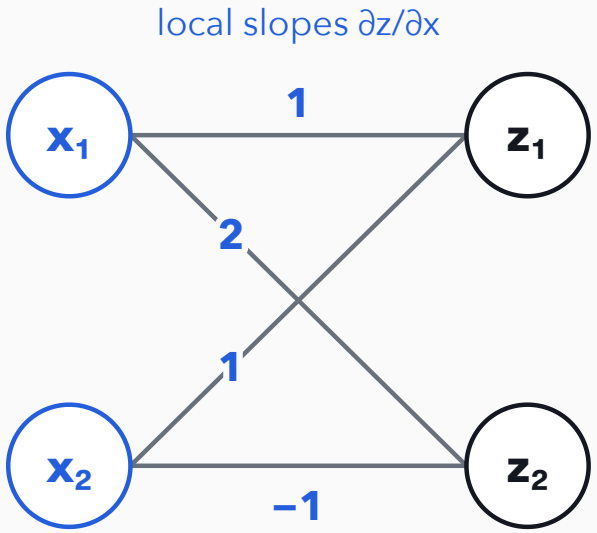
Momentum averages the pushes: across the valley they cancel. RMSProp scales each coordinate by its recent size. Adam does both.

> With a memory of past gradients, all three beat the plain step. Each uses its own step size from Lecture 7.

Example B of Lectures 5 and 7, with each method's step size from Lecture 7. Numbers: scripts/sci/p2.py.

From Lecture 8

Backprop through a layer multiplies by J^T



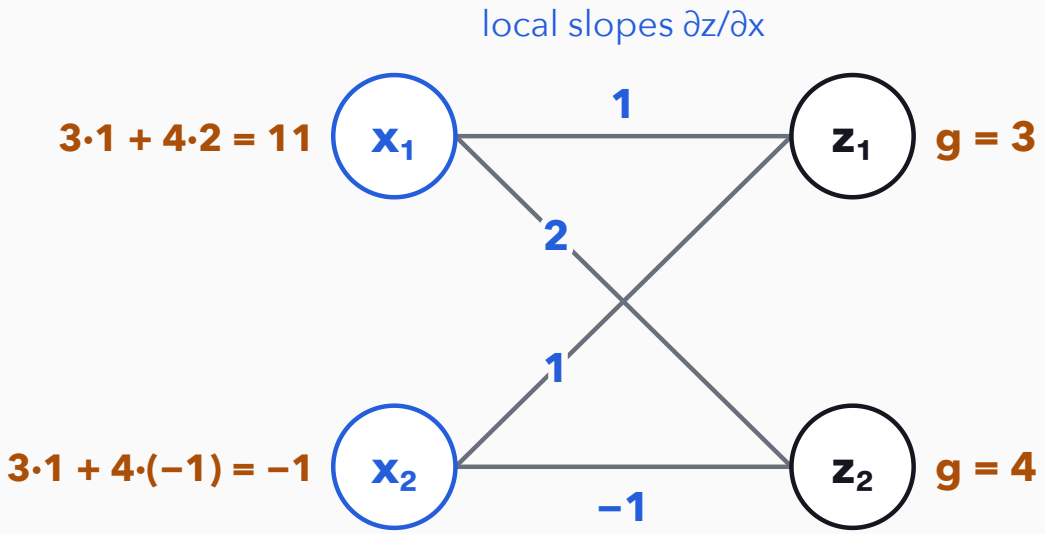
$$\begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 1 \\ 2 & -1 \end{pmatrix}}_{J} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

> Lecture 8's layer. The slopes on the arrows are the entries of the Jacobian J. If the gradient (3, 4) arrives at z, what reaches x?

Example: Lecture 8 (the autograd deck). Numbers: scripts/sci/p2.py.

From Lecture 8

Backprop through a layer multiplies by J^T



$$\begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 1 \\ 2 & -1 \end{pmatrix}}_{J} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

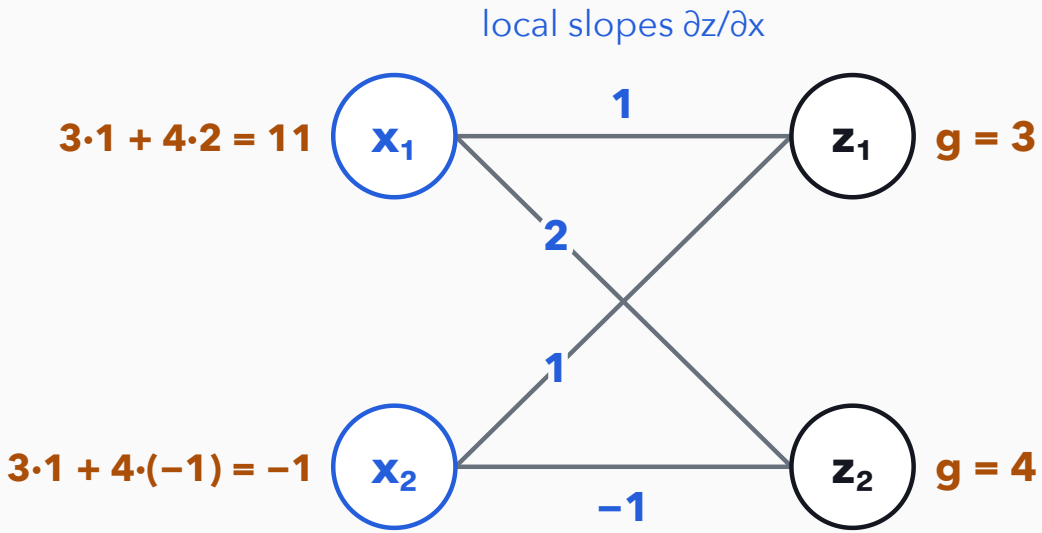
$$\nabla_x L = J^\top g = \begin{pmatrix} 1 & 2 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 11 \\ -1 \end{pmatrix}$$

> Each input adds up what every output sends back along its arrows. Written as matrices, that sum is J^T times the gradient.

Example: Lecture 8 (the autograd deck). Numbers: scripts/sci/p2.py.

From Lecture 8

Backprop through a layer multiplies by J^T



$$\begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 1 \\ 2 & -1 \end{pmatrix}}_{J} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

$$\nabla_x L = J^\top g = \begin{pmatrix} 1 & 2 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 11 \\ -1 \end{pmatrix}$$

```
x = torch.tensor([1., 2.], requires_grad=True)
z = torch.stack([x[0] + x[1], 2*x[0] - x[1]])
z.backward(torch.tensor([3., 4.]))
x.grad          # tensor([11., -1.])
```

Autograd computes $J^T g$ directly and never builds J. For a layer of 4,096 units, J would have about 1.7×10^7 entries.

Example: Lecture 8 (the autograd deck). Numbers: scripts/sci/p2.py.

From Lecture 9

Softmax overflows; the loss takes logits

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

	logits z	e^z	p
small	(2, 1, 0)	7.39, 2.72, 1.00	0.665, 0.245, 0.090

> Softmax turns logits into probabilities. For (2, 1, 0) it is easy. What should (1000, 999, 998) give?

Example: Lecture 9 (valid outputs). Computed in float32 by scripts/sci/p2.py.

From Lecture 9

Softmax overflows; the loss takes logits

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

	logits z	e^z	p
small	(2, 1, 0)	7.39, 2.72, 1.00	0.665, 0.245, 0.090
large, direct	(1000, 999, 998)	`inf`, `inf`, `inf`	`nan`, `nan`, `nan`

> Computed directly in float32, e to the 1000 overflows to inf, and inf divided by inf is nan.

Example: Lecture 9 (valid outputs). Computed in float32 by scripts/sci/p2.py.

From Lecture 9

Softmax overflows; the loss takes logits

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

	logits z	e^z	p
small	(2, 1, 0)	7.39, 2.72, 1.00	0.665, 0.245, 0.090
large, direct	(1000, 999, 998)	inf, inf, inf	nan, nan, nan
minus 1000	(0, −1, −2)	1.00, 0.37, 0.14	0.665, 0.245, 0.090

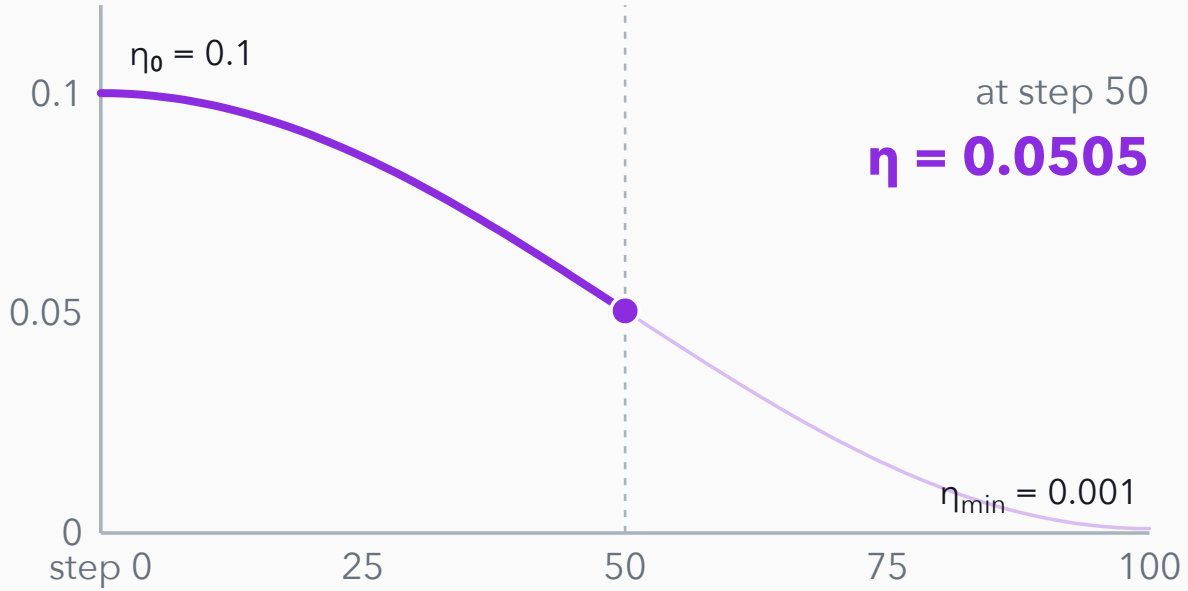
```
loss = F.cross_entropy(logits, y)   # takes logits, subtracts the max inside
```

> Subtracting the same number from every logit leaves softmax unchanged. Subtract the largest: the same three probabilities, with nothing to overflow.

Example: Lecture 9 (valid outputs). Computed in float32 by scripts/sci/p2.py.

From Lecture 9

Large steps early, small steps late



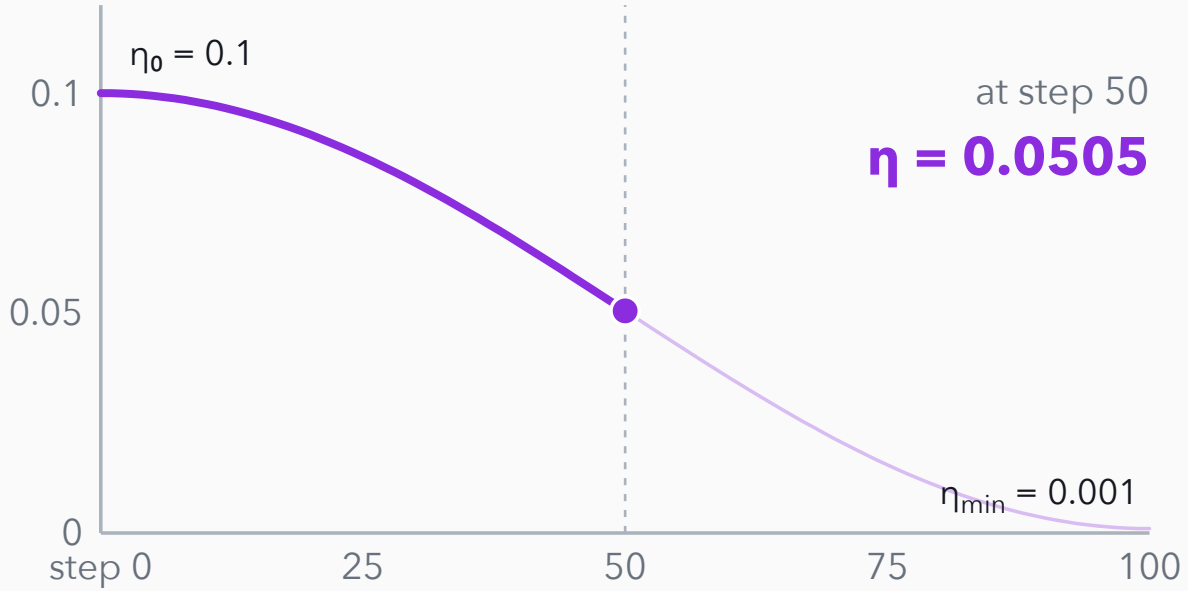
$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_0 - \eta_{\min})\left(1 + \cos\frac{\pi t}{T}\right)$$

Lecture 9's cosine schedule over T = 100 steps. Drag the step: halfway, η is 0.0505. Why would we want the step to shrink at all?

Example: Lecture 9 (learning-rate schedules). Numbers: scripts/sci/p2.py.

From Lecture 9

Large steps early, small steps late



$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_0 - \eta_{\min})\left(1 + \cos\frac{\pi t}{T}\right)$$

Near the minimum a mini-batch gradient is mostly noise, and η sets how far the noise throws θ: a noise of 0.5 moves it 0.1 at η = 0.2, 0.02 at η = 0.04.

Large steps cover ground early; small steps settle late. The schedule changes only η, so it works with any optimizer.

Example: Lecture 9 (learning-rate schedules). Numbers: scripts/sci/p2.py.

Check: the loss went up

We train Lecture 5's line fit with SGD, one point per step, $\eta = 0.1$. The first two lines it prints are on the right. Is the code broken?

```
step 0   loss 7.13
step 1   loss 7.21
```

> Two ideas from this part answer it. Talk to your neighbour for a minute.

Example A of Lecture 5. Numbers: scripts/sci/p2.py.

Check: the loss went up

We train Lecture 5's line fit with SGD, one point per step, $\eta = 0.1$. The first two lines it prints are on the right. Is the code broken?

```
step 0   loss 7.13
step 1   loss 7.21
```

Not necessarily. That step used point 2 alone: its own loss fell, 0.50 to 0.41, while points 3 and 4 got worse. Judge the trend over many steps; if it keeps rising, lower η.

> One noisy step is not a trend.

Example A of Lecture 5. Numbers: scripts/sci/p2.py.

Part 2 in one slide

1. Backprop gives every gradient in one sweep; a layer multiplies by J^T.
2. A mini-batch gradient is cheap, right on average, noisy per draw.
3. Momentum and Adam shape each step; a schedule shrinks it.

MLP classifier

Data	pairs (x, y): an input and its class
Model	an MLP gives scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	backprop on a mini-batch; Adam and a schedule set the step
At test time	the class with the largest p

Next: Part 3, where a low training loss turns out to be the wrong thing to celebrate.

Part 3 · Doing well on data you have not seen

Lecture 10 asked what happens when a model fits its training data too well. It changed three rows of the recipe: Data, Loss and At test time.

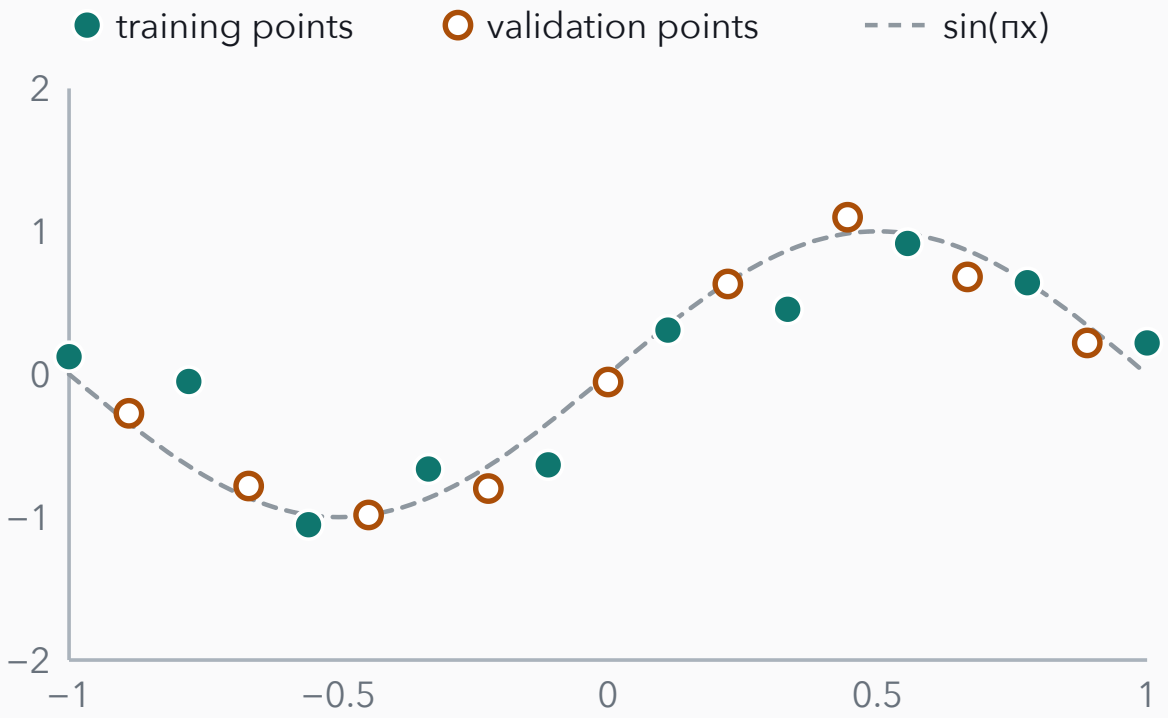
1. measure the error on points the fit never saw
2. add a penalty on large weights to the loss
3. add noise to training: dropout, smoothed targets, crops and flips
4. let validation choose; read the test set once

Lecture 10

> By the end you can say why training error is the wrong judge, and which split chooses the strength of a regulariser.

From Lecture 10

Ten points, ten weights



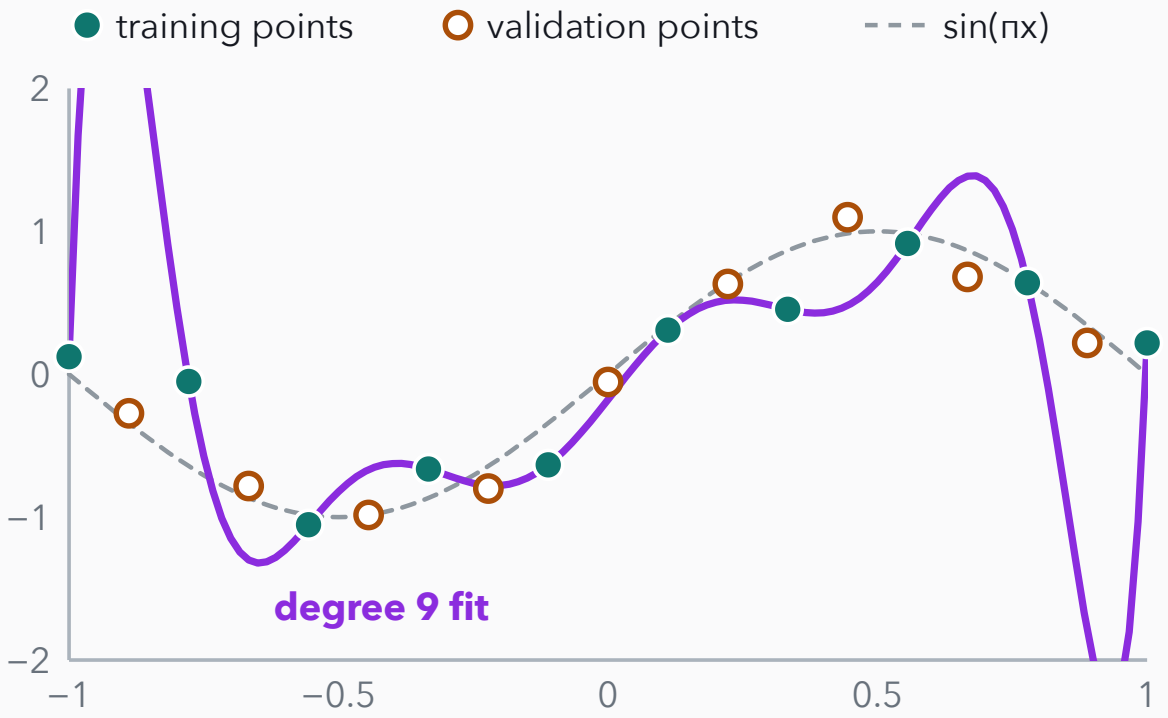
$$\hat{y} = w_0 + w_1 x + \cdots + w_9 x^9$$

> Ten points from sin(πx) plus noise of size 0.25, and nine validation points at the midpoints. A degree 9 polynomial has ten weights. What will its training error be?

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

From Lecture 10

Ten points, ten weights



$$\hat{y} = w_0 + w_1 x + \cdots + w_9 x^9$$

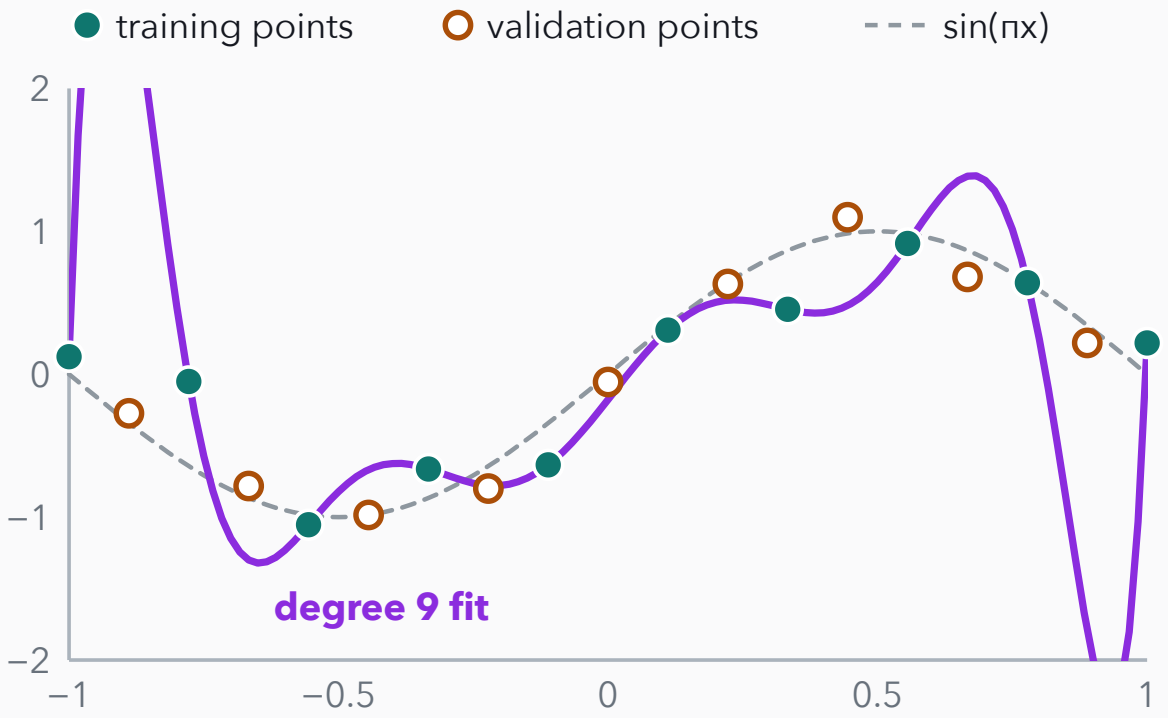
$$\text{training error} = 0.00$$

With ten weights and ten points it passes through every training point, so the training error is 0.00.

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

From Lecture 10

Ten points, ten weights



$$\hat{y} = w_0 + w_1 x + \cdots + w_9 x^9$$

$$\text{training error} = 0.00$$

$$\text{validation error} = 1.70$$

$$\text{size of the weights } \|w\| = 465$$

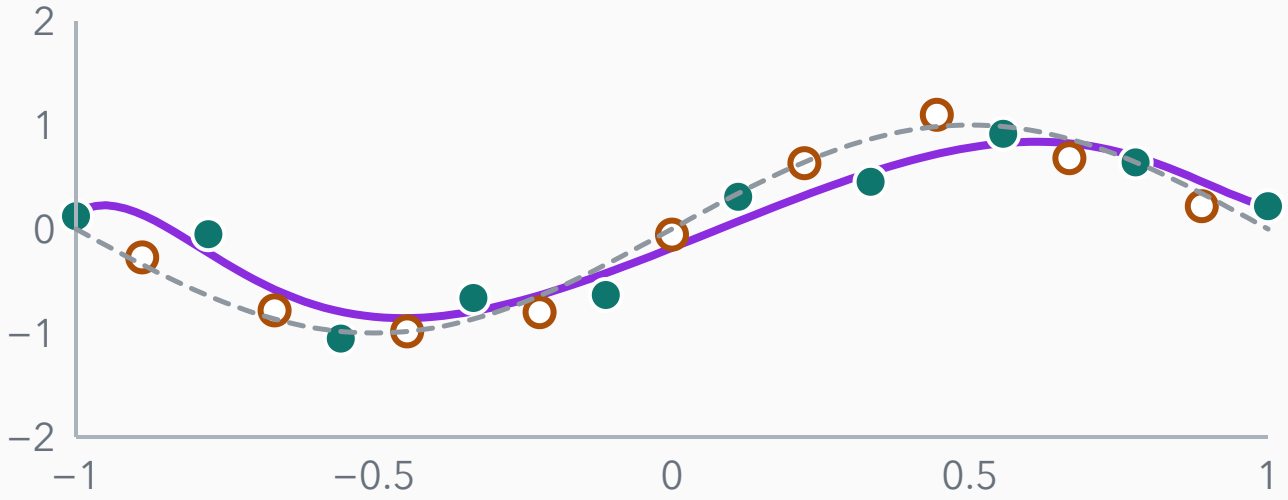
> Between the points it swings far from sin(πx): the validation error is 1.70, about 27 times the noise variance 0.0625.

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

From Lecture 10

Make large weights cost something

$$\mathcal{J}(w) = \frac{1}{2n}\sum_i (\hat{y}_i - y_i)^2 + \frac{\lambda}{2}\|w\|^2$$



λ	training	validation	‖w‖
0	0.000	1.704	465.4
10^{-6}	0.014	0.151	35.1
10^{-5}	0.016	0.103	9.1
10^{-4}	0.018	0.077	5.6
10^{-3}	0.023	0.065	3.9
10^{-2}	0.062	0.111	1.8
10^{-1}	0.147	0.226	0.8
1	0.286	0.405	0.2

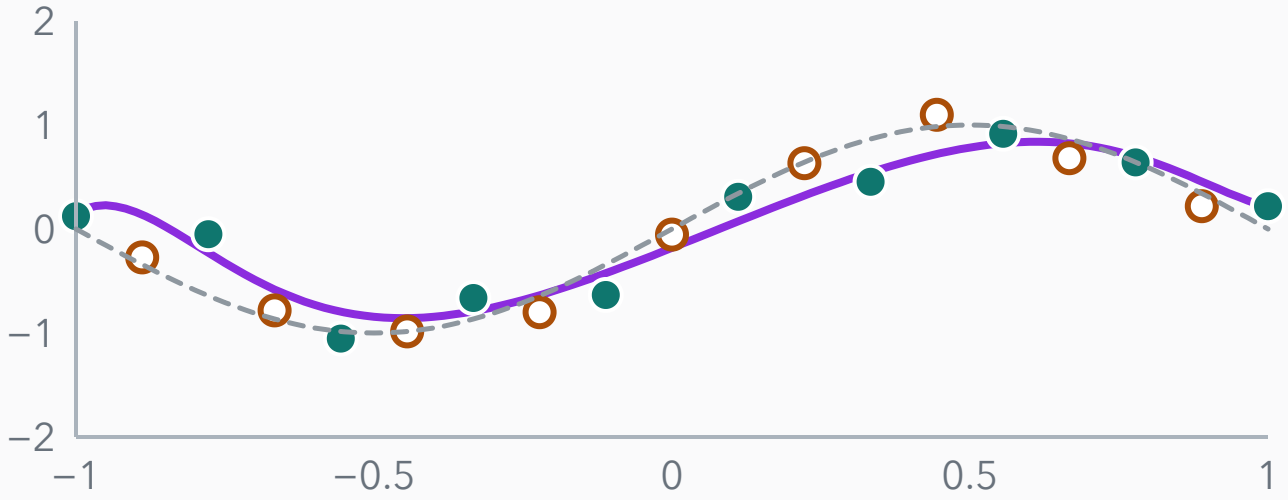
The loss gets a second term that grows with the weights. Drag λ from 0 upwards. Which column of the table should choose λ?

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

From Lecture 10

Make large weights cost something

$$\mathcal{J}(w) = \frac{1}{2n}\sum_i (\hat{y}_i - y_i)^2 + \frac{\lambda}{2}\|w\|^2$$



λ	training	validation	‖w‖
0	0.000	1.704	465.4
10^{-6}	0.014	0.151	35.1
10^{-5}	0.016	0.103	9.1
10^{-4}	0.018	0.077	5.6
10^{-3}	0.023	0.065	3.9
10^{-2}	0.062	0.111	1.8
10^{-1}	0.147	0.226	0.8
1	0.286	0.405	0.2

Validation picks λ = 10^{-3}: error 0.065, with ‖w‖ = 3.9. Training error is always lowest at λ = 0.

Too little λ leaves the swings; too much flattens the curve. Only validation sees both mistakes.

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

From Lecture 10

Four ways to prefer a simpler fit

Weight decay Loss

$$\theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\, g$$

θ = 5, g = 3, η = 0.1, λ = 0.2: $0.98 \times 5 - 0.30 = 4.60$

Dropout hidden units

Label smoothing Loss, its target

Crop and flip Data

> Weight decay is the penalty of the last slide, seen in the update: shrink θ first, then take the usual step.

Worked numbers as in Lecture 10. The crop is the lecture's v2.RandomCrop(32, padding=4) with a horizontal flip. Numbers: scripts/sci/p3.py.

From Lecture 10

Four ways to prefer a simpler fit

Weight decay Loss

$$\theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\, g$$

θ = 5, g = 3, η = 0.1, λ = 0.2: $0.98 \times 5 - 0.30 = 4.60$

Dropout hidden units

$$\tilde{h}_i = \frac{m_i}{q}\, h_i$$

Keep rate q = 0.5:
$(2,\ 4,\ 6,\ 8) \odot (1,\ 0,\ 1,\ 0)/0.5 = (4,\ 0,\ 12,\ 0)$

Label smoothing Loss, its target

Crop and flip Data

> Dropout zeroes hidden units at random in training and divides the survivors by the keep rate q, so the average is unchanged.

Worked numbers as in Lecture 10. The crop is the lecture's v2.RandomCrop(32, padding=4) with a horizontal flip. Numbers: scripts/sci/p3.py.

From Lecture 10

Four ways to prefer a simpler fit

Weight decay Loss

$$\theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\, g$$

θ = 5, g = 3, η = 0.1, λ = 0.2: $0.98 \times 5 - 0.30 = 4.60$

Dropout hidden units

$$\tilde{h}_i = \frac{m_i}{q}\, h_i$$

Keep rate q = 0.5:
$(2,\ 4,\ 6,\ 8) \odot (1,\ 0,\ 1,\ 0)/0.5 = (4,\ 0,\ 12,\ 0)$

Label smoothing Loss, its target

$$\tilde{y} = (1 - \epsilon)\, y + \epsilon/K$$

K = 5, ε = 0.1, right class b:
$\tilde{y} = (0.02,\ 0.92,\ 0.02,\ 0.02,\ 0.02)$

Crop and flip Data

> Label smoothing moves a little of the target to the wrong classes, so the loss stops pushing once p_b reaches 0.92.

Worked numbers as in Lecture 10. The crop is the lecture's v2.RandomCrop(32, padding=4) with a horizontal flip. Numbers: scripts/sci/p3.py.

From Lecture 10

Four ways to prefer a simpler fit

Weight decay Loss

$$\theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\, g$$

θ = 5, g = 3, η = 0.1, λ = 0.2: $0.98 \times 5 - 0.30 = 4.60$

Dropout hidden units

$$\tilde{h}_i = \frac{m_i}{q}\, h_i$$

Keep rate q = 0.5:
$(2,\ 4,\ 6,\ 8) \odot (1,\ 0,\ 1,\ 0)/0.5 = (4,\ 0,\ 12,\ 0)$

Label smoothing Loss, its target

$$\tilde{y} = (1 - \epsilon)\, y + \epsilon/K$$

K = 5, ε = 0.1, right class b:
$\tilde{y} = (0.02,\ 0.92,\ 0.02,\ 0.02,\ 0.02)$

Crop and flip Data

$$32 \times 32 \text{ image, padded by } 4$$

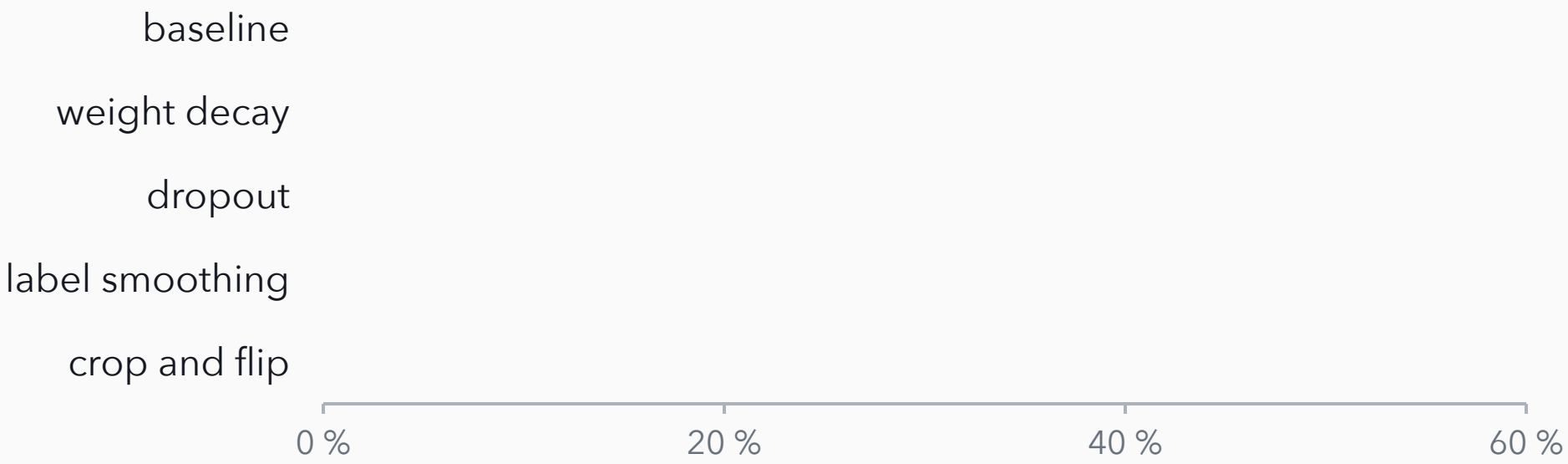
9×9 crop positions, $\times\, 2$ flips: 162 views, one label.

> Augmentation changes the Data row: each epoch sees a fresh crop and flip of every image.

Worked numbers as in Lecture 10. The crop is the lecture's v2.RandomCrop(32, padding=4) with a horizontal flip. Numbers: scripts/sci/p3.py.

From Lecture 10

One change at a time, on CIFAR-10



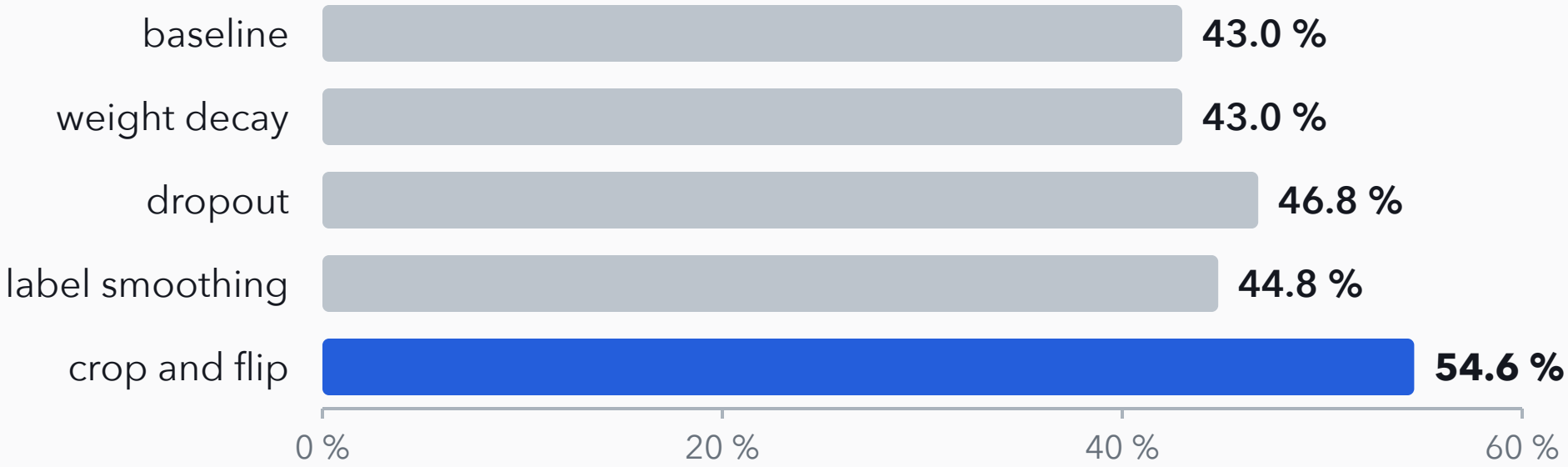
Test accuracy. 2,000 training images, the same CNN and 35 epochs in every run.

> Lecture 10 trained one small CNN five times, changing one thing each time. Which change do you expect to help most?

Recorded: Lecture 10, “Controlled CIFAR-10 ablations” (lecture7/diagrams/cifar_regularization_experiment.py, one seed), not retrained.

From Lecture 10

One change at a time, on CIFAR-10



Test accuracy. 2,000 training images, the same CNN and 35 epochs in every run.

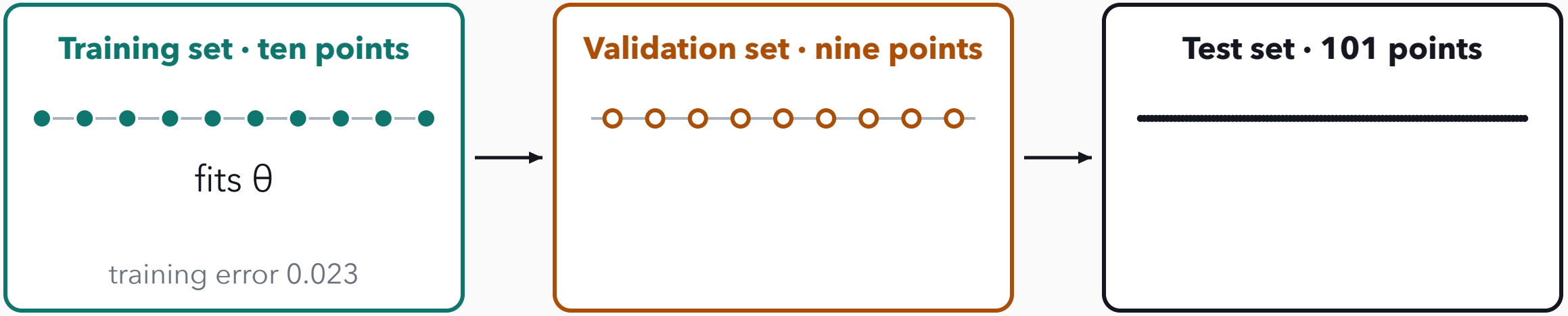
Crop and flip: 54.6 %, 11.6 points above the baseline. Weight decay (λ = 0.01) changed nothing.

> With only 2,000 images, new views of the data did more than any change to the loss or the network.

Recorded: Lecture 10, "Controlled CIFAR-10 ablations" (lecture7/diagrams/cifar_regularization_experiment.py, one seed), not retrained.

From Lecture 10

Train fits, validation chooses, test reports



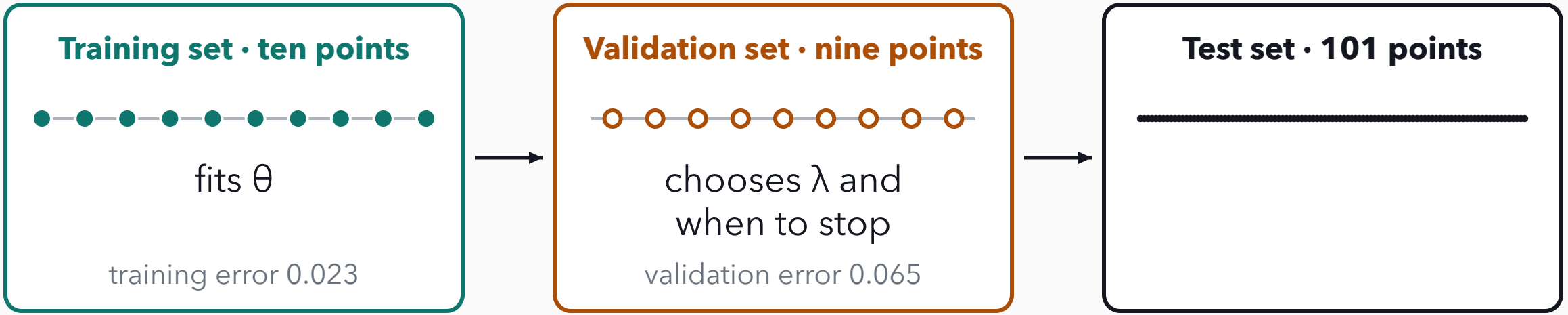
The ten-point lab, with the fit at $\lambda = 10^{-3}$. Each dot is one input x on the interval from −1 to 1.

> The training set has one job: gradient steps on it fit the weights θ.

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

From Lecture 10

Train fits, validation chooses, test reports



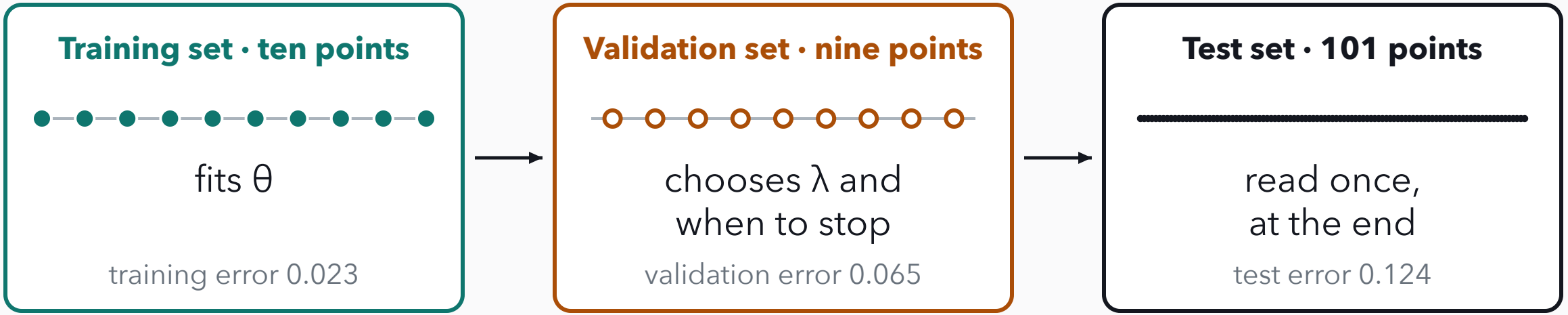
The ten-point lab, with the fit at $\lambda = 10^{-3}$. Each dot is one input x on the interval from −1 to 1.

> Every choice we make (λ, when to stop, which regulariser) is made on validation. Here it picked $\lambda = 10^{-3}$.

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

From Lecture 10

Train fits, validation chooses, test reports



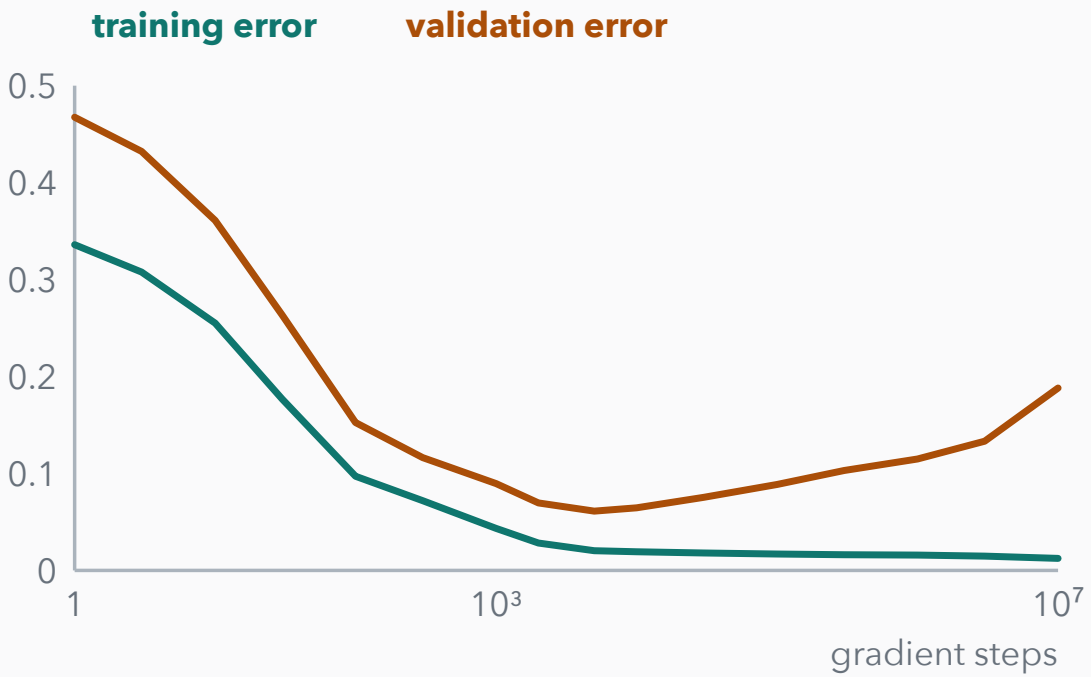
The ten-point lab, with the fit at $\lambda = 10^{-3}$. Each dot is one input x on the interval from −1 to 1.

> The test set is read once, at the end, for the report. If it changes a knob, it has become a second validation set.

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

From Lecture 10

Check: the curves part ways



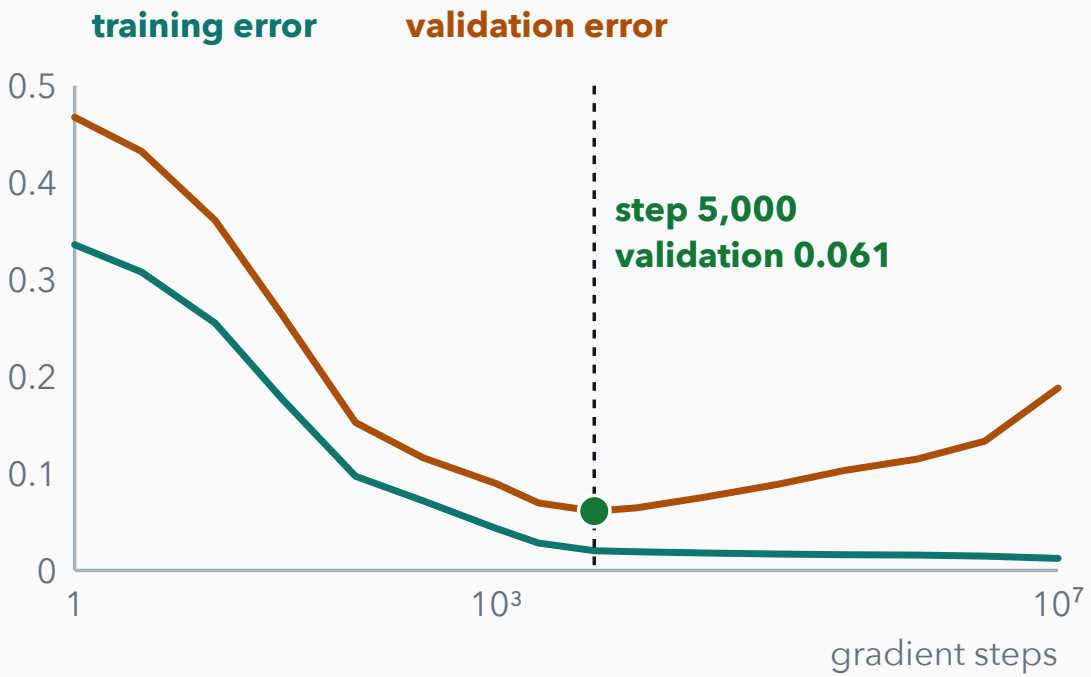
Training error keeps falling while validation error has started to rise. What do you do?

> Gradient descent on the ten-point lab, from w = 0. Training error falls from 0.336 to 0.012; validation error turns up after step 5,000.

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

From Lecture 10

Check: the curves part ways



Training error keeps falling while validation error has started to rise. What do you do?

Stop, and restore the step with the lowest validation error: step 5,000, 0.061. To do better, add a regulariser and let validation choose its strength.

Restoring step 5,000 gives test error 0.125, as good as the penalty's 0.124. Both choices were made on validation.

The ten-point lab of Lecture 10 (seed 21), ported from its deck. Numbers: scripts/sci/p3.py.

Part 3 in one slide

1. Training error is the wrong judge: 0.00 on the ten points, 1.70 between them.
2. A penalty, dropout, smoothed targets and new views add a preference the data did not give.
3. Validation chose $\lambda = 10^{-3}$ and when to stop; the test set was read once.

MLP classifier, regularised

Data	three splits; crops and flips add views
Model	an MLP gives scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\sum_k \tilde{y}_k \ln p_k + \frac{\lambda}{2}\|\theta\|^2$
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	dropout off; the class with the largest p; test read once

Next: the model has been an MLP. Language needs one that can look back at earlier tokens.

Part 4 · Predicting the next token

This part kept one task, predicting the next token, and changed how the model gathers the context it predicts from.

1. Text becomes tokens; each position predicts the next one.
2. Self-attention: each token mixes the values of the others.
3. Mask the future, add heads, stack blocks.
4. Generate one token at a time.

Lectures 11 to 15

> By the end you can say how a token reads the others, and why training hides the future.

From Lecture 11

One name, six guesses, a loss for each

context		target
- - -	→	**a**
- - a	→	**a**
- a a	→	**b**
a a b	→	**i**
a b i	→	**d**
b i d	→	**-**

p(next character | a a b), the eight largest



> The name aabid, read three characters at a time. Each window gives one training pair. Given **a a b**, what should the model output?

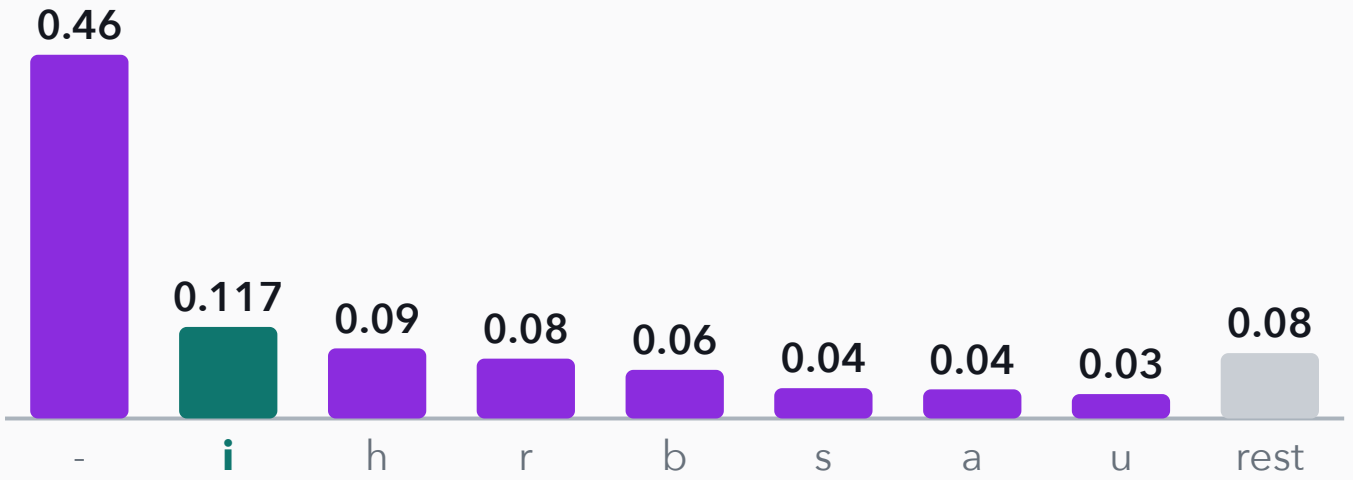
The character model is the one Lecture 11 trained on Indian names (part1.html). Numbers: scripts/sci/p4.py.

From Lecture 11

One name, six guesses, a loss for each

context		target
- - -	→	**a**
- - a	→	**a**
- a a	→	**b**
a a b	→	**i**
a b i	→	**d**
b i d	→	**-**

p(next character | a a b), the eight largest



A probability for each of the 27 characters. The target **i** gets **0.117**; the boundary -, which ends the name, gets 0.46.

The character model is the one Lecture 11 trained on Indian names (part1.html). Numbers: scripts/sci/p4.py.

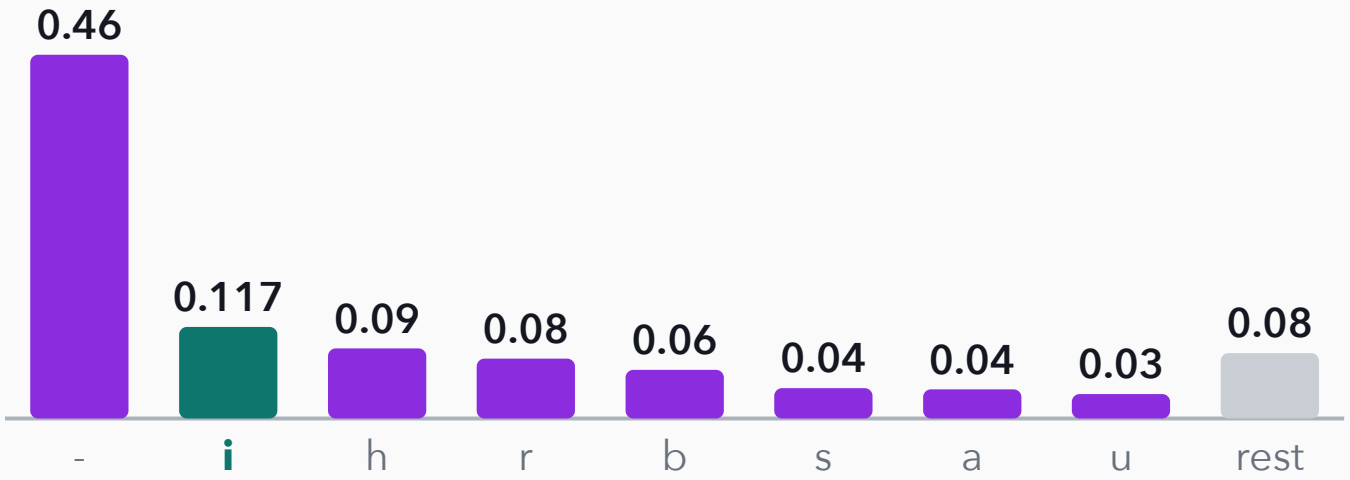
From Lecture 11

One name, six guesses, a loss for each

context		target
- - -	→	**a**
- - a	→	**a**
- a a	→	**b**
a a b	→	**i**
a b i	→	**d**
b i d	→	**-**

$$\text{loss} = -\ln p(\text{i} \mid \text{aab})$$
$$= -\ln 0.117 = 2.15$$

p(next character | a a b), the eight largest



> The loss is the negative log of the target's probability, in nats. Each of the six pairs adds one such term.

The character model is the one Lecture 11 trained on Indian names (part1.html). Numbers: scripts/sci/p4.py.

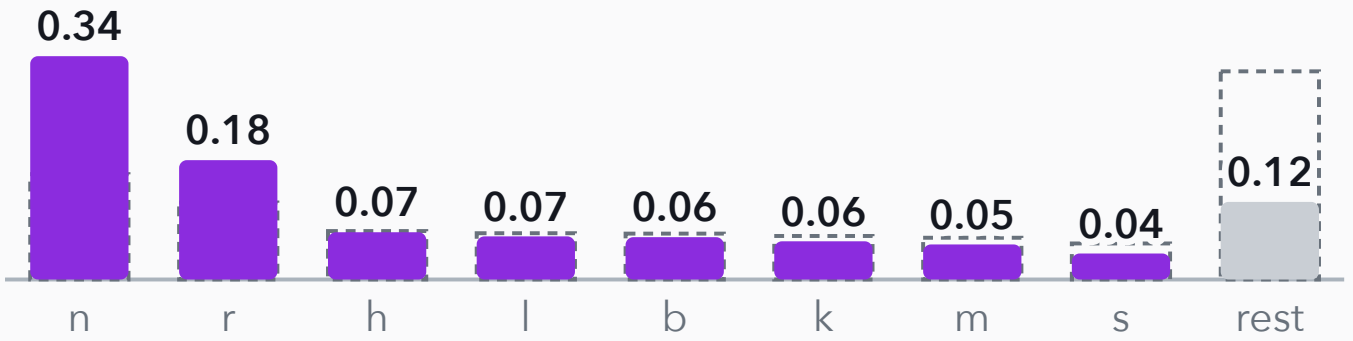
From Lecture 11

One name, six guesses, a loss for each

context		target
- - -	→	a
- - a	→	a
- a a	→	b
a a b	→	**i**
a b i	→	d
b i d	→	-

$$\text{loss} = -\ln p(\text{i} \mid \text{aab})$$
$$= -\ln 0.117 = 2.15$$

p(next | - s a) at T = 0.50: n gets 0.34



At test time there is no target, so we sample. The model starts a new name with - s a. Drag the temperature T.

The character model is the one Lecture 11 trained on Indian names (part1.html). Numbers: scripts/sci/p4.py.

From Lecture 12

One word, two sentences

Sentence A The fisherman sat beside the river **bank** and watched the ___

Sentence B She deposited the cheque at the **bank** and watched the ___

lookup row	water	finance	person	glue
$E_{\text{tok}}[\text{bank}]$	0.70	0.70	0.00	0.70

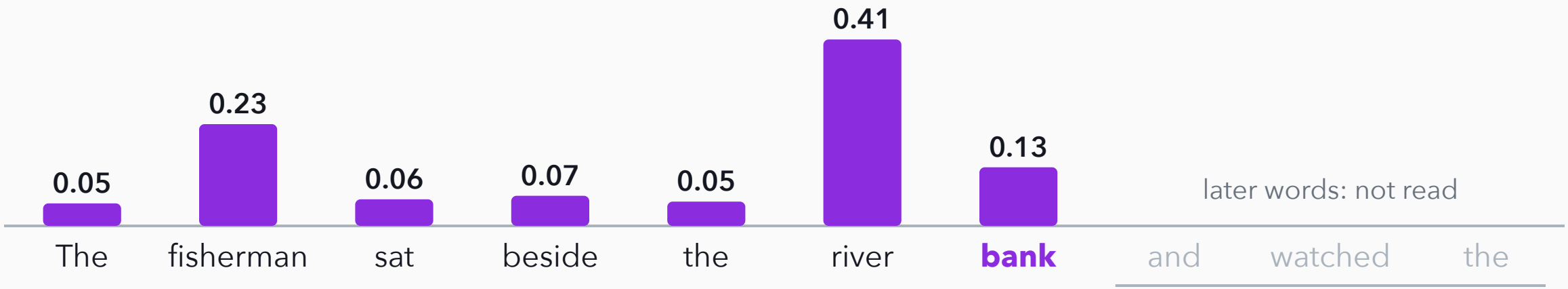
> Both sentences give bank the same lookup row. How can the model tell a river bank from a bank that takes cheques?

The bank model is the hand-built one of Lectures 12 and 13 (attention.html). Numbers: scripts/sci/p4.py.

From Lecture 12

One word, two sentences

Sentence A: the weights bank puts on each word it can read (they sum to 1)



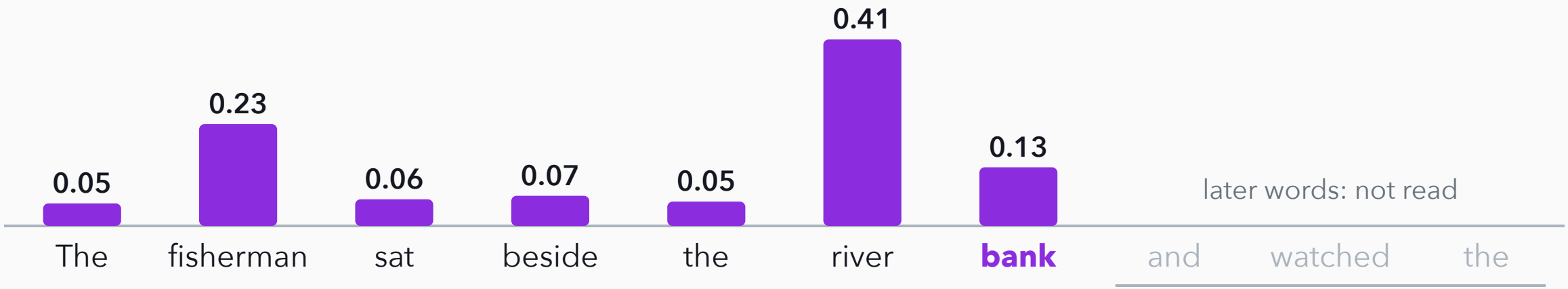
> Bank's query is matched with every key up to itself. In A, river gets 0.41 and fisherman 0.23. Switch to B.

The bank model is the hand-built one of Lectures 12 and 13 (attention.html). Numbers: scripts/sci/p4.py.

From Lecture 12

One word, two sentences

Sentence A: the weights bank puts on each word it can read (they sum to 1)



Bank's message mixes the values of the words it reads: $m_7 = \sum_{j \le 7} \alpha_{7j}\, v_j$. Same lookup row, two mixtures.

> So bank's new row moves toward water in A and toward money in B. Lecture 12 chose the matrices by hand to show this.

The bank model is the hand-built one of Lectures 12 and 13 (attention.html). Numbers: scripts/sci/p4.py.

From Lecture 14

One attention layer, a piece per click

```
S = Q @ K.T                # scores
```

$$S = QK^\top, \qquad S_{ij} = q_i \cdot k_j$$

$$Q = EW_Q, \quad K = EW_K, \quad V = EW_V$$

One row of S per token, one column per token it might read.

> Stack the token rows into E. One product scores every query against every key.

The lecture writes the same steps as two lines: A = (Q @ K.T / d_k**0.5 + mask).softmax(dim=-1) and E_new = E + (A @ V) @ W_O. Numbers: scripts/sci/p4.py.

From Lecture 14

One attention layer, a piece per click

```
S = Q @ K.T              # scores
S = S / d_k ** 0.5       # scale
```

$$\mathrm{softmax}([-8,\ 8]) = [1.13 \times 10^{-7},\ 1.00]$$

$$\mathrm{softmax}\left([-8,\ 8]/\sqrt{64}\right) = [0.12,\ 0.88]$$

With $d_k = 64$, raw scores spread about ±8. Divided, they spread about ±1.

> Divide by $\sqrt{d_k}$. Unscaled, the weaker key gets 1.13×10^{-7}; scaled, it gets 0.12. The order is kept.

The lecture writes the same steps as two lines: A = (Q @ K.T / d_k**0.5 + mask).softmax(dim=-1) and E_new = E + (A @ V) @ W_O. Numbers: scripts/sci/p4.py.

From Lecture 14

One attention layer, a piece per click

```
S = Q @ K.T              # scores
S = S / d_k ** 0.5       # scale
S = S + mask             # hide the future
```

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$[1,\ 2,\ 3,\ 4] + [0,\ 0,\ -\infty,\ -\infty]$$

$$\xrightarrow{\text{softmax}} [0.27,\ 0.73,\ 0,\ 0]$$

> The mask adds −∞ wherever j > i, so those weights come out exactly 0 and the rest still sum to 1.

The lecture writes the same steps as two lines: A = (Q @ K.T / d_k**0.5 + mask).softmax(dim=-1) and E_new = E + (A @ V) @ W_O. Numbers: scripts/sci/p4.py.

From Lecture 14

One attention layer, a piece per click

```
S = Q @ K.T              # scores
S = S / d_k ** 0.5       # scale
S = S + mask             # hide the future
A = S.softmax(dim=-1)    # weights
H = A @ V                # mix the values
```

$$m_i = \sum_{j \le i} A_{ij}\, v_j$$

$$m_7 = 0.41\, v_{\text{river}} + 0.23\, v_{\text{fisherman}} + \cdots = [1.89,\ 0.07]$$

Row 7 of A holds the bars of the last slide.

> The softmax makes each row weights that sum to 1. They mix the values into a message m_i of two numbers.

The lecture writes the same steps as two lines: A = (Q @ K.T / d_k**0.5 + mask).softmax(dim=-1) and E_new = E + (A @ V) @ W_O. Numbers: scripts/sci/p4.py.

From Lecture 14

One attention layer, a piece per click

```
S = Q @ K.T              # scores
S = S / d_k ** 0.5       # scale
S = S + mask             # hide the future
A = S.softmax(dim=-1)    # weights
H = A @ V                # mix the values
E_new = E + H @ W_O      # project, add
```

	water	finance	person	glue
e_7	**0.70**	0.70	0.10	0.80
$\Delta e_7 = m_7 W_O$	**1.51**	−0.30	0.58	0.18
e'_7	**2.21**	0.40	0.68	0.98

$$E' = E + \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V\, W_O$$

> W_O turns the message into an update of width 4, and the residual adds it. Bank's water coordinate goes from 0.70 to 2.21.

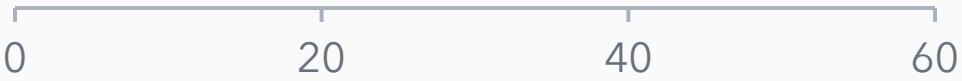
The lecture writes the same steps as two lines: A = (Q @ K.T / d_k**0.5 + mask).softmax(dim=-1) and E_new = E + (A @ V) @ W_O. Numbers: scripts/sci/p4.py.

From Lecture 14

Perplexity on stories the models never saw

$$\text{perplexity} = \exp\left(\text{mean of } -\ln p(\text{true next token})\right)$$

model	parameters	perplexity on held-out stories (lower is better)
Fixed-window MLP	2,332,832	
One-head attention	1,321,120	
Four-head attention	1,321,120	



If every true next token got probability 0.25, the perplexity would be 4: as unsure as a choice among four. Which model scores lowest?

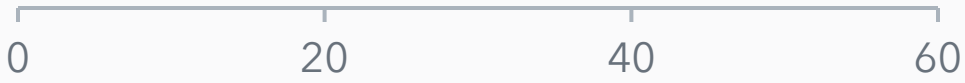
TinyStories, 4000 tokens, context 64, mean of three seeds; recorded from Lecture 14 (part3.html, capstone-evidence.json), not retrained.

From Lecture 14

Perplexity on stories the models never saw

$$\text{perplexity} = \exp\left(\text{mean of } -\ln p(\text{true next token})\right)$$

model	parameters	perplexity on held-out stories (lower is better)
Fixed-window MLP	2,332,832	51.59
One-head attention	1,321,120	31.34
Four-head attention	1,321,120	28.78



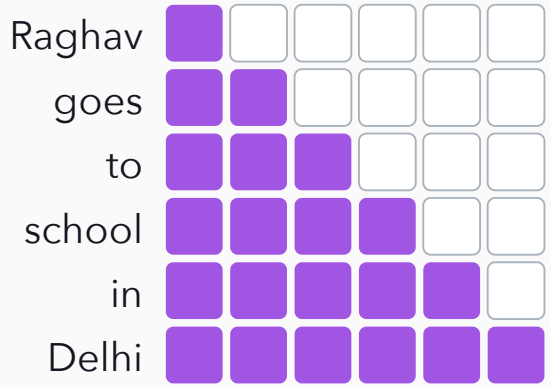
Four heads: 28.78, with 1,321,120 parameters against the MLP's 2,332,832. Attention reads further for fewer weights.

TinyStories, 4000 tokens, context 64, mean of three seeds; recorded from Lecture 14 (part3.html, capstone-evidence.json), not retrained.

From Lecture 15

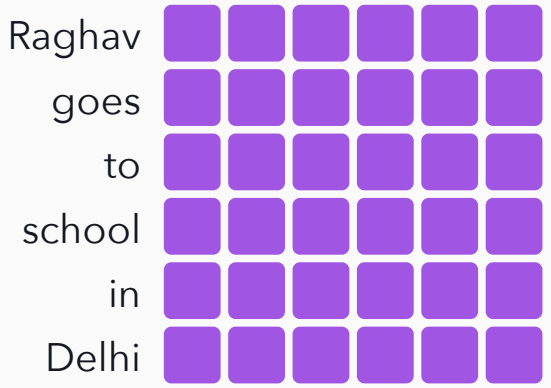
One block, three families

Decoder-only



each word reads the ones before it

Encoder



every word reads every word

Encoder-decoder



each Hindi word reads every English word

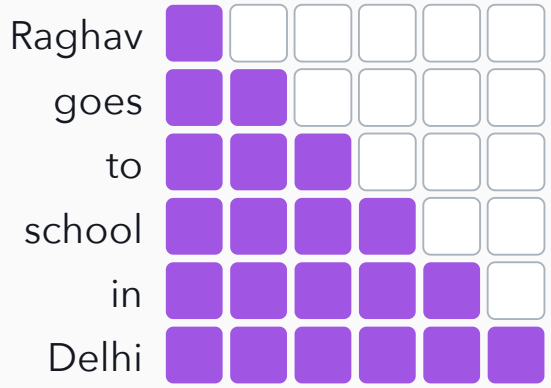
> One sentence, “Raghav goes to school in Delhi”. A filled cell lets that row read that column. Which family can write text one word at a time?

The sentence and its Hindi translation are Lecture 15’s running example (data/p4.json).

From Lecture 15

One block, three families

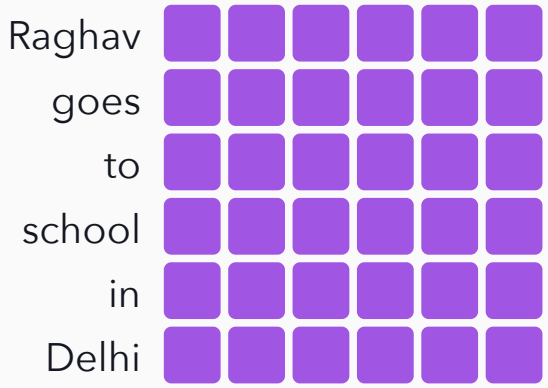
Decoder-only



each word reads the ones before it

Predicts: the next word

Encoder



every word reads every word

Predicts: a label per word (Delhi: place)

Encoder-decoder



each Hindi word reads every English word

Predicts: the next Hindi word

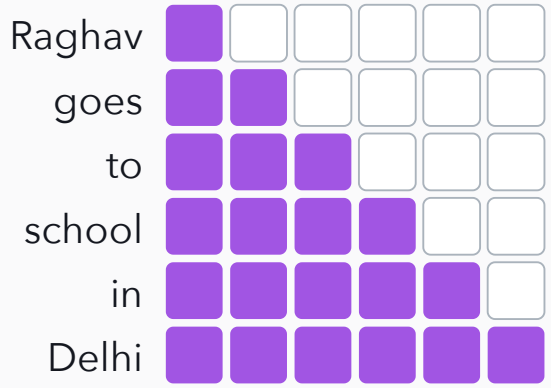
> The training target follows from the mask: next words where the future is hidden, labels where every word is seen.

The sentence and its Hindi translation are Lecture 15's running example (data/p4.json).

From Lecture 15

One block, three families

Decoder-only

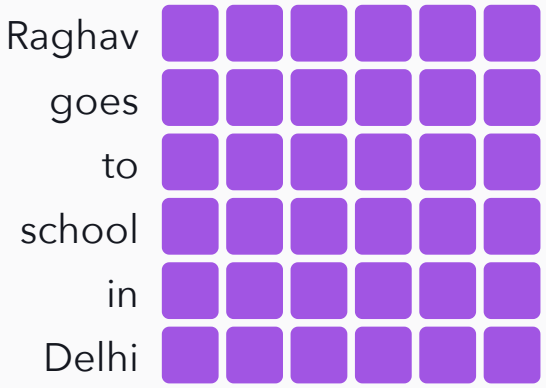


each word reads the ones before it

Predicts: the next word

Use: completing text (GPT)

Encoder



every word reads every word

Predicts: a label per word (Delhi: place)

Use: tagging names (BERT)

Encoder-decoder



each Hindi word reads every English word

Predicts: the next Hindi word

Use: translation

> The mask and the readout choose the family; the attention block is the same in all three.

The sentence and its Hindi translation are Lecture 15's running example (data/p4.json).

Check: the forgotten mask

You forget the mask. The training loss falls to almost zero. Why is that bad news?

```
S = Q @ K.T / d_k ** 0.5
# S = S + mask    (forgotten)
A = S.softmax(dim=-1)
```

> Two ideas meet here: what the target of each position is, and what the mask hides.

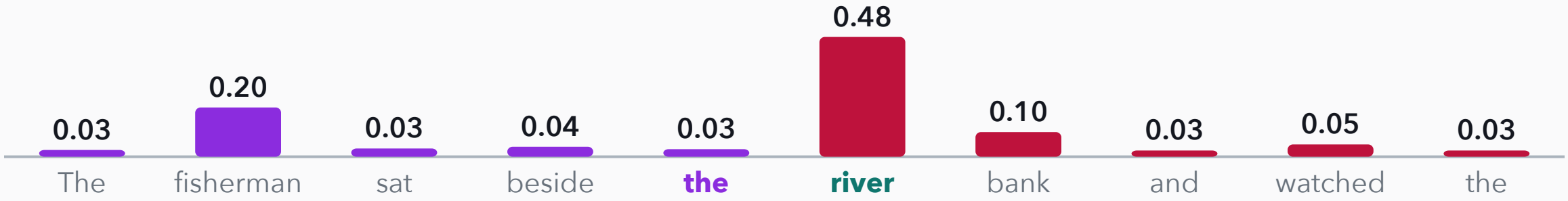
The bank model is the hand-built one of Lectures 12 and 13 (attention.html). Numbers: scripts/sci/p4.py.

Check: the forgotten mask

You forget the mask. The training loss falls to almost zero. Why is that bad news?

```
S = Q @ K.T / d_k ** 0.5
# S = S + mask    (forgotten)
A = S.softmax(dim=-1)
```

Row 5, “the” (target: river), no mask. Red: words it should not see.



Every position can read the token it is asked to predict, so it learns to copy. At test time that token does not exist yet.

Row 5 puts 0.67 of its weight on later words: 0.48 on river, its own answer.

The bank model is the hand-built one of Lectures 12 and 13 (attention.html). Numbers: scripts/sci/p4.py.

Part 4 in one slide

1. Data: text as tokens; each target is the next token.
2. Model: each token mixes the values of the tokens before it.
3. At test time: sample, append, repeat.

Transformer language model

Data	text as tokens; the target at each position is the next token
Model	embeddings, then blocks of masked self-attention
Output	p over the vocabulary, at every position
Loss	$-\ln p(\text{next token})$, averaged over positions
Update	a gradient step, as before
At test time	sample a token, append it, repeat

Next: the same block, fed image patches.

Part 5 · Images as tokens, labels as language

Part 4's transformer read text. These lectures gave it images, then tied the images to words and to concepts.

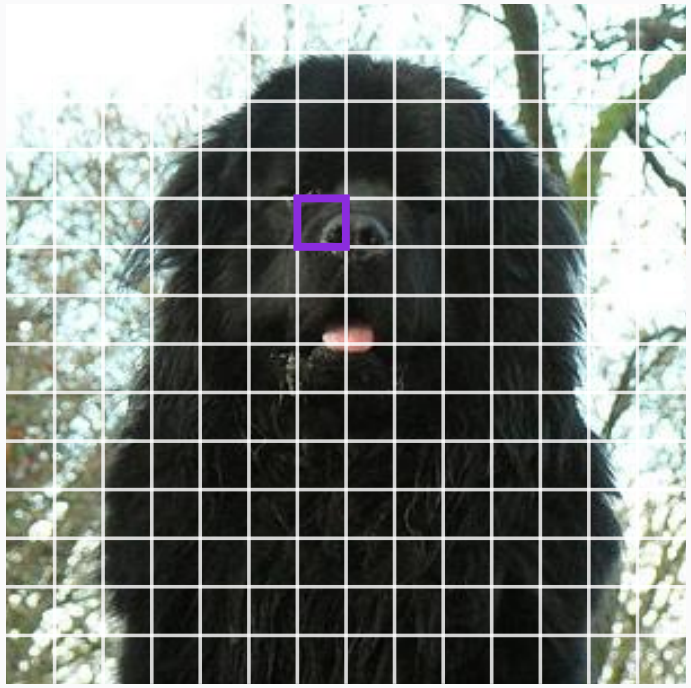
1. cut the image into patches and reuse the transformer
2. train an image encoder and a text encoder to agree
3. hand the image tokens to a language model
4. make the model reason through concepts a person can check

Lectures 16 to 22

> Aim: for each of the four models, say what it is trained on, what it outputs and what it does at test time.

From Lecture 16

A photo becomes a sequence of patch tokens



```
p = F.unfold(img, 16, stride=16).mT  # B, 196, 768
```

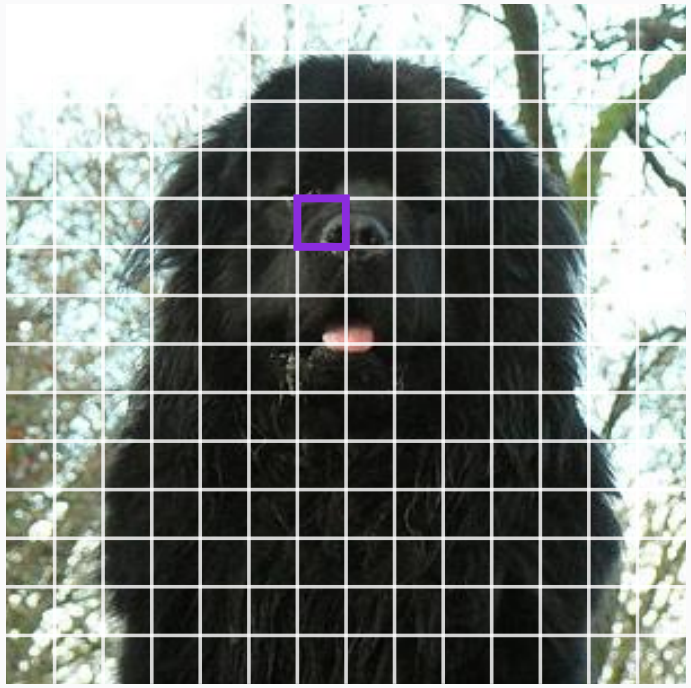
patch 16: 14 × 14 = 196 tokens · with [CLS], 197² = 38,809 attention scores per head

> Lecture 16's photo at 224 × 224, cut into square patches; each patch is one token. Switch the patch size: what happens to the tokens and the attention cost?

Photo: newfoundland_31 from Oxford-IIIT Pet (Parkhi, Vedaldi, Zisserman and Jawahar), CC BY-SA 4.0, as the model sees it. ViT-Tiny (vit_tiny_patch16_224); the prediction is Lecture 16's saved run. Numbers: scripts/sci/p5.py.

From Lecture 16

A photo becomes a sequence of patch tokens



```
p = F.unfold(img, 16, stride=16).mT  # B, 196, 768
x = embed(p)                         # B, 196, 192
```

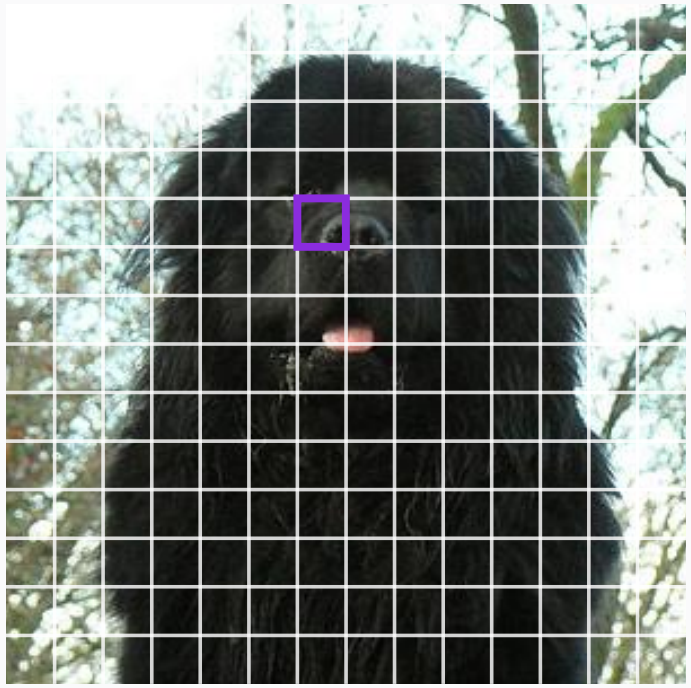
patch 16: 14 × 14 = 196 tokens · with [CLS], 197² = 38,809 attention scores per head

> At patch size 16, each patch is 16 × 16 × 3 = 768 numbers. One shared linear map turns every patch into 192 features.

Photo: newfoundland_31 from Oxford-IIIT Pet (Parkhi, Vedaldi, Zisserman and Jawahar), CC BY-SA 4.0, as the model sees it. ViT-Tiny (vit_tiny_patch16_224); the prediction is Lecture 16's saved run. Numbers: scripts/sci/p5.py.

From Lecture 16

A photo becomes a sequence of patch tokens



```
p = F.unfold(img, 16, stride=16).mT  # B, 196, 768
x = embed(p)                         # B, 196, 192
x = prepend_cls(x) + pos             # B, 197, 192
for block in blocks:                 # 12 blocks
    x = block(x)                     # B, 197, 192
```

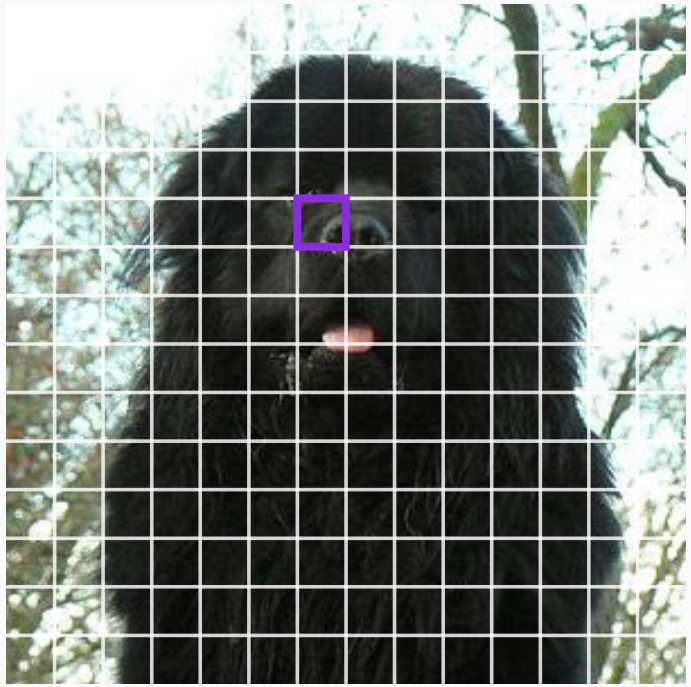
patch 16: 14 × 14 = 196 tokens · with [CLS], 197² = 38,809 attention scores per head

> A learned [CLS] row and a position vector per row give 197 × 192. Then twelve attention blocks with no mask, the encoder of Part 4.

Photo: newfoundland_31 from Oxford-IIIT Pet (Parkhi, Vedaldi, Zisserman and Jawahar), CC BY-SA 4.0, as the model sees it. ViT-Tiny (vit_tiny_patch16_224); the prediction is Lecture 16's saved run. Numbers: scripts/sci/p5.py.

From Lecture 16

A photo becomes a sequence of patch tokens



```
p = F.unfold(img, 16, stride=16).mT  # B, 196, 768
x = embed(p)                         # B, 196, 192
x = prepend_cls(x) + pos             # B, 197, 192
for block in blocks:                 # 12 blocks
    x = block(x)                     # B, 197, 192
logits = head(norm(x)[:, 0])         # B, 1,000
```

top class	**Newfoundland**	**95.73 %**
	Tibetan mastiff	1.63 %
	briard	0.67 %

patch 16: 14 × 14 = 196 tokens · with [CLS], 197² = 38,809 attention scores per head

> The final [CLS] row goes through a linear head with 1,000 outputs, then a softmax. The right breed comes first, far ahead of the next two.

Photo: newfoundland_31 from Oxford-IIIT Pet (Parkhi, Vedaldi, Zisserman and Jawahar), CC BY-SA 4.0, as the model sees it. ViT-Tiny (vit_tiny_patch16_224); the prediction is Lecture 16's saved run. Numbers: scripts/sci/p5.py.

From Lecture 17

CLIP: a softmax along each row of the batch

cosine of each photo with each caption

	"cat"	"dog"	"car"
cat photo	**0.97**	0.65	0.40
dog photo	0.61	**0.96**	0.39
car photo	0.37	0.36	**0.97**

> Lecture 17's batch: three photos and their three captions, each encoded to a unit vector. Which cells of this matrix should training push up?

The authored 3-D batch of Lecture 17 (cat, dog, car). The loss after 100 steps is the lecture's run. Numbers: scripts/sci/p5.py.

From Lecture 17

CLIP: a softmax along each row of the batch

cosine of each photo with each caption

	"cat"	"dog"	"car"
cat photo	**0.97**	0.65	0.40
dog photo	0.61	**0.96**	0.39
car photo	0.37	0.36	**0.97**

÷ τ, then

softmax along a row

P: each photo picks a caption

	"cat"	"dog"	"car"
cat photo	**0.54**	0.29	0.17
dog photo	0.27	**0.55**	0.18
car photo	0.19	0.18	**0.63**

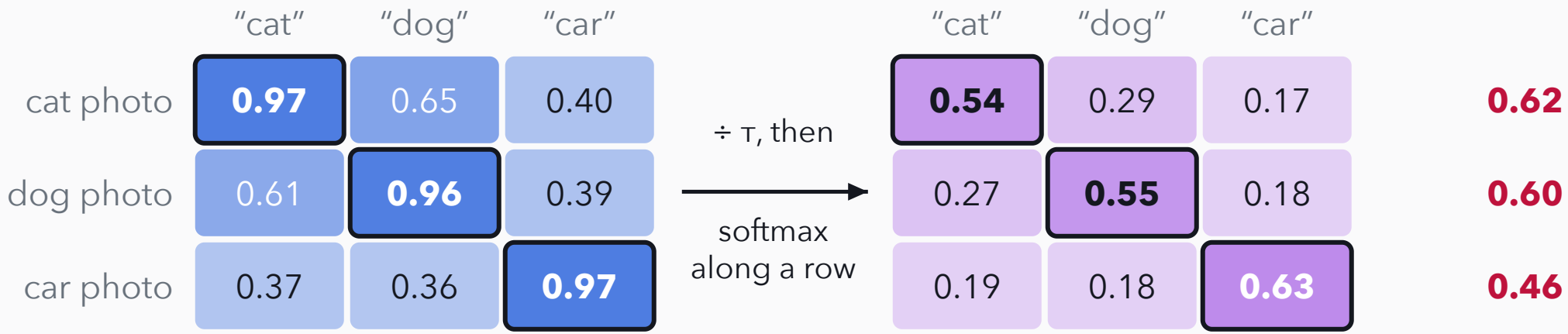
Divide by the temperature τ = 0.5, then a softmax along each row: each photo picks a caption. Drag τ and watch the rows sharpen or flatten.

The authored 3-D batch of Lecture 17 (cat, dog, car). The loss after 100 steps is the lecture's run. Numbers: scripts/sci/p5.py.

From Lecture 17

CLIP: a softmax along each row of the batch

cosine of each photo with each caption | **P: each photo picks a caption** | **−ln P, paired cell**



loss 0.56 the mean of the rows (0.56) and of the columns (0.56)

a model that cannot tell them apart gives each caption 1/3: ln 3 = 1.10

> The loss is −ln of the paired cell, averaged over the rows and again down the columns, where each caption picks a photo.

The authored 3-D batch of Lecture 17 (cat, dog, car). The loss after 100 steps is the lecture's run. Numbers: scripts/sci/p5.py.

From Lecture 17

CLIP: a softmax along each row of the batch

cosine of each photo with each caption

	"cat"	"dog"	"car"
cat photo	**0.97**	0.65	0.40
dog photo	0.61	**0.96**	0.39
car photo	0.37	0.36	**0.97**

÷ τ, then

softmax along a row

P: each photo picks a caption

	"cat"	"dog"	"car"
cat photo	**0.54**	0.29	0.17
dog photo	0.27	**0.55**	0.18
car photo	0.19	0.18	**0.63**

−ln P, paired cell

0.62

0.60

0.46

loss 0.56 the mean of the rows (0.56) and of the columns (0.56)

a model that cannot tell them apart gives each caption 1/3: ln 3 = 1.10

after 100 steps at τ = 0.5: 0.12

> Training moves both encoders and the temperature so that the diagonal wins. On this one batch the loss keeps falling.

The authored 3-D batch of Lecture 17 (cat, dog, car). The loss after 100 steps is the lecture's run. Numbers: scripts/sci/p5.py.

From Lecture 17

At test time, each class is written as a caption



cosine of the photo with each caption

“a photo of a dog”

“a photo of a cat”

“a photo of a bird”

> The ViT could only answer with its stored classes. With CLIP we write each class as a caption, encode it, and compare. Which caption wins?

Photo: newfoundland_31 from Oxford-IIIT Pet (Parkhi, Vedaldi, Zisserman and Jawahar), CC BY-SA 4.0. Cosines: frozen CLIP ViT-B/32, Lecture 17’s saved measurements. Numbers: scripts/sci/p5.py.

From Lecture 17

At test time, each class is written as a caption



cosine of the photo with each caption

“a photo of a dog” **0.2723**

“a photo of a cat” 0.2152

“a photo of a bird” 0.2184

> The photo goes to the caption with the largest cosine. All three scores come from one image vector; the captions are encoded once.

Photo: newfoundland_31 from Oxford-IIIT Pet (Parkhi, Vedaldi, Zisserman and Jawahar), CC BY-SA 4.0. Cosines: frozen CLIP ViT-B/32, Lecture 17’s saved measurements. Numbers: scripts/sci/p5.py.

From Lecture 17

At test time, each class is written as a caption



cosine of the photo with each caption

“a photo of a cat” 0.2152

“a photo of a bird” **0.2184**

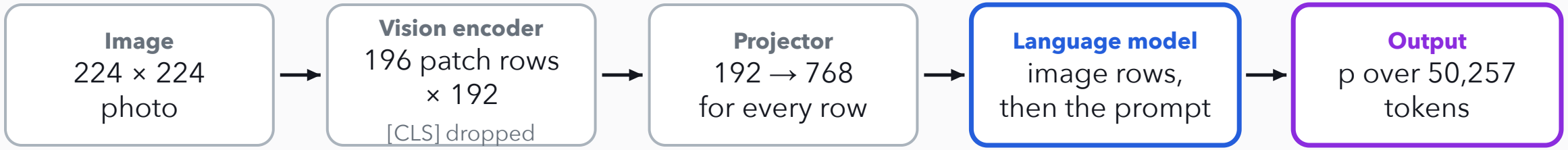
“a photo of a car” 0.2179

> Now leave “dog” out of the list. CLIP still picks one of the captions we wrote, “a photo of a bird”, ahead of “a photo of a car” by 0.0005.

Photo: newfoundland_31 from Oxford-IIIT Pet (Parkhi, Vedaldi, Zisserman and Jawahar), CC BY-SA 4.0. Cosines: frozen CLIP ViT-B/32, Lecture 17’s saved measurements. Numbers: scripts/sci/p5.py.

From Lecture 18

A VLM puts the image in front of a language model

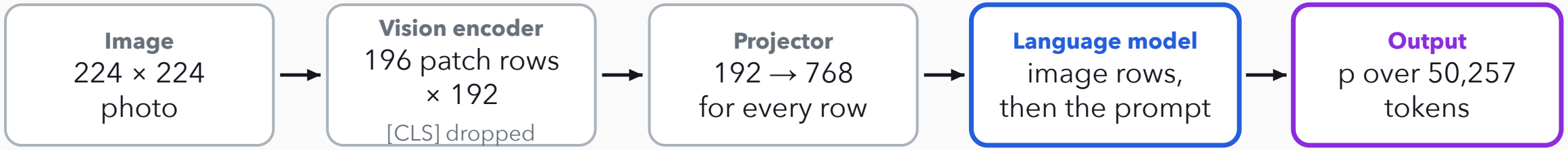


> Lectures 18 and 19 kept the language model and gave it the image. For one photo and a 13-token prompt, how many positions does it read?

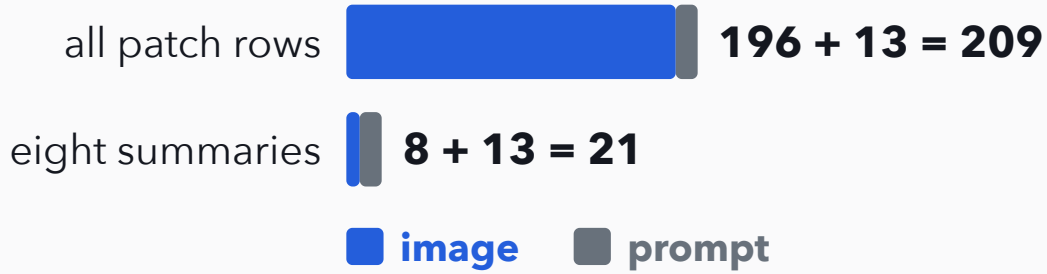
DeiT-Tiny rows and DistilGPT2 in Lectures 18 and 19's notebook; the answer probabilities are the lecture's chosen numbers. Numbers: scripts/sci/p5.py.

From Lecture 18

A VLM puts the image in front of a language model



positions the language model reads



> Every patch row becomes a position in front of the prompt. Lecture 19's eight learned summary vectors read all 196 rows and shorten the prefix.

DeiT-Tiny rows and DistilGPT2 in Lectures 18 and 19's notebook; the answer probabilities are the lecture's chosen numbers. Numbers: scripts/sci/p5.py.

From Lecture 18

A VLM puts the image in front of a language model

Image
224 × 224 photo

→

Vision encoder
196 patch rows × 192
[CLS] dropped

→

Projector
192 → 768 for every row

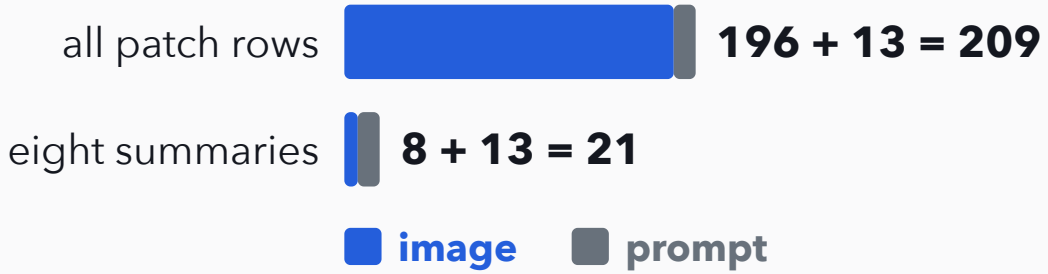
→

Language model
image rows, then the prompt

→

Output
p over 50,257 tokens

positions the language model reads



target	"A"	"black"	"dog"	"."	end
p	0.20	0.40	0.50	0.80	0.90
−ln p	1.61	0.92	0.69	0.22	0.11

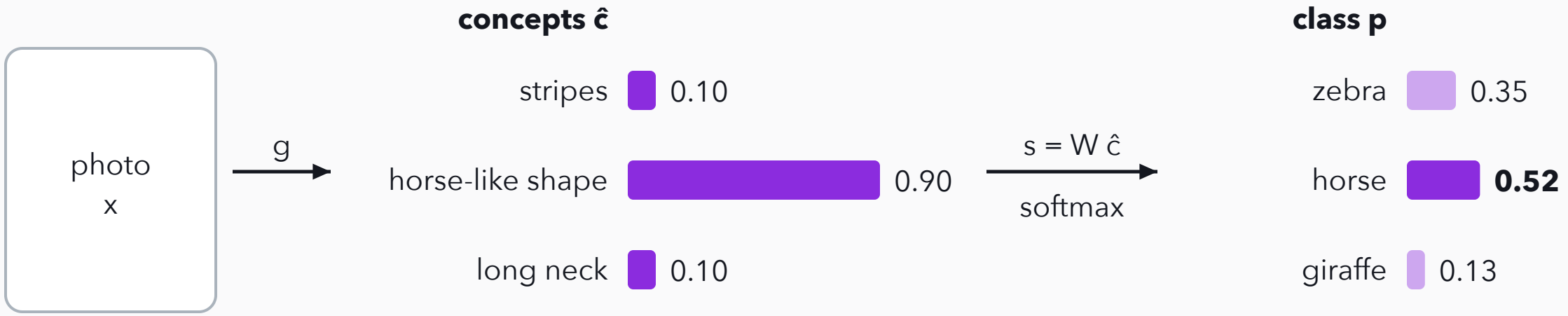
mean over the answer: **0.709**

> Training on the reference answer "A black dog.": the loss is −ln p of each answer token, given the image and the tokens before it.

DeiT-Tiny rows and DistilGPT2 in Lectures 18 and 19's notebook; the answer probabilities are the lecture's chosen numbers. Numbers: scripts/sci/p5.py.

From Lecture 20

A concept bottleneck: the class is read from concepts alone

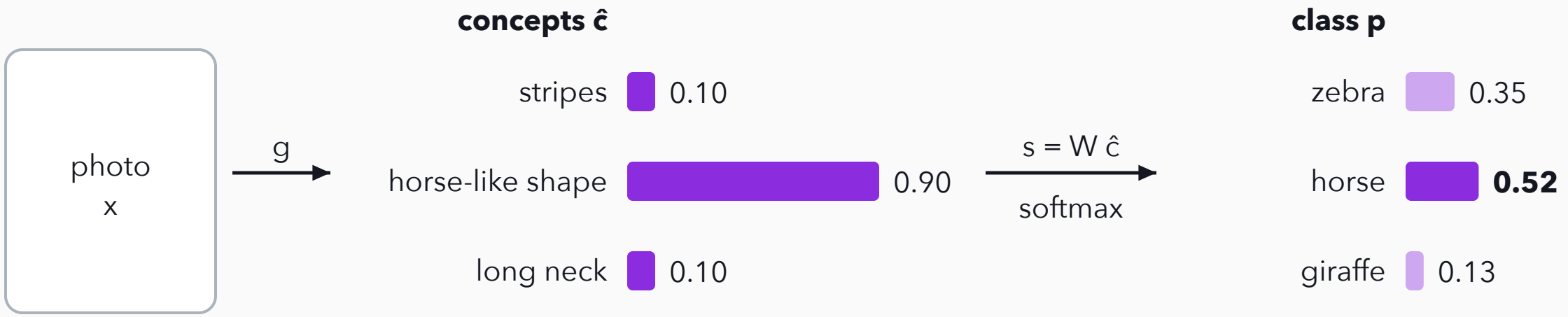


> Lecture 20's worksheet: three concepts, then one linear head W. The model says horse. A person sees stripes: correct it to 1. What happens?

The zebra, horse and giraffe worksheet of Lecture 20; weights from the cbm repository. Numbers: scripts/sci/p5.py.

From Lecture 20

A concept bottleneck: the class is read from concepts alone



$$\Delta s = W\,\Delta \hat{c} = 0.9 \times (3, -1, -1) = (2.7, -0.9, -0.9)$$

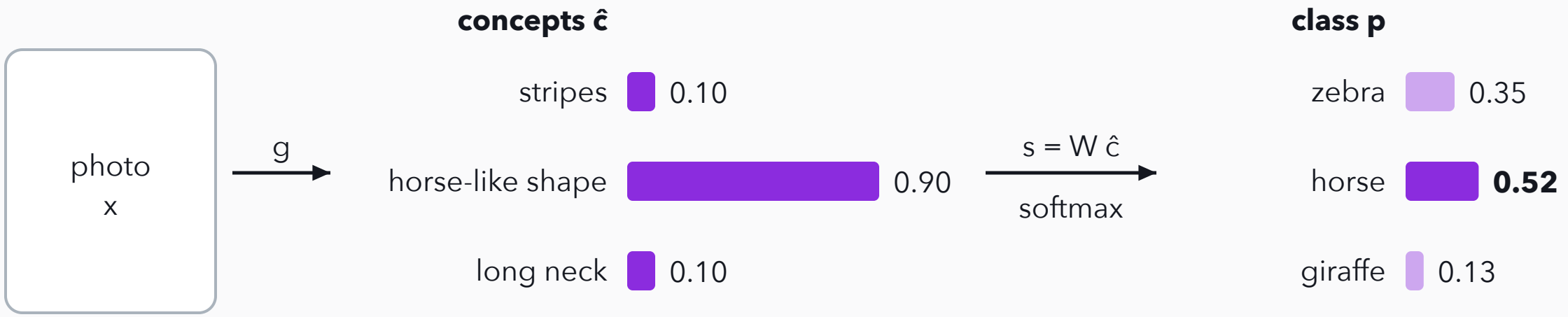
$$p_{\text{zebra}} = 0.35 \;\rightarrow\; 0.95$$

> Only the stripes column of W moves the scores. Nothing is retrained: the same head, given the corrected concept, now says zebra.

The zebra, horse and giraffe worksheet of Lecture 20; weights from the cbm repository. Numbers: scripts/sci/p5.py.

From Lecture 20

A concept bottleneck: the class is read from concepts alone



$$\Delta s = W\,\Delta \hat{c} = 0.9 \times (3, -1, -1) = (2.7, -0.9, -0.9)$$

$$p_{\text{zebra}} = 0.35 \;\rightarrow\; 0.95$$

$$\mathcal{L} = \lambda_c\,\mathcal{L}_c + \lambda_y\,\mathcal{L}_y$$

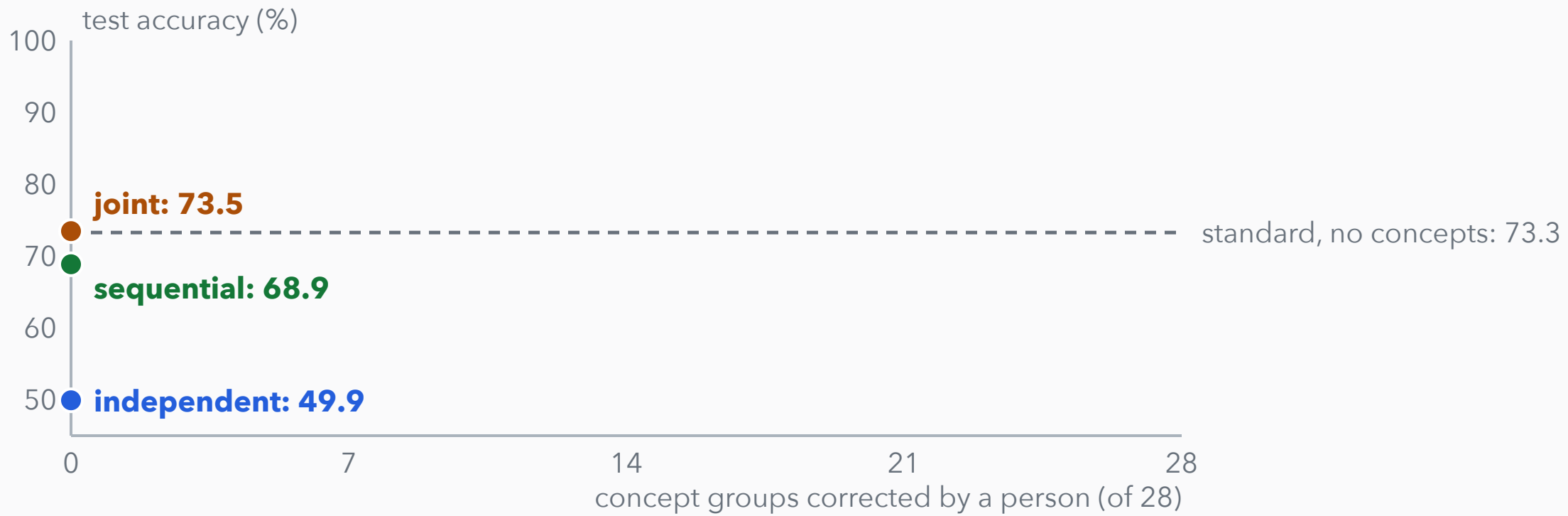
$$-\ln 0.8 = 0.22 \quad \text{(one concept, label } 1\text{)}$$

> Training needs a label for every concept as well as the class: a concept loss on ĉ plus the usual class cross-entropy.

The zebra, horse and giraffe worksheet of Lecture 20; weights from the cbm repository. Numbers: scripts/sci/p5.py.

From Lecture 20

On real birds: accuracy, then accuracy after corrections

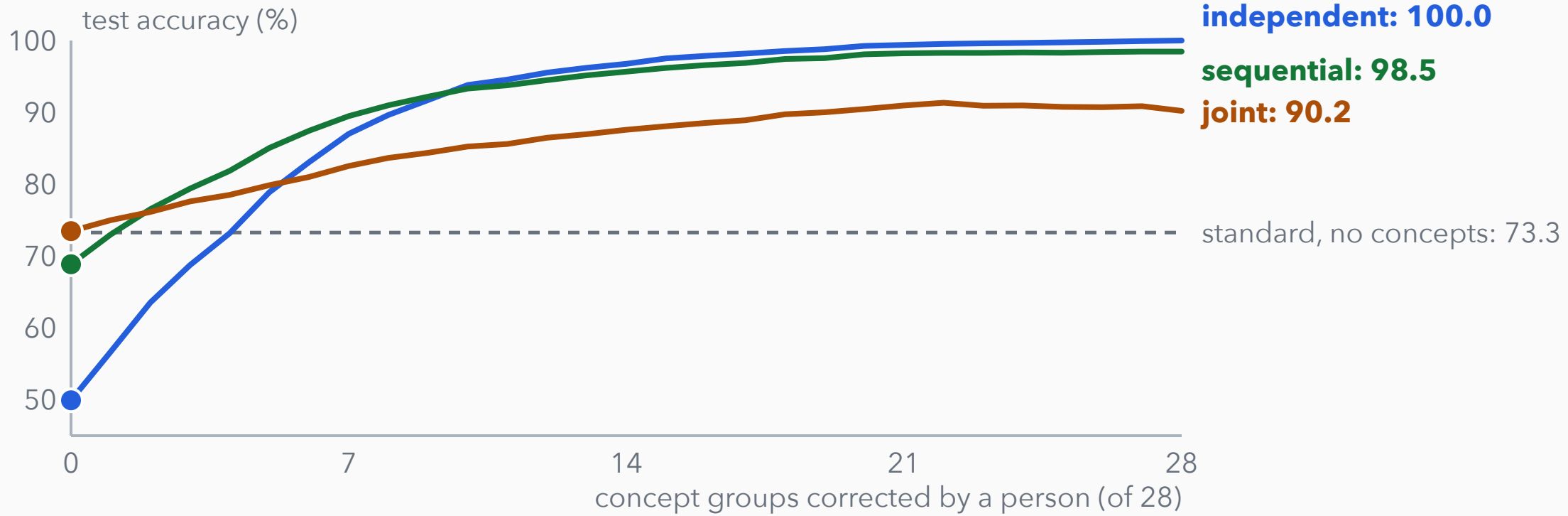


> Lecture 20's run on CUB: 200 bird species, 112 concepts in 28 groups. Which model would you deploy if a bird expert will correct its concepts?

CUB, frozen CLIP ViT-B/32 features, one run (Lecture 20, from the cbm repository). Numbers: scripts/sci/p5.py.

From Lecture 20

On real birds: accuracy, then accuracy after corrections



> Correct one group of concepts after another. The least accurate model gains the most: its head was trained on true concepts, so it trusts corrected ones.

CUB, frozen CLIP ViT-B/32 features, one run (Lecture 20, from the cbm repository). Numbers: scripts/sci/p5.py.

Check: which of these need class labels?

Vision Transformer
outputs the class of a photo

CLIP
outputs which caption fits which photo

Vision-language model
outputs words about a photo

Concept bottleneck model
outputs concepts, then a class

> Which of these four models need a class label on every training photo? Think about what the Data row of each held.

Check: which of these need class labels?

Vision Transformer

outputs the class of a photo

trained on photos with a class label

CLIP

outputs which caption fits which photo

trained on photos with their captions

Vision-language model

outputs words about a photo

trained on a photo, a question and its answer

Concept bottleneck model

outputs concepts, then a class

trained on photos with concept labels and a class

The ViT and the concept bottleneck model, which needs concept labels too.

> The Data row is where these models differ most; every loss is still $-\ln p$.

Part 5 in one slide

1. Patches make 196 tokens for Part 4's encoder.
2. CLIP's classes are the captions in the batch.
3. A VLM reads image rows; a bottleneck's concepts can be corrected.

CLIP

Data	photos with their captions; no class labels
Model	an image encoder and a text encoder
Output	for each photo, p over the captions in the batch
Loss	$-\ln p(\text{its own caption})$, and the same from caption to photo
Update	a gradient step, as before
At test time	write each class as a caption, pick the closest

Next: so far every model predicted a label or a token; next, models of the data itself.

Part 6 · Learning a distribution

Lectures 23 to 25 learned whole distributions, over a network's weights and over images. The loss became a KL, and the model gained a sampled code.

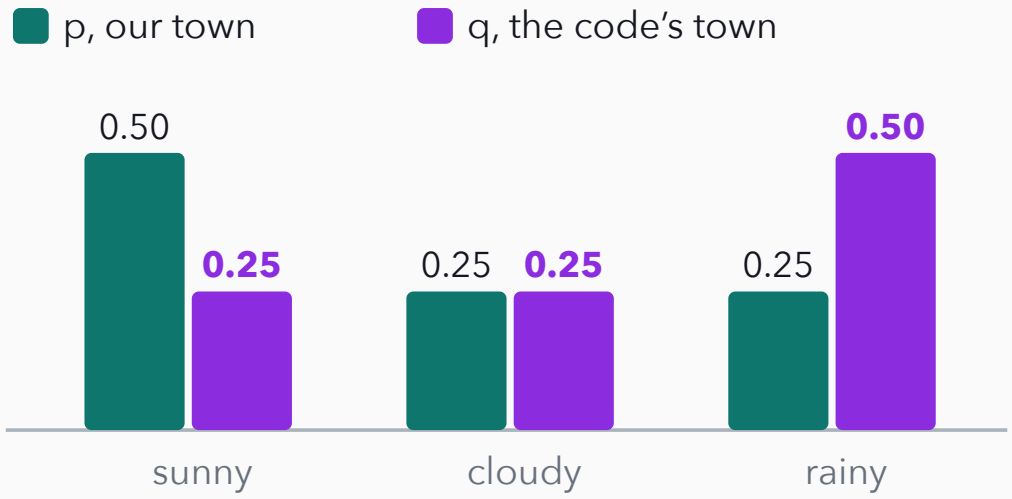
1. measure how far one distribution is from another: the KL divergence
2. when the posterior is out of reach, fit a simpler q by raising the ELBO
3. sample through $\theta = \mu + \sigma\epsilon$ so that gradients flow
4. put an encoder and a decoder around a code z: the VAE

Lectures 23 to 25

> By the end you can say why the VAE's loss is a reconstruction error plus a KL.

From Lecture 23

The KL divergence: the extra cost of the wrong code



$$H(p, q) = \sum_x p(x) \log_2 \frac{1}{q(x)}$$

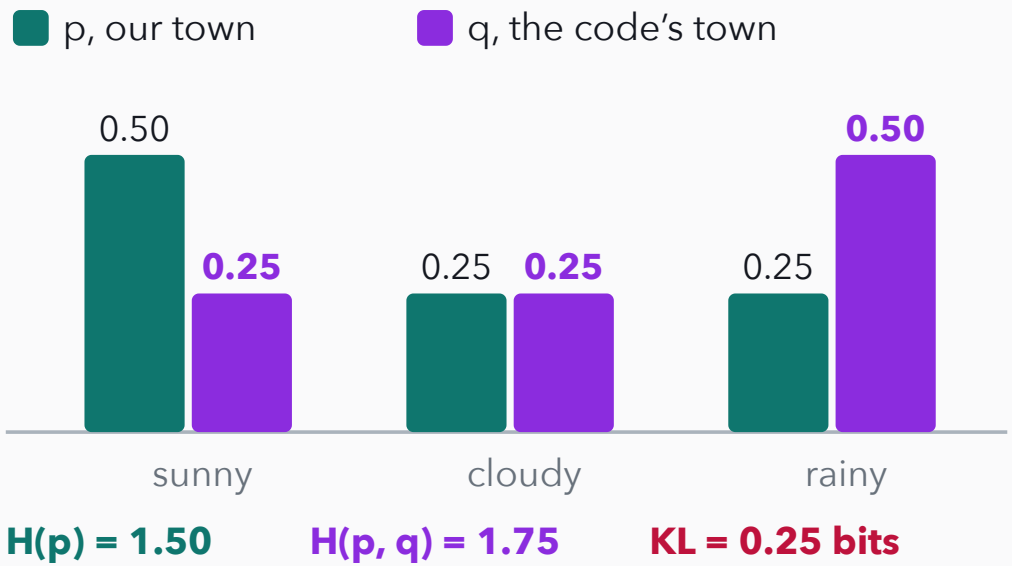
the bits a day, when the days come from p and the code was built for q

> Lecture 23's weather. Our days follow p; we borrow the code another town built for its weather, q. How many extra bits a day does that cost?

Weather and classifier from Lecture 23; coin and mixture fit from Lecture 24. Numbers: scripts/sci/p6.py.

From Lecture 23

The KL divergence: the extra cost of the wrong code



$$H(p, q) = \sum_x p(x) \log_2 \frac{1}{q(x)}$$

the bits a day, when the days come from p and the code was built for q

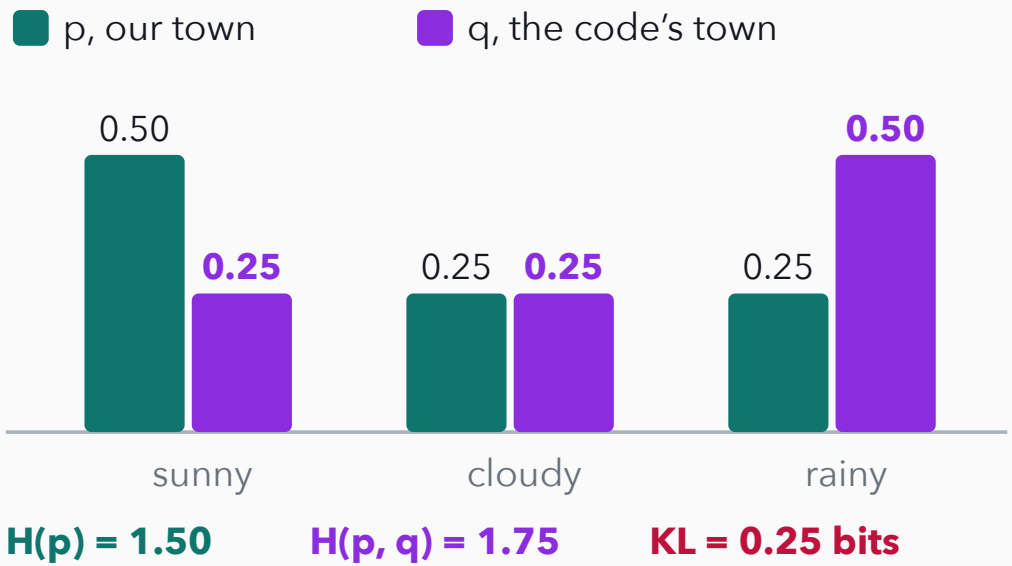
$$\begin{aligned} \mathrm{KL}(p \,\|\, q) &= H(p, q) - H(p) \\ &= 1.75 - 1.50 = 0.25 \text{ bits} \end{aligned}$$

H(p) is what the best code for our days costs. The KL is the extra we pay. Drag q towards p: at q = p the extra is zero.

Weather and classifier from Lecture 23; coin and mixture fit from Lecture 24. Numbers: scripts/sci/p6.py.

From Lecture 23

The KL divergence: the extra cost of the wrong code



$$H(p, q) = \sum_x p(x) \log_2 \frac{1}{q(x)}$$

the bits a day, when the days come from p and the code was built for q

$$\begin{aligned} \mathrm{KL}(p \,\|\, q) &= H(p, q) - H(p) \\ &= 1.75 - 1.50 = 0.25 \text{ bits} \end{aligned}$$

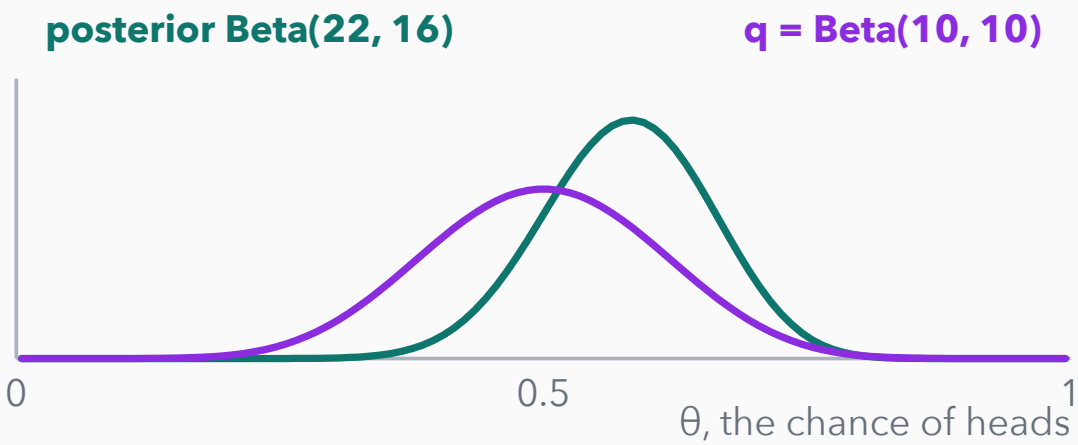
$$\mathrm{KL}(\text{label} \,\|\, \text{prediction}) = -\ln 0.60 = 0.51 \text{ nats}$$

> Part 1's loss is the same quantity. The label is p, one-hot on cat; the prediction gives cat 0.60. The label's entropy is 0, so cross-entropy is a KL.

Weather and classifier from Lecture 23; coin and mixture fit from Lecture 24. Numbers: scripts/sci/p6.py.

From Lecture 24

The ELBO: a lower bound we can raise



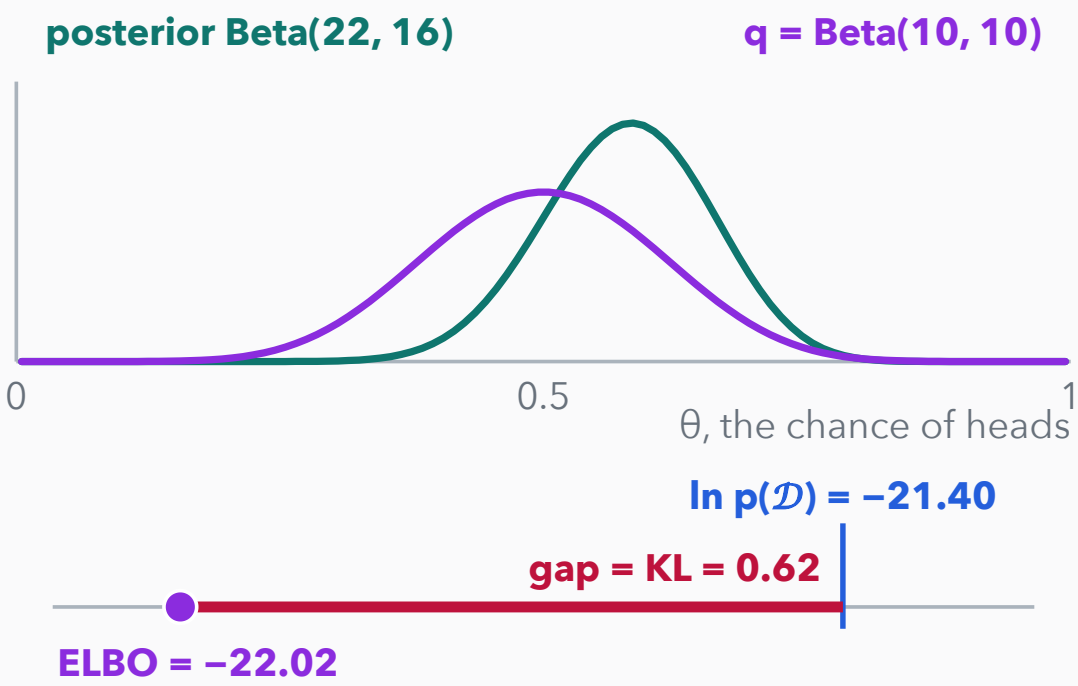
$$\ln p(\mathcal{D}) = \ln \int p(\mathcal{D} \mid \theta)\, p(\theta)\, d\theta = -21.40$$

> Lecture 24's coin: a Beta(5, 3) prior, then 17 heads in 30 tosses. For a coin the evidence can be computed, so we can check VI against it.

Weather and classifier from Lecture 23; coin and mixture fit from Lecture 24. Numbers: scripts/sci/p6.py.

From Lecture 24

The ELBO: a lower bound we can raise



$$\ln p(\mathcal{D}) = \ln \int p(\mathcal{D} \mid \theta)\, p(\theta)\, d\theta = -21.40$$

$$\ln p(\mathcal{D}) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big)$$

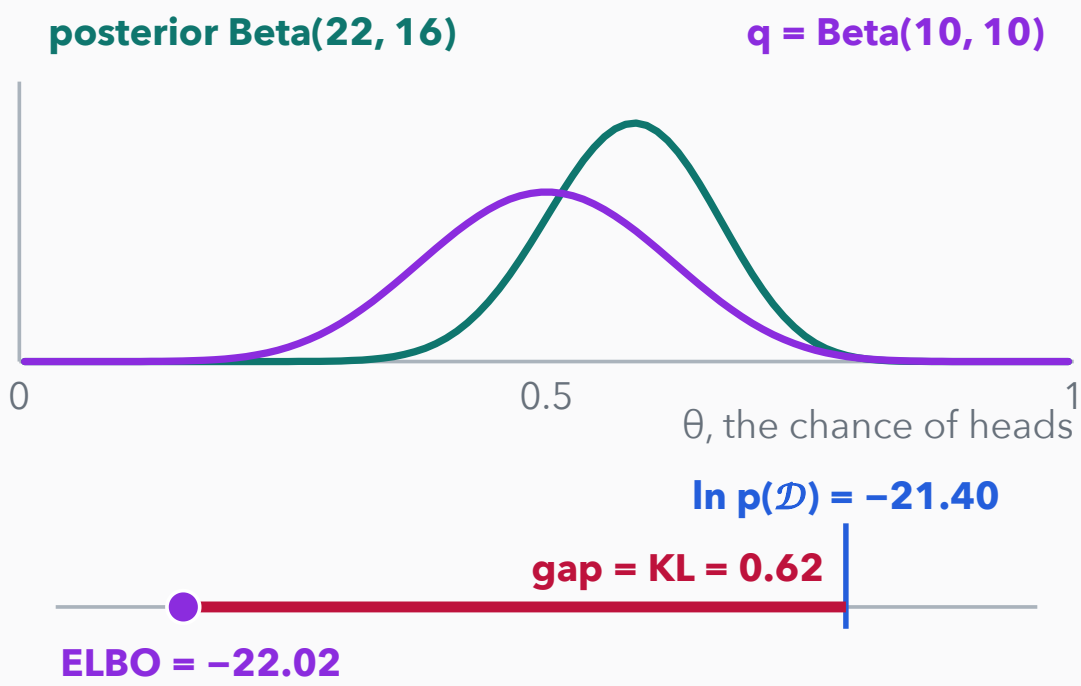
$$-21.40 = -22.02 + 0.62$$

> Pick any q, here Beta(10, 10). The evidence splits into the ELBO, which we can compute, and the KL to the posterior, which we cannot.

Weather and classifier from Lecture 23; coin and mixture fit from Lecture 24. Numbers: scripts/sci/p6.py.

From Lecture 24

The ELBO: a lower bound we can raise



$$\ln p(\mathcal{D}) = \ln \int p(\mathcal{D} \mid \theta)\, p(\theta)\, d\theta = -21.40$$

$$\ln p(\mathcal{D}) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big)$$
$$-21.40 = -22.02 + 0.62$$

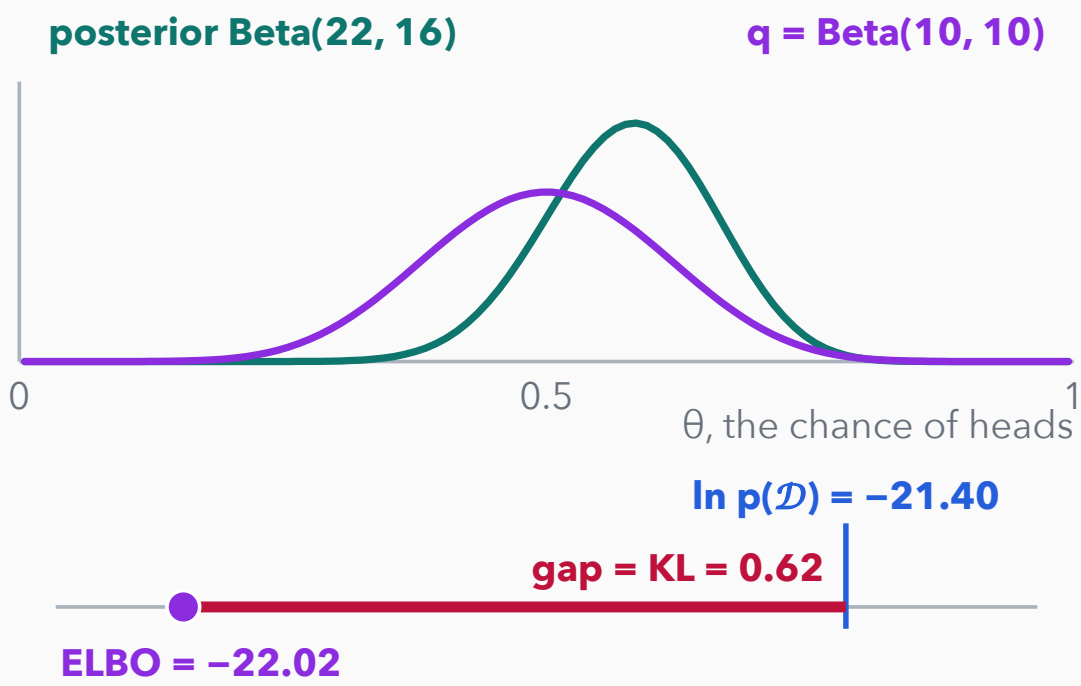
$$\mathrm{ELBO}(q) = \mathbb{E}_q\big[\ln p(\mathcal{D} \mid \theta)\big] - \mathrm{KL}\big(q \,\|\, p(\theta)\big)$$
$$= -21.56 - 0.46$$

The ELBO needs only q, the likelihood and the prior: fit the data, and stay near the prior.

Weather and classifier from Lecture 23; coin and mixture fit from Lecture 24. Numbers: scripts/sci/p6.py.

From Lecture 24

The ELBO: a lower bound we can raise



$$\ln p(\mathcal{D}) = \ln \int p(\mathcal{D} \mid \theta)\, p(\theta)\, d\theta = -21.40$$

$$\ln p(\mathcal{D}) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big)$$
$$-21.40 = -22.02 + 0.62$$

$$\begin{aligned}\mathrm{ELBO}(q) &= \mathbb{E}_q\big[\ln p(\mathcal{D} \mid \theta)\big] - \mathrm{KL}\big(q \,\|\, p(\theta)\big) \\ &= -21.56 - 0.46\end{aligned}$$

At Beta(22, 16) the gap is 0.00: the ELBO reaches ln p(𝒟), so this q is the posterior.

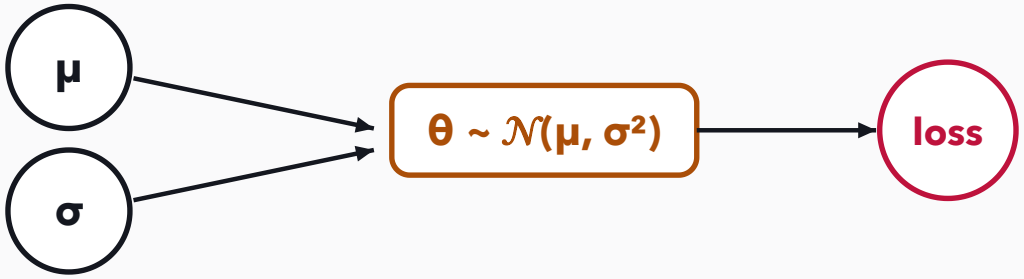
Drag q. The total never changes, so whatever the ELBO gains, the KL loses. Where does the gap close?

Weather and classifier from Lecture 23; coin and mixture fit from Lecture 24. Numbers: scripts/sci/p6.py.

From Lecture 24

Gradients through a sample: θ = μ + σε

θ drawn by q.sample()




```
theta = q.sample()
```

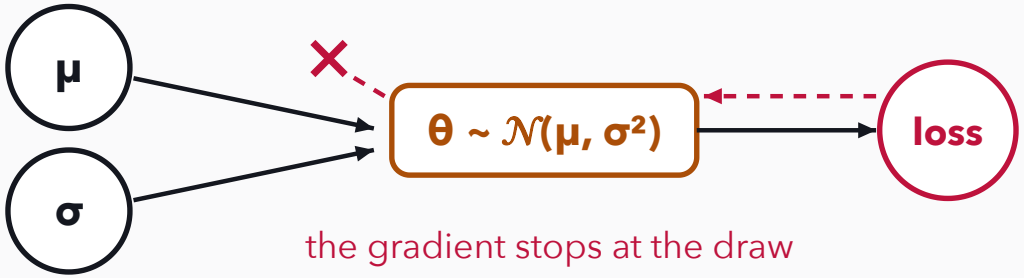
In Lecture 24 this fit left q at $\mathcal{N}(6.08, 0.10^2)$ for 400 steps, with the KL still 53.4.

> Lecture 24 fitted q by drawing θ from it and averaging the loss. The code ran, and q did not move. Where does the gradient stop?

From Lecture 24

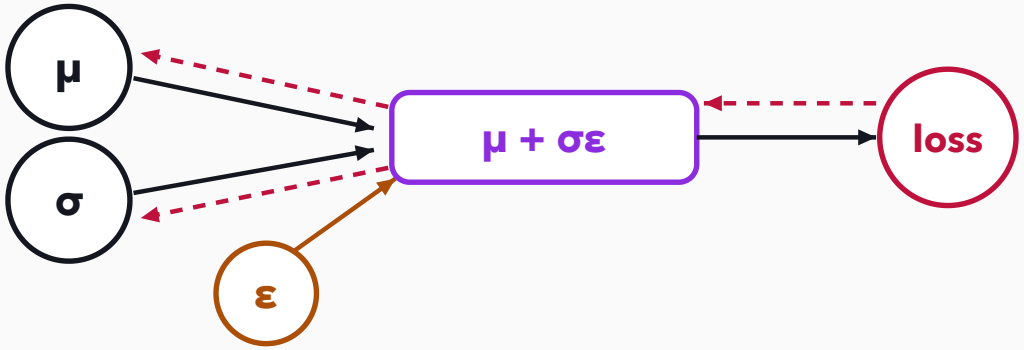
Gradients through a sample: θ = μ + σε

θ drawn by q.sample()



the gradient stops at the draw

θ = μ + σε




```
theta = q.sample()
```

In Lecture 24 this fit left q at $\mathcal{N}(6.08, 0.10^2)$ for 400 steps, with the KL still 53.4.

$$\epsilon \sim \mathcal{N}(0,1), \qquad \theta = \mu + \sigma\epsilon$$

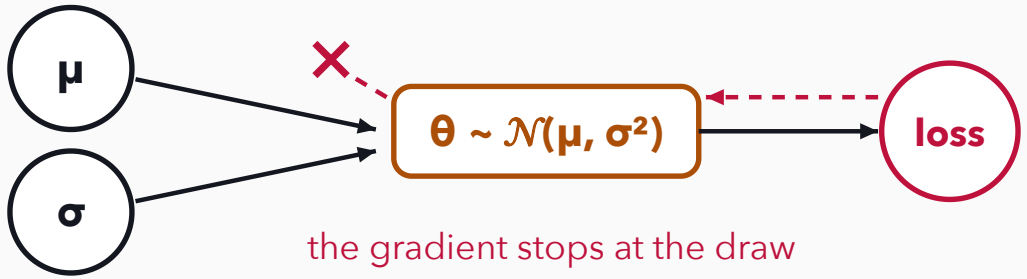
Move the randomness into ε, a fixed input with no parameters. Then θ = μ + σε is ordinary arithmetic, and backprop goes through it.

Weather and classifier from Lecture 23; coin and mixture fit from Lecture 24. Numbers: scripts/sci/p6.py.

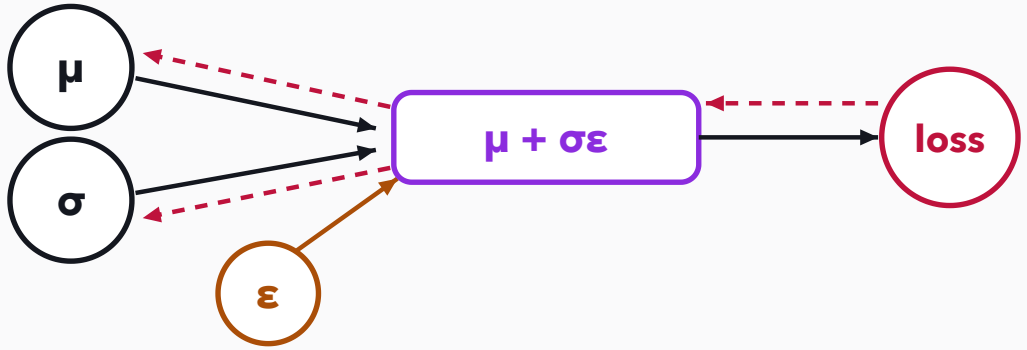
From Lecture 24

Gradients through a sample: θ = μ + σε

θ drawn by q.sample()



θ = μ + σε




```
theta = q.sample()
```

In Lecture 24 this fit left q at $\mathcal{N}(6.08, 0.10^2)$ for 400 steps, with the KL still 53.4.

$$\epsilon \sim \mathcal{N}(0,1), \qquad \theta = \mu + \sigma\epsilon$$

$$\theta = 0 + 2 \times (-0.75) = -1.5$$

$$\frac{\partial \theta}{\partial \mu} = 1, \qquad \frac{\partial \theta}{\partial \sigma} = \epsilon = -0.75$$

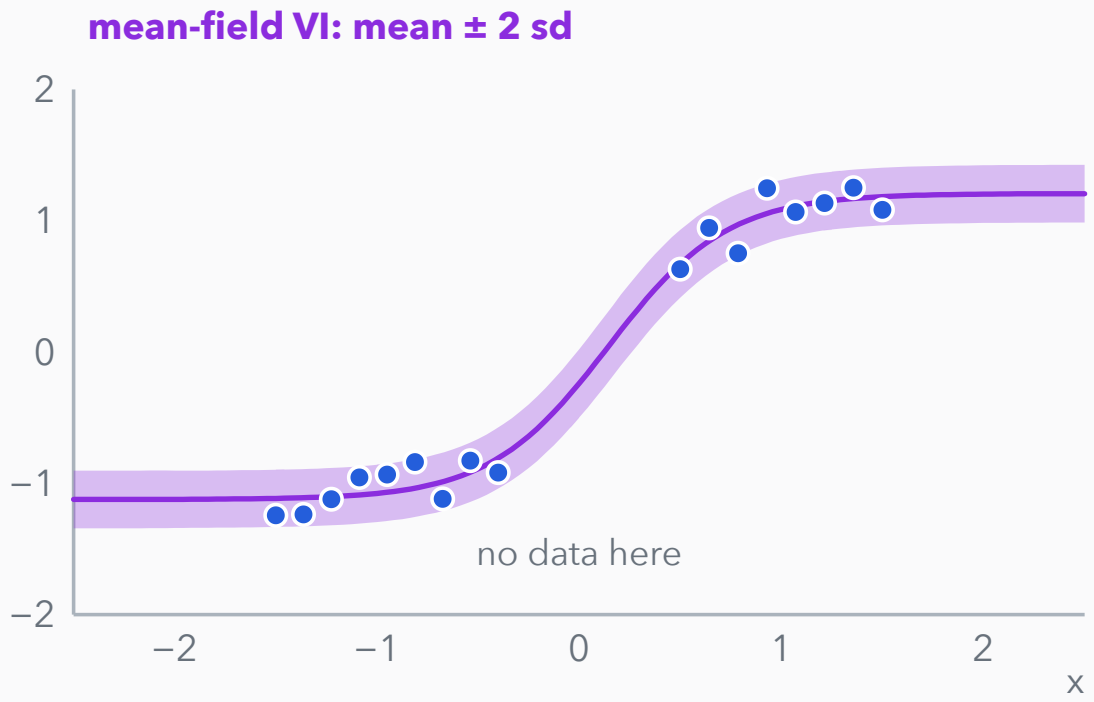
```
theta = q.rsample()    # one word changed
```

Put in one draw. With rsample() the same fit ends at $\mathcal{N}(0.43, 0.72^2)$, KL 0.056.

Weather and classifier from Lecture 23; coin and mixture fit from Lecture 24. Numbers: scripts/sci/p6.py.

From Lecture 24

A Bayesian network: predictions with error bars



$$\text{ELBO} = \underbrace{8.10}_{\text{fit the data}} - \underbrace{15.28}_{\text{stay near the prior}} = -7.18$$

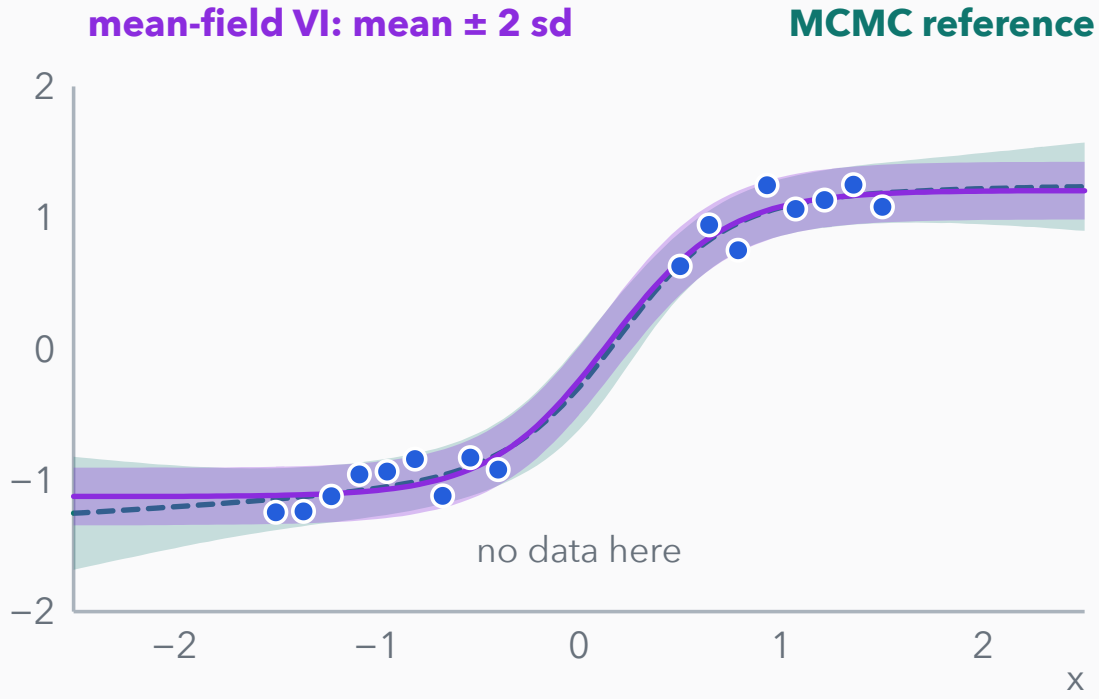
up from −1,234.59 at the first step, after 4,000 steps of 16 sampled networks each

> Lecture 23's tiny network: seven weights, 17 points with a gap. VI fitted q, a mean and a width per weight. Where should it be least sure?

The tiny network of Lectures 23 and 24; bands from generative/lectures/lecture1/data/science.json. Numbers: scripts/sci/p6.py.

From Lecture 24

A Bayesian network: predictions with error bars



$$\text{ELBO} = \underbrace{8.10}_{\text{fit the data}} - \underbrace{15.28}_{\text{stay near the prior}} = -7.18$$

up from −1,234.59 at the first step, after 4,000 steps of 16 sampled networks each

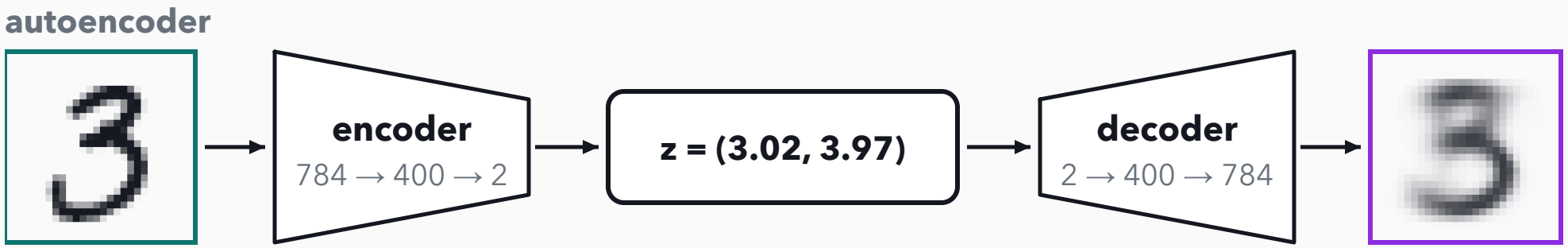
In the gap the band's sd grows from 0.11 to 0.13. The reference is wider still: 0.16 in the gap and 0.22 at x = −2.5, against 0.11.

At test time we draw networks from q and average them. VI widens its band in the gap, but less than the reference does: reverse KL makes q too sure.

The tiny network of Lectures 23 and 24; bands from generative/lectures/lecture1/data/science.json. Numbers: scripts/sci/p6.py.

From Lecture 25

An autoencoder, and the VAE that grows out of it

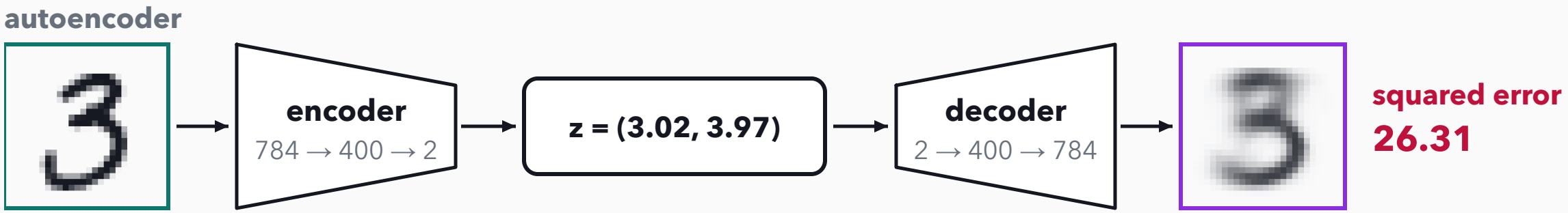


> Lecture 25's autoencoder on MNIST test digit #90, our 3: 784 pixels squeezed into two numbers and rebuilt. What does its loss compare?

MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

From Lecture 25

An autoencoder, and the VAE that grows out of it

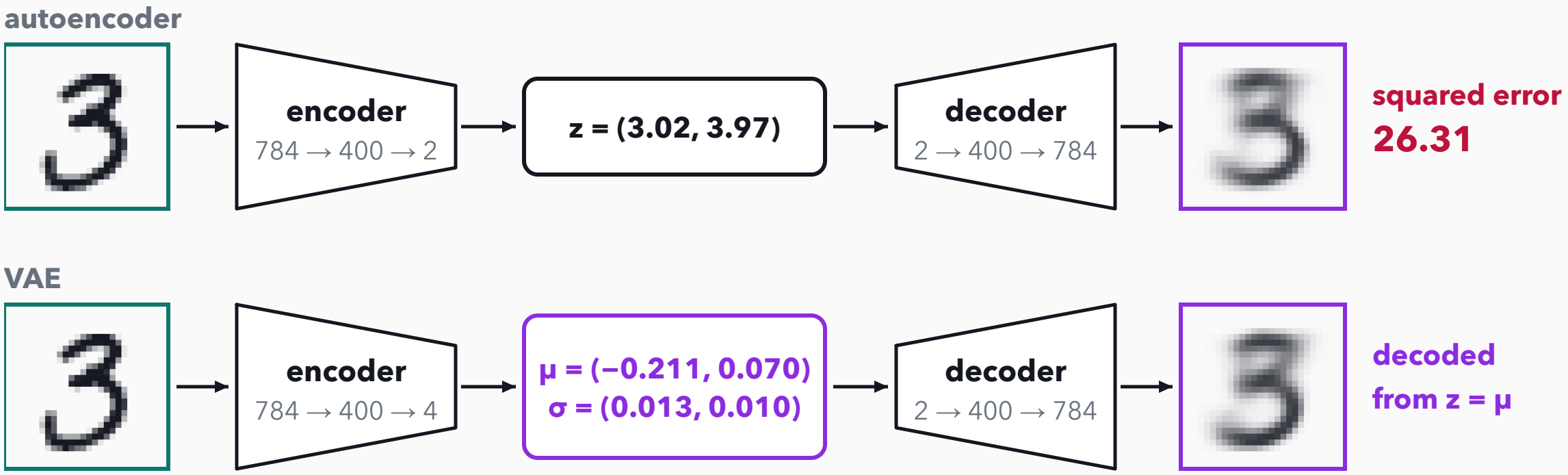


> The loss is the squared error between the image and its rebuild, summed over all 784 pixels. Random codes still decode to junk: nothing decided where codes live.

MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

From Lecture 25

An autoencoder, and the VAE that grows out of it



> The VAE's encoder returns a small cloud: a mean and a width for each code number. In training, the decoder gets a draw $z = \mu + \sigma\varepsilon$.

MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

From Lecture 25

The VAE's loss: reconstruction plus KL

```
x_hat, mu, logvar = model(x)
```

$$-\mathrm{ELBO}(x) = \mathbb{E}_q\left[\frac{1}{2\cdot 0.2^2}\left\|x - f_\theta(z)\right\|^2\right] + \mathrm{KL}\left(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\right) + \text{const}$$

> Lecture 25's loss in its own four lines. The model returns the rebuilt image and the encoder's cloud, its mean and log-variance.

MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

From Lecture 25

The VAE's loss: reconstruction plus KL

```
x_hat, mu, logvar = model(x)
recon = ((x_hat - x)**2).sum(1) / (2 * 0.2**2)
```

$$-\mathrm{ELBO}(x) = \mathbb{E}_q\left[\frac{1}{2\cdot 0.2^2}\left\|x - f_\theta(z)\right\|^2\right] + \mathrm{KL}\left(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\right) + \text{const}$$

reconstruction, our 3

$$\mathbb{E}_q[\ln p(x \mid z)] = 251.31$$

= 541.35 − 12.5 × 23.20

Gaussian pixels with σ = 0.2 turn the log-likelihood into squared error times 12.5, plus a constant. Here is that term for our 3.

MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

From Lecture 25

The VAE's loss: reconstruction plus KL

```
x_hat, mu, logvar = model(x)
recon = ((x_hat - x)**2).sum(1) / (2 * 0.2**2)
kl = 0.5 * (mu**2 + logvar.exp() - logvar - 1).sum(1)
```

$$-\mathrm{ELBO}(x) = \mathbb{E}_q\left[\frac{1}{2 \cdot 0.2^2}\left\|x - f_\theta(z)\right\|^2\right] + \mathrm{KL}\left(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\right) + \text{const}$$

reconstruction, our 3

$\mathbb{E}_q[\ln p(x \mid z)] = 251.31$

= 541.35 − 12.5 × 23.20

KL term, our 3

$\mathrm{KL} = 7.94$

The KL to $\mathcal{N}(0, I)$ has a closed form for two Gaussians. It pulls every image's cloud towards the prior.

MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

From Lecture 25

The VAE's loss: reconstruction plus KL

```
x_hat, mu, logvar = model(x)
recon = ((x_hat - x)**2).sum(1) / (2 * 0.2**2)
kl = 0.5 * (mu**2 + logvar.exp() - logvar - 1).sum(1)
loss = (recon + kl).mean()
```

$$-\mathrm{ELBO}(x) = \mathbb{E}_q\left[\frac{1}{2\cdot 0.2^2}\left\|x - f_\theta(z)\right\|^2\right] + \mathrm{KL}\left(q_\phi(z \mid x)\,\|\,\mathcal{N}(0, I)\right) + \text{const}$$

reconstruction, our 3

$\mathbb{E}_q[\ln p(x \mid z)] = 251.31$

= 541.35 − 12.5 × 23.20

KL term, our 3

$\mathrm{KL} = 7.94$

the budget, our 3

$\mathrm{ELBO} = 251.31 - 7.94 = 243.37$

$\ln p(x) = 244.07,\ \text{gap}\ 0.70$

Together they are minus 6.2's ELBO, for one image. It sits just below ln p(x); the gap is the KL to the true posterior.

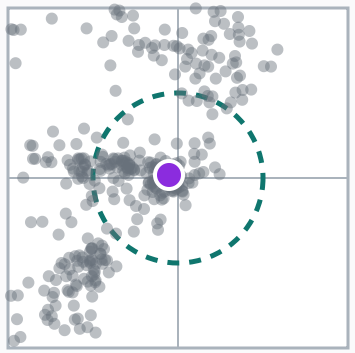
MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

From Lecture 25

The VAE's loss: reconstruction plus KL

```
x_hat, mu, logvar = model(x)
recon = ((x_hat - x)**2).sum(1) / (2 * 0.2**2)
kl = 0.5 * (mu**2 + logvar.exp() - logvar - 1).sum(1)
loss = (recon + kl).mean()
```

codes of 400 test digits



decoded from z ~ $\mathcal{N}(0, I)$



β = 1

KL per image: 9.02

squared error: 30.77

39 codes lie off this square

dashed circle: 2 sd of $\mathcal{N}(0, I)$

Weight the KL by β and train again. Click through the seven trained VAEs: when do the codes fill the circle, and when do samples look like digits?

MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

From Lecture 25

Check: the two ends of the β sweep

With β = 256 the KL term is 0.00 nats per image. What do the generated digits look like, and why?

β = 1: KL 9.02 per image



> Answer in pairs before the click. You need two ideas from this part: when a KL is zero, and what the decoder can learn from a code.

MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

From Lecture 25

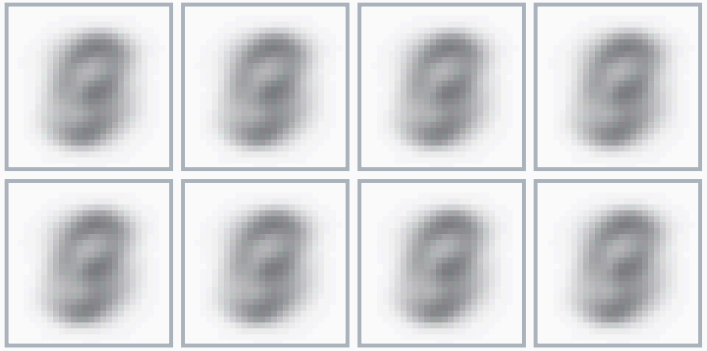
Check: the two ends of the β sweep

With β = 256 the KL term is 0.00 nats per image. What do the generated digits look like, and why?

β = 1: KL 9.02 per image



β = 256: KL 0.00 per image



All the same blur, the average digit. A KL of 0 means $q(z \mid x) = \mathcal{N}(0, I)$ for every image: the code says nothing about x, so the decoder draws the mean.

> Rebuilds fail too: squared error rises from 30.77 at β = 1 to 52.90 at β = 256.

MNIST test digit #90 and the trained networks of Lecture 25 (generative/lectures/lecture2/data). Numbers and images: scripts/sci/p6.py.

Part 6 in one slide

1. $\mathrm{KL}(p \,\|\, q)$: the extra cost of q when the data come from p.
2. No posterior? Raise the ELBO: $\ln p(\mathcal{D}) = \mathrm{ELBO} + \mathrm{KL}(q \,\|\, \text{posterior})$.
3. $\theta = \mu + \sigma\epsilon$ passes gradients through a sample; the VAE trains on minus the ELBO.

Variational autoencoder

Data	images alone, no labels
Model	an encoder $q(z \mid x)$ and a decoder $p(x \mid z)$
Output	a Gaussian over the code z, then over the image
Loss	$-\mathrm{ELBO} = \text{reconstruction error} + \mathrm{KL}(q \,\|\, \mathcal{N}(0, I))$
Update	a gradient step, through $z = \mu + \sigma\epsilon$
At test time	draw $z \sim \mathcal{N}(0, I)$ and decode it

Next: one more change of output, from one label to a list of objects.

Part 7 · From one label to a list of objects

Every model so far gave one answer per input. A detector returns a list of objects, as long as the photo needs.

1. The output is a list of boxes, each with a class and a score.
2. IoU says how well two boxes agree.
3. Every grid cell predicts, and the loss compares cell by cell.
4. At test time, keep the confident boxes and drop the duplicates.

Lecture 26

> By the end you can say how a network with a fixed-size output returns a list whose length changes from photo to photo.

From Lecture 26

From one label to a list of boxes



> Lecture 26’s courtyard photo. Before any boxes: if we gave it to a classifier from Part 5, what would come back?

Photo: Lecture 26’s courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. Classifier: ResNet-50 on ImageNet, recorded from Lecture 26 (data/models.json). Boxes and scores: YOLOv8n, recorded from Lecture 26 (data/science.json). Numbers: scripts/sci/p7.py.

From Lecture 26

From one label to a list of boxes



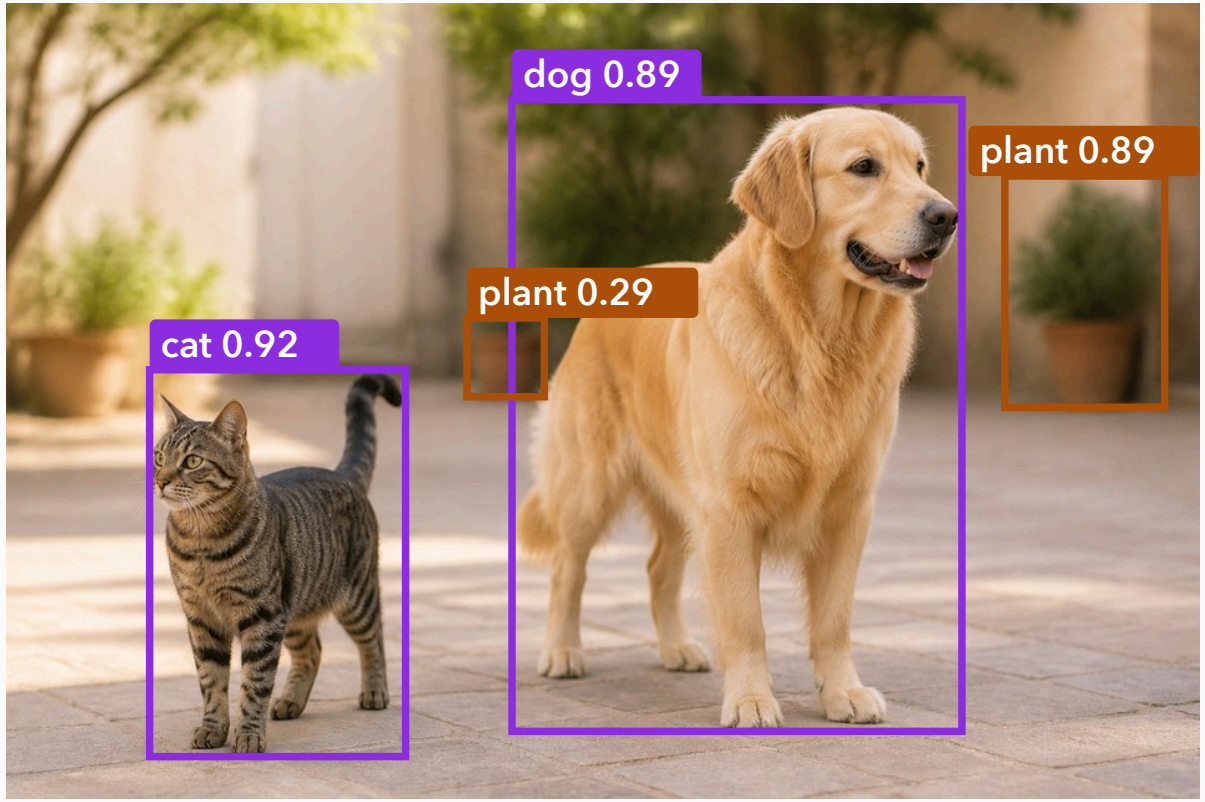
A classifier gives one label for the whole photo: **golden retriever, 0.19**.

> One label for the whole photo, with its probability. The cat, the plants, and where anything sits, are all lost.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. Classifier: ResNet-50 on ImageNet, recorded from Lecture 26 (data/models.json). Boxes and scores: YOLOv8n, recorded from Lecture 26 (data/science.json). Numbers: scripts/sci/p7.py.

From Lecture 26

From one label to a list of boxes



A classifier gives one label for the whole photo: **golden retriever, 0.19**.

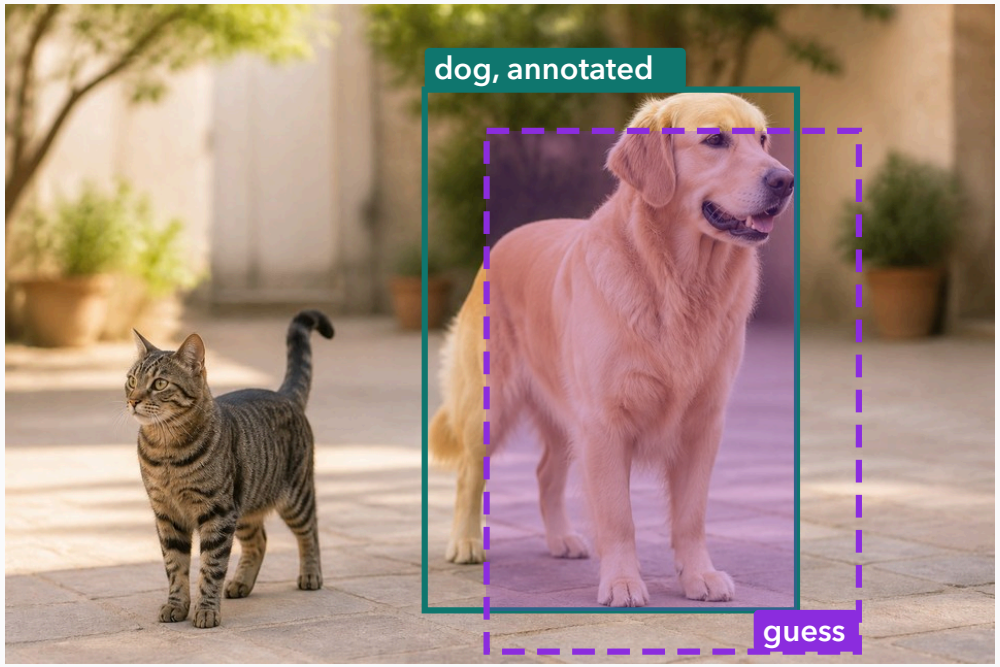
class	score	box (x_1, y_1, x_2, y_2)
cat	0.92	(116, 295, 321, 606)
potted plant	0.89	(803, 139, 932, 325)
dog	0.89	(407, 78, 769, 586)
potted plant	0.29	(371, 253, 433, 317)

> A detector returns a list: one row per object, each a class, a score and a box. Another photo gives a list of another length.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. Classifier: ResNet-50 on ImageNet, recorded from Lecture 26 (data/models.json). Boxes and scores: YOLOv8n, recorded from Lecture 26 (data/science.json). Numbers: scripts/sci/p7.py.

From Lecture 26

Do two boxes agree? Intersection over union



overlap 301 × 465 = 139,965 union 224,645 IoU **0.62**

$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)}$$

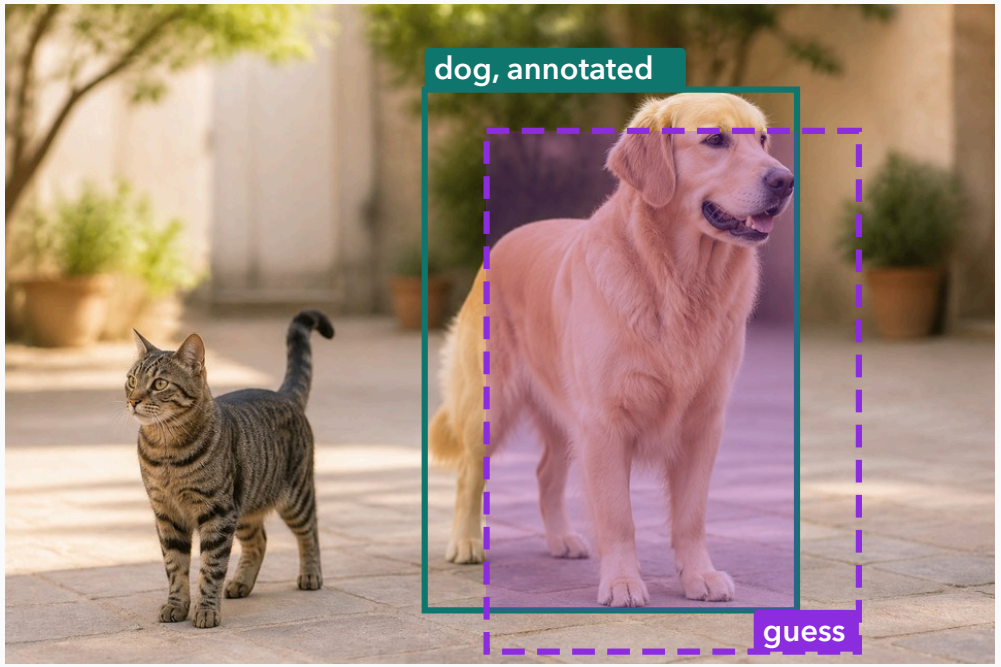
It is 1 for the same box and 0 for boxes that do not touch.

> The **annotated dog** and a **guess** the sliders move. Lecture 26's guess sat 60 px right and 40 px down. What IoU would you give it?

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. The dog box: Lecture 26's annotation; the guess: its box moved by hand. Numbers: scripts/sci/p7.py.

From Lecture 26

Do two boxes agree? Intersection over union



overlap 301 × 465 = 139,965 union 224,645 IoU **0.62**

$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)}$$

It is 1 for the same box and 0 for boxes that do not touch.

The lecture's guess:

$$\frac{139{,}965}{224{,}645} = 0.62$$

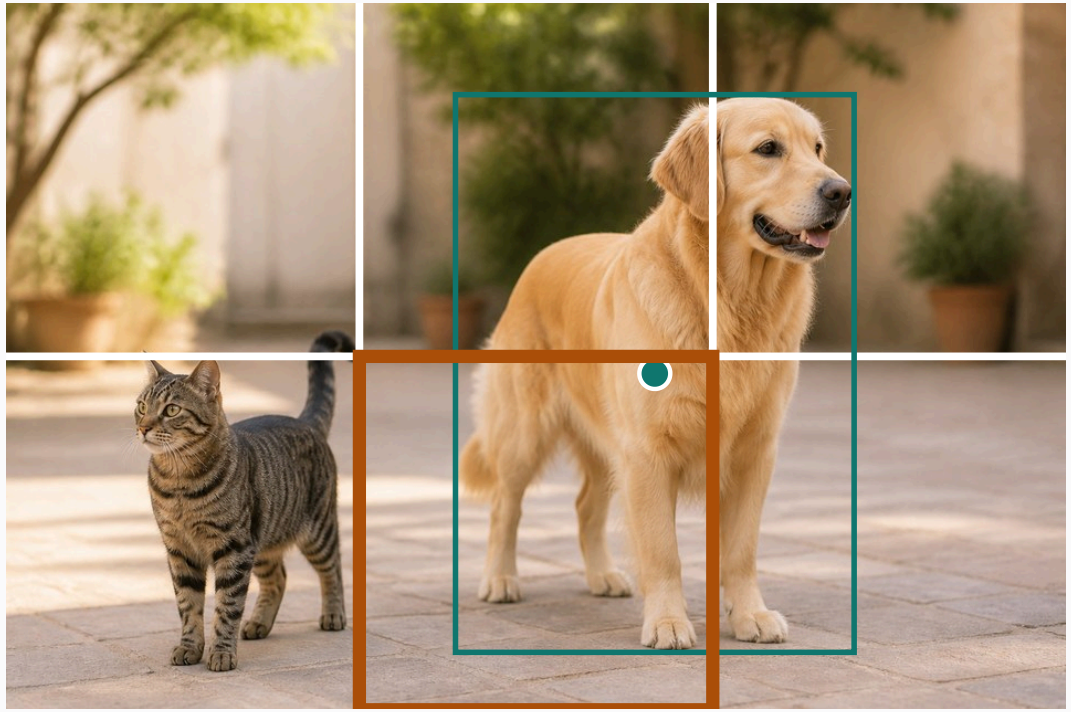
At least 0.5, so it counts as a match.

> A small shift already costs a lot. IoU decides three things today: which boxes NMS deletes, which detections count as right, and in training which cell owns a box.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. The dog box: Lecture 26's annotation; the guess: its box moved by hand. Numbers: scripts/sci/p7.py.

From Lecture 26

The loss, one grid cell at a time



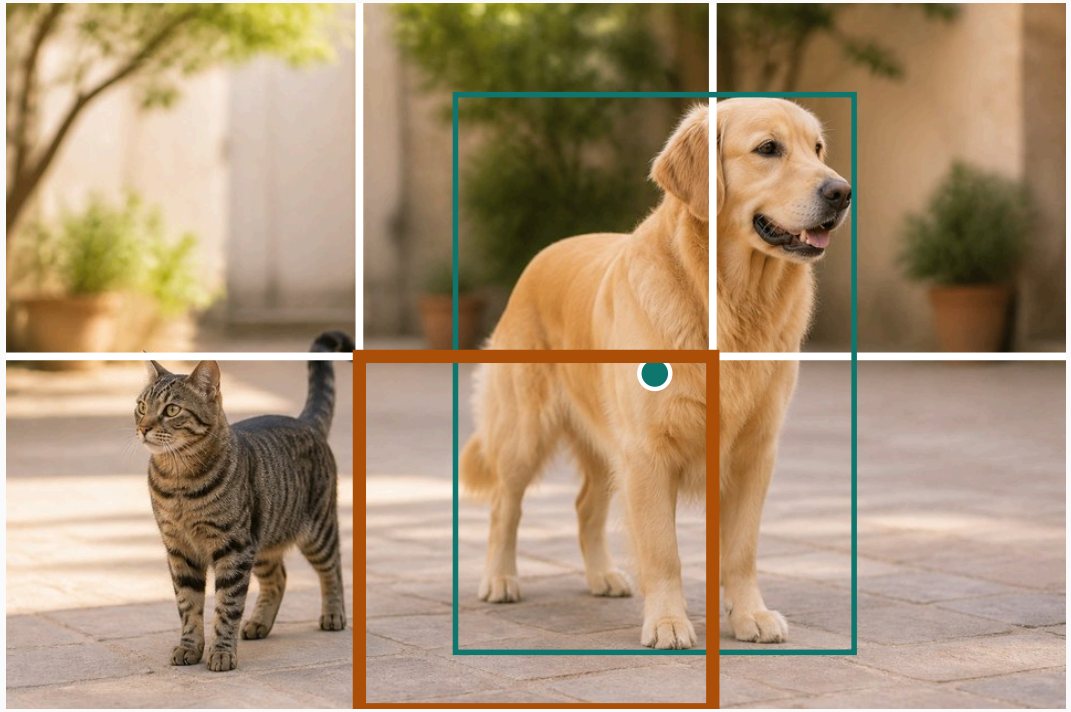
	object?	b_x	b_y	b_w	b_h	cat	dog
network $\hat{y}$	0.49	0.48	0.48	1.19	1.07	0.52	0.51
target y	1	0.84	0.05	1.13	1.58	0	1

A 3 × 2 grid of 320-pixel cells. The dog's centre lies in cell (1, 1), so that cell's **target** holds its box. The untrained **network** writes this.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. The network: Lecture 26's small grid CNN; its table before training and its loss after training recorded from data/models.json. Numbers: scripts/sci/p7.py.

From Lecture 26

The loss, one grid cell at a time



	object?	b_x	b_y	b_w	b_h	cat	dog
network $\hat{y}$	0.49	0.48	0.48	1.19	1.07	0.52	0.51
target y	1	0.84	0.05	1.13	1.58	0	1
squared	0.26	0.13	0.19	0.003	0.26	0.27	0.24

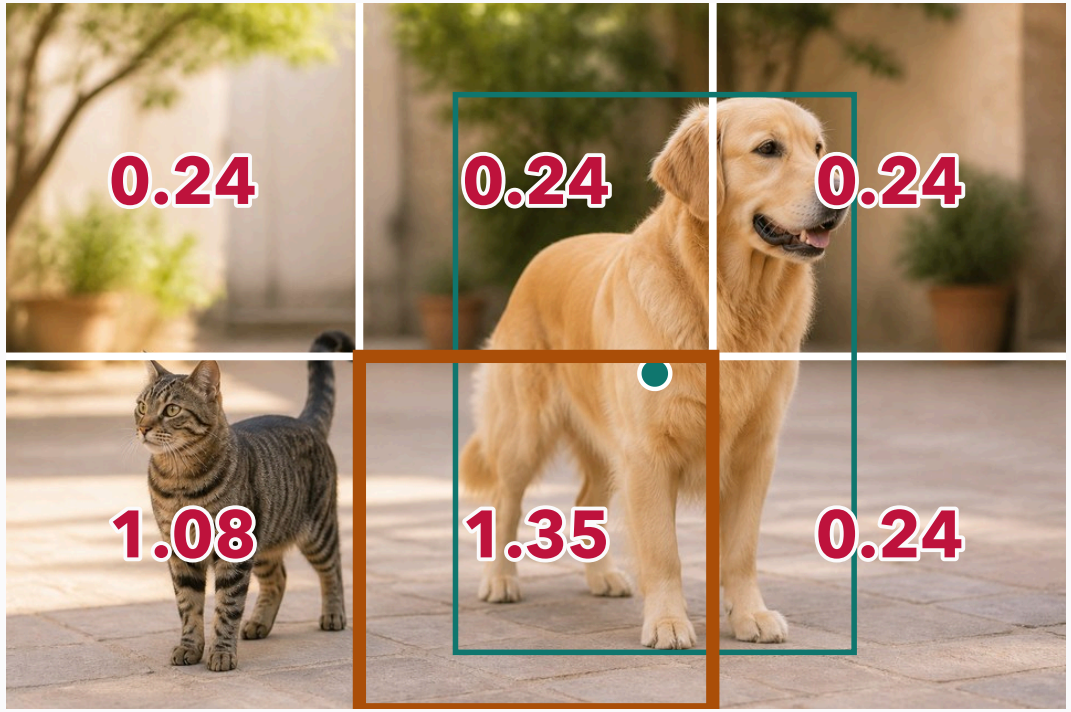
$$\sum_{i=1}^{7} (\hat{y}_i - y_i)^2 = 1.35$$

> Square each difference and add: the **loss** of the dog's cell. An empty cell pays only for its object? score.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. The network: Lecture 26's small grid CNN; its table before training and its loss after training recorded from data/models.json. Numbers: scripts/sci/p7.py.

From Lecture 26

The loss, one grid cell at a time



	object?	b_x	b_y	b_w	b_h	cat	dog
network $\hat{y}$	0.49	0.48	0.48	1.19	1.07	0.52	0.51
target y	1	0.84	0.05	1.13	1.58	0	1
squared	0.26	0.13	0.19	0.003	0.26	0.27	0.24

$$\sum_{i=1}^{7} (\hat{y}_i - y_i)^2 = 1.35$$

$$\mathcal{L} = \text{the sum over all six cells} = 3.38$$

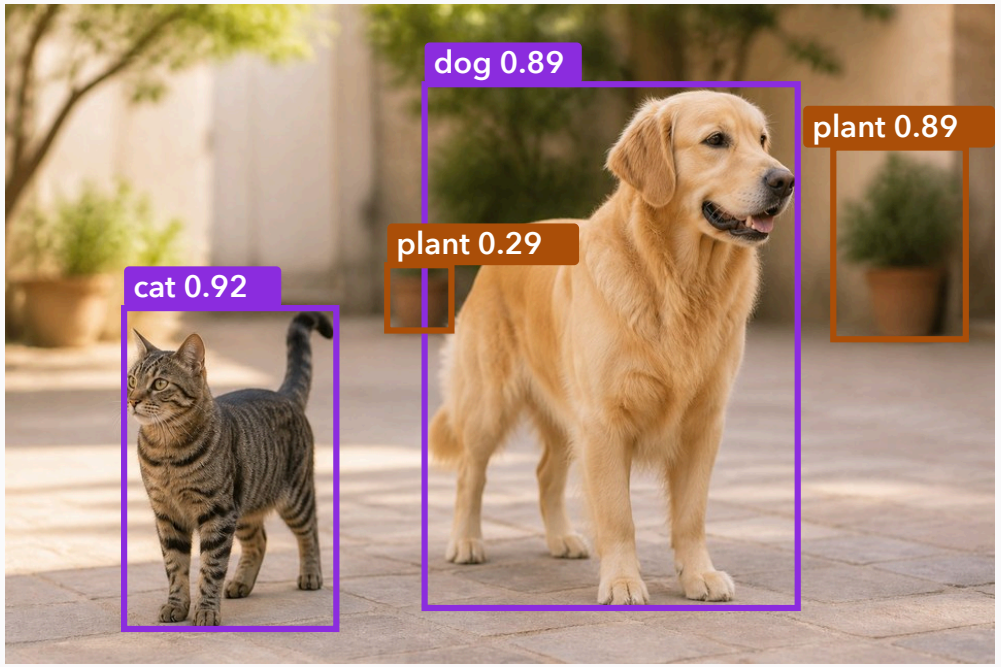
After 300 Adam steps on this one photo: **0.002**.

> Every cell's loss is written in its cell. Gradient steps on their sum move the network's table toward the target table.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. The network: Lecture 26's small grid CNN; its table before training and its loss after training recorded from data/models.json. Numbers: scripts/sci/p7.py.

From Lecture 26

Keep the confident boxes, then drop the duplicates



score ≥ 0.25: **31** boxes after NMS: **4**

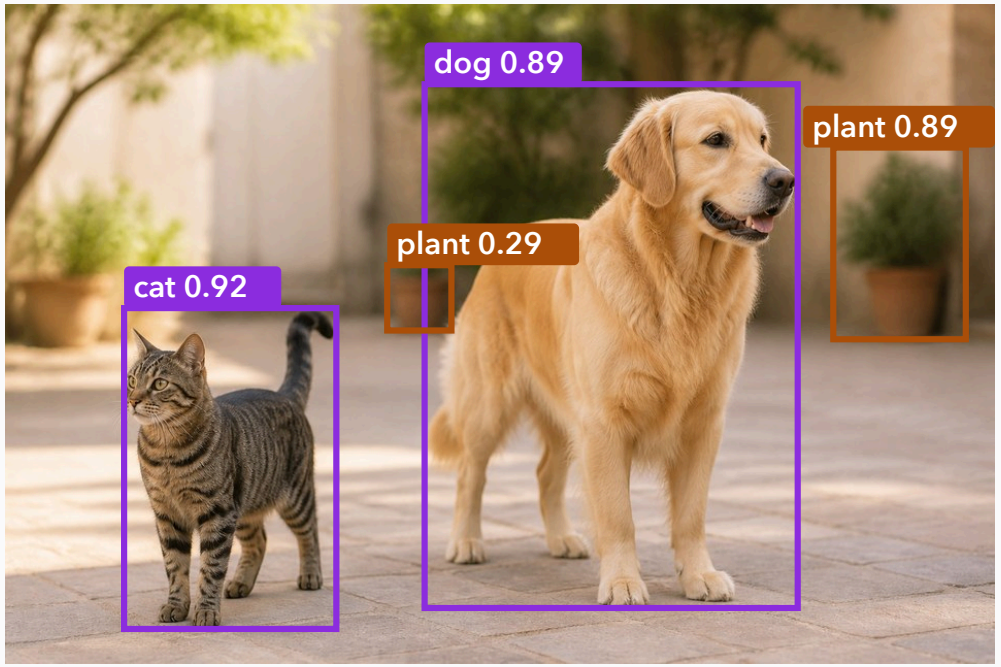
records, three grids	**12,600**
score ≥ 0.25	**31**
after NMS	**?**

> YOLOv8n writes a box and scores in every cell of three grids. 31 boxes pass a score of 0.25, several per object. How do we keep one each?

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. Boxes and scores: YOLOv8n, recorded from Lecture 26 (data/science.json). Class-aware NMS rerun here at every threshold. Numbers: scripts/sci/p7.py.

From Lecture 26

Keep the confident boxes, then drop the duplicates



score ≥ 0.25: **31** boxes after NMS: **4**

records, three grids	**12,600**
score ≥ 0.25	**31**
after NMS	**?**

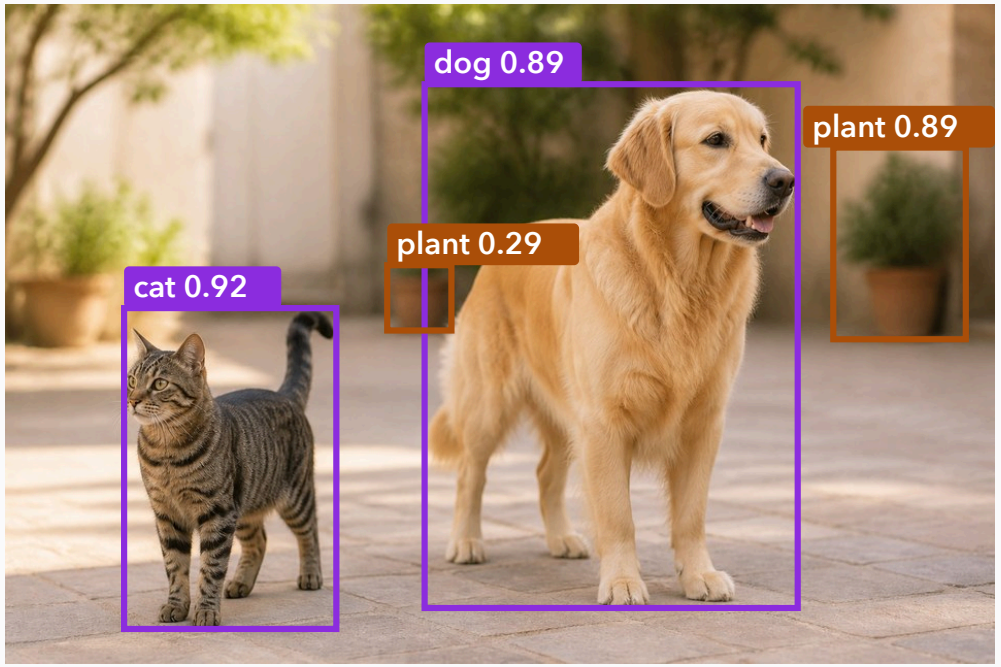
1. Keep the box with the best score.
2. Delete boxes of its class with IoU above 0.7 with it.
3. Repeat on the boxes left.

> Non-maximum suppression (NMS) keeps the best box and deletes its near copies, one class at a time. Press NMS in the figure.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. Boxes and scores: YOLOv8n, recorded from Lecture 26 (data/science.json). Class-aware NMS rerun here at every threshold. Numbers: scripts/sci/p7.py.

From Lecture 26

Keep the confident boxes, then drop the duplicates



score ≥ 0.25: **31** boxes after NMS: **4**

records, three grids	**12,600**
score ≥ 0.25	**31**
after NMS	**4**

1. Keep the box with the best score.
2. Delete boxes of its class with IoU above 0.7 with it.
3. Repeat on the boxes left.

The nine other dog boxes had IoU 0.96 or more with the kept one.

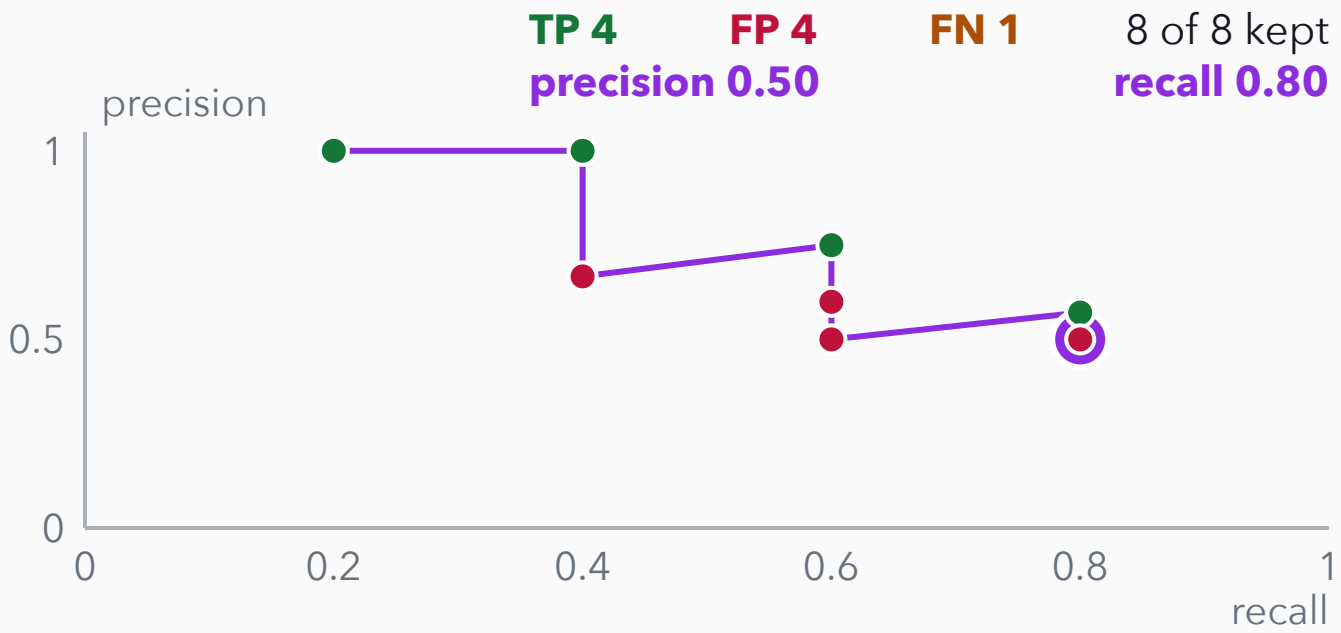
> Four detections survive, one per object: the list of 7.1. A fixed number of records became a list of any length.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. Boxes and scores: YOLOv8n, recorded from Lecture 26 (data/science.json). Class-aware NMS rerun here at every threshold. Numbers: scripts/sci/p7.py.

From Lecture 26

How good is the list? Precision, recall and AP

rank	score	IoU ≥ 0.5?
#1	0.95	TP
#2	0.90	TP
#3	0.80	FP
#4	0.70	TP
#5	0.60	FP
#6	0.45	FP
#7	0.35	TP
#8	0.20	FP



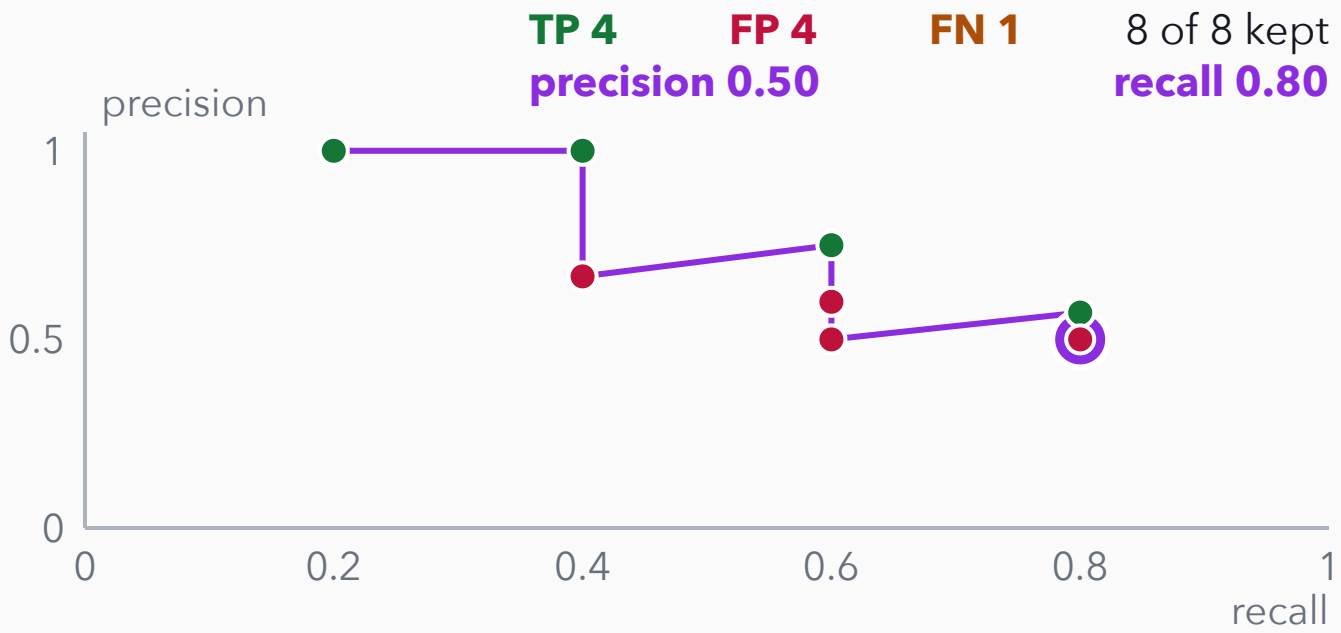
> Lecture 26's teaching example: five dogs, eight ranked boxes, each a TP or FP by IoU ≥ 0.5. Keep more boxes, one at a time: where does the point go?

The example: Lecture 26's dog park, five annotated dogs and eight made-up ranked boxes (data/dog-park.json); TP, FP, precision, recall and AP recomputed here. Numbers: scripts/sci/p7.py.

From Lecture 26

How good is the list? Precision, recall and AP

rank	score	IoU ≥ 0.5?
#1	0.95	TP
#2	0.90	TP
#3	0.80	FP
#4	0.70	TP
#5	0.60	FP
#6	0.45	FP
#7	0.35	TP
#8	0.20	FP



All eight boxes: precision 4/8 = 0.5, recall 4/5 = 0.8. Lowering the threshold moves the point right and, on the whole, down.

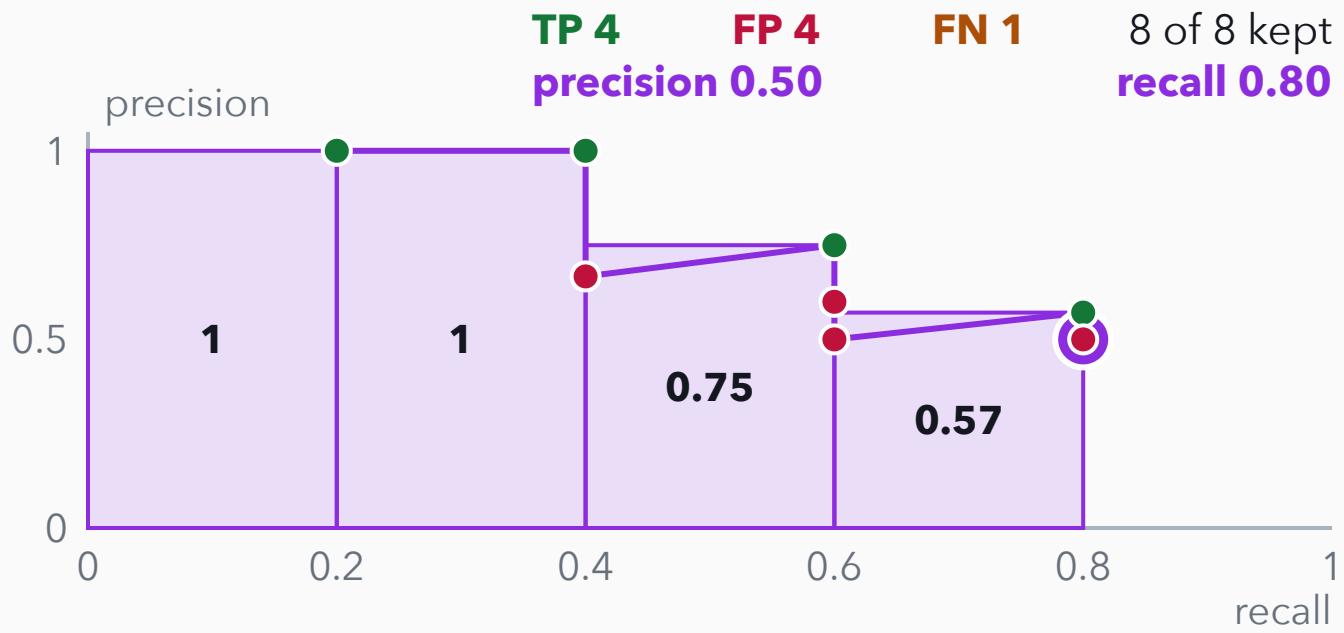
Each threshold is one point on the curve. Recall can only grow as boxes are added; precision falls whenever a wrong box comes in.

The example: Lecture 26's dog park, five annotated dogs and eight made-up ranked boxes (data/dog-park.json); TP, FP, precision, recall and AP recomputed here. Numbers: scripts/sci/p7.py.

From Lecture 26

How good is the list? Precision, recall and AP

rank	score	IoU ≥ 0.5?
#1	0.95	TP
#2	0.90	TP
#3	0.80	FP
#4	0.70	TP
#5	0.60	FP
#6	0.45	FP
#7	0.35	TP
#8	0.20	FP

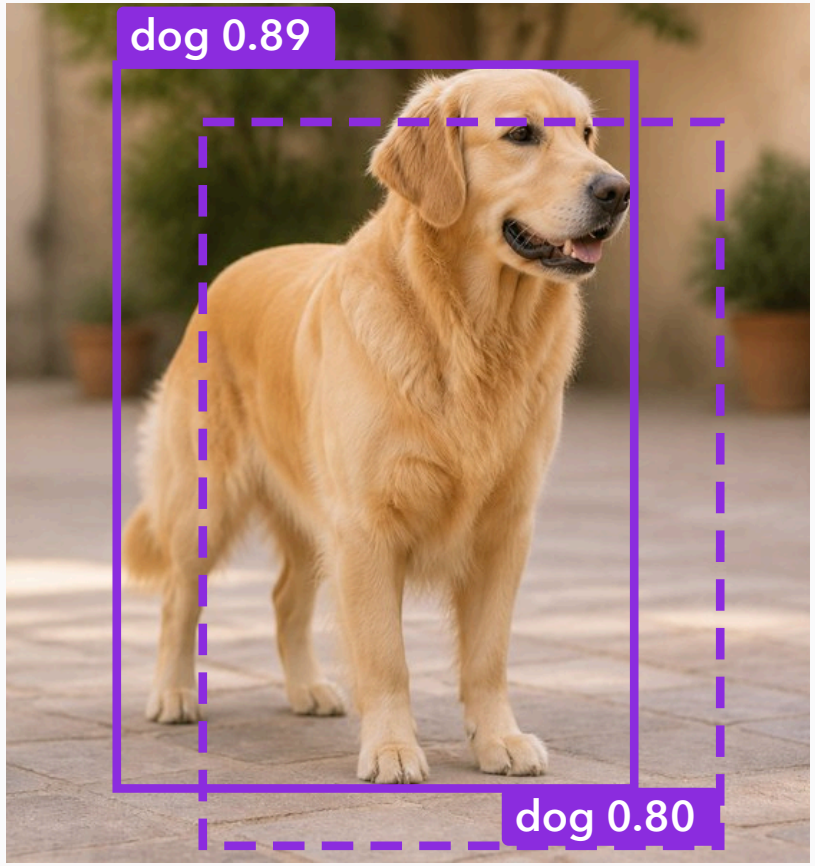


$$\text{AP} = 0.2 \times 1 + 0.2 \times 1 + 0.2 \times 0.75 + 0.2 \times 0.57 \approx 0.66$$

One number for the whole curve: the area under its upper envelope, the average precision (AP).

The example: Lecture 26's dog park, five annotated dogs and eight made-up ranked boxes (data/dog-park.json); TP, FP, precision, recall and AP recomputed here. Numbers: scripts/sci/p7.py.

Check: two boxes on one dog

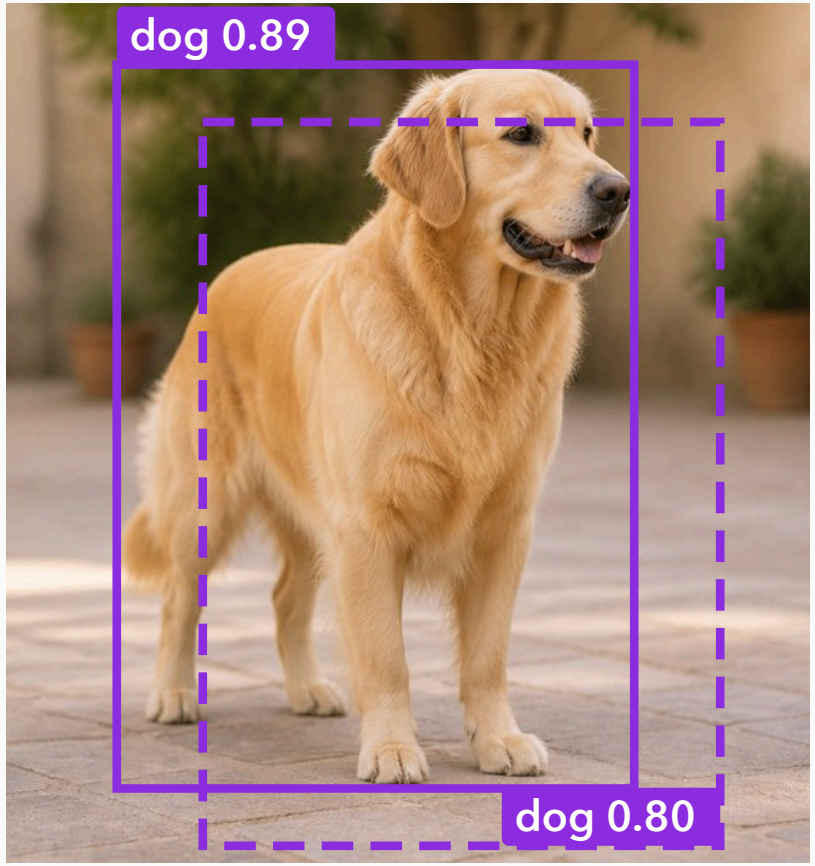


Two dog boxes: one on the dog (0.89) and the guess of 7.2 (0.80), with IoU 0.62. NMS deletes same-class boxes above 0.7; a match needs IoU ≥ 0.5. How many dogs are reported, and what is the precision?

> Two ideas at once, IoU and NMS. Work it out with your neighbour before the next click.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. The two scores are made up for this question; the IoU is the guess of 7.2. Numbers: scripts/sci/p7.py.

Check: two boxes on one dog



Two dog boxes: one on the dog (0.89) and the guess of 7.2 (0.80), with IoU 0.62. NMS deletes same-class boxes above 0.7; a match needs IoU ≥ 0.5. How many dogs are reported, and what is the precision?

Two: 0.62 is below 0.7, so NMS keeps both. The first box finds the dog; the second is a duplicate, a false positive. Precision 1/2 = 0.5.

NMS let the duplicate through, and evaluation counted it against us. An NMS threshold below 0.62 would delete it and give precision 1.

Photo: Lecture 26's courtyard. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed. The two scores are made up for this question; the IoU is the guess of 7.2. Numbers: scripts/sci/p7.py.

Part 7 in one slide

1. Every grid cell writes a box and scores.
2. IoU compares boxes; the loss compares cells.
3. A threshold and NMS turn 12,600 records into four boxes.

Object detector

Data	photos, each with its boxes and their classes
Model	a network with one output record per grid cell
Output	per cell: a box, an objectness score and class scores
Loss	box, objectness and class errors, summed over the cells
Update	a gradient step, as before
At test time	keep the confident boxes, drop duplicates (NMS)

Next: all seven parts side by side.

IoU and NMS in PyTorch

```python
def box_iou(a, b):  # a: (N, 4), b: (M, 4)
    lt = torch.max(a[:, None, :2], b[:, :2])
    rb = torch.min(a[:, None, 2:], b[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(-1)
    area_a = (a[:, 2:] - a[:, :2]).prod(-1)
    area_b = (b[:, 2:] - b[:, :2]).prod(-1)
    union = area_a[:, None] + area_b - inter
    return inter / union

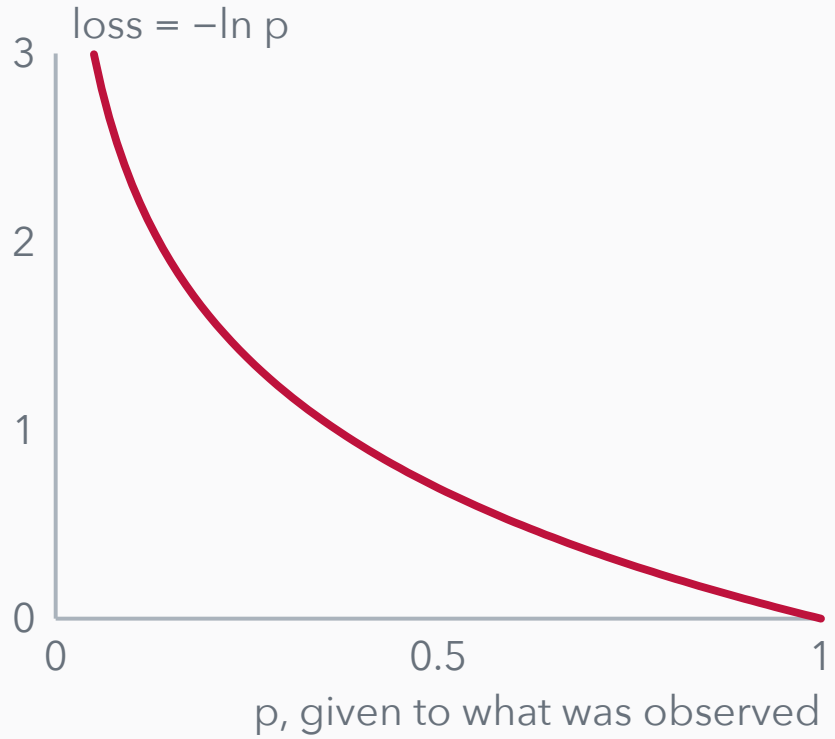
a = torch.tensor([[407., 83., 768., 588.]])
b = torch.tensor([[467., 123., 828., 628.]])
box_iou(a, b)  # 0.623
```

```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)
    keep = []
    while order.numel() > 0:
        i, rest = order[:1], order[1:]
        keep.append(i)
        ious = box_iou(boxes[i], boxes[rest])[0]
        order = rest[ious <= thr]
    return torch.cat(keep)
```

> Why does nms run once per class in a real detector?

Code: Lecture 26 (iou-code, nms-code).

The same loss, five times

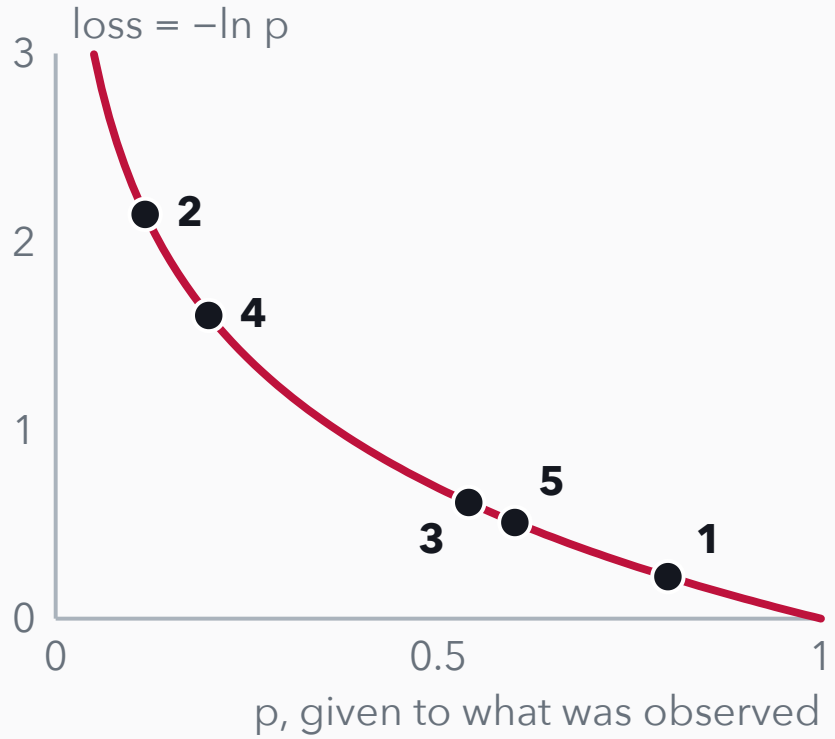


	Lecture	What was observed	p
1	2	**the label y = 1**	**0.80**
2	11	**the letter i after "aab"**	**0.12**
3	17	**the cat photo's own caption**	**0.54**
4	19	**the first answer token, "A"**	**0.20**
5	23 and 24	**the class cat**	**0.60**

> Five models from five parts of the course. Each gave a probability **p** to what was observed. Which one was penalised most?

Every number is read from this deck's Parts 1, 4, 5 and 6 (scripts/sci/p8.py).

The same loss, five times

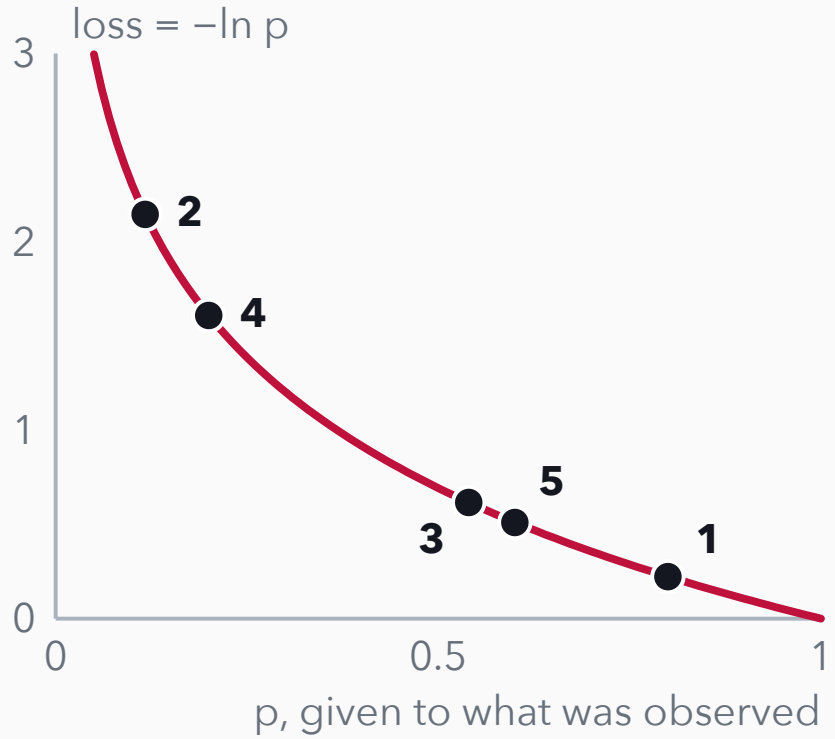


	Lecture	What was observed	p	−ln p
1	2	**the label y = 1**	**0.80**	**0.22**
2	11	**the letter i after "aab"**	**0.12**	**2.15**
3	17	**the cat photo's own caption**	**0.54**	**0.62**
4	19	**the first answer token, "A"**	**0.20**	**1.61**
5	23 and 24	**the class cat**	**0.60**	**0.51**

> Each loss is **−ln p**, so all five sit on one curve.

Every number is read from this deck's Parts 1, 4, 5 and 6 (scripts/sci/p8.py).

The same loss, five times



	Lecture	What was observed	p	−ln p
1	2	**the label y = 1**	**0.80**	**0.22**
2	11	**the letter i after "aab"**	**0.12**	**2.15**
3	17	**the cat photo's own caption**	**0.54**	**0.62**
4	19	**the first answer token, "A"**	**0.20**	**1.61**
5	23 and 24	**the class cat**	**0.60**	**0.51**

Row 2, the character model of Lecture 11. Its **p = 0.12** is the smallest, so its loss **2.15** is the largest.

The loss reads one number, the probability given to what was observed.

Every number is read from this deck's Parts 1, 4, 5 and 6 (scripts/sci/p8.py).

The same block, in four models

	tokens	mask	the output is read at
Language model Lectures 12 to 14	the sun rose in the east		every position: the next token

a word piece an image patch [CLS] · dark outline: the output is read here · filled mask cell: may attend

> Part 4 built one block: self-attention under a mask. A language model hides the future and predicts the next token at every position.

Parts 4 and 5 of this deck; Lectures 12 to 19.

The same block, in four models

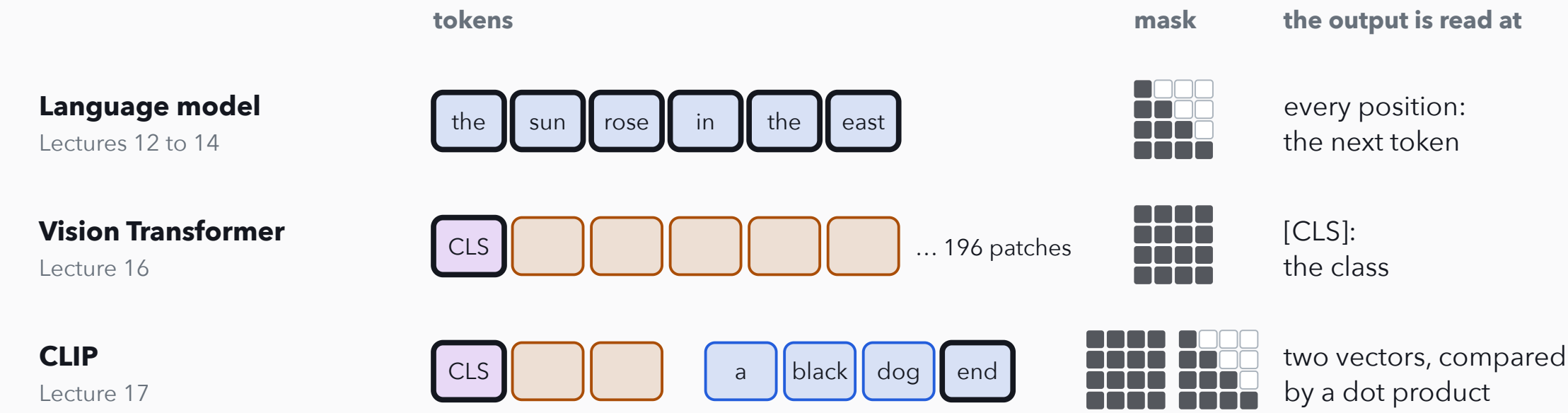
	tokens	mask	the output is read at
Language model Lectures 12 to 14	the · sun · rose · in · the · east		every position: the next token
Vision Transformer Lecture 16	CLS · … 196 patches		[CLS]: the class

a word piece · an image patch · [CLS] · dark outline: the output is read here · filled mask cell: may attend

> Lecture 16 changed what a token is. A photo became 196 patches and a [CLS] token; nothing is hidden, and the class is read at [CLS].

Parts 4 and 5 of this deck; Lectures 12 to 19.

The same block, in four models

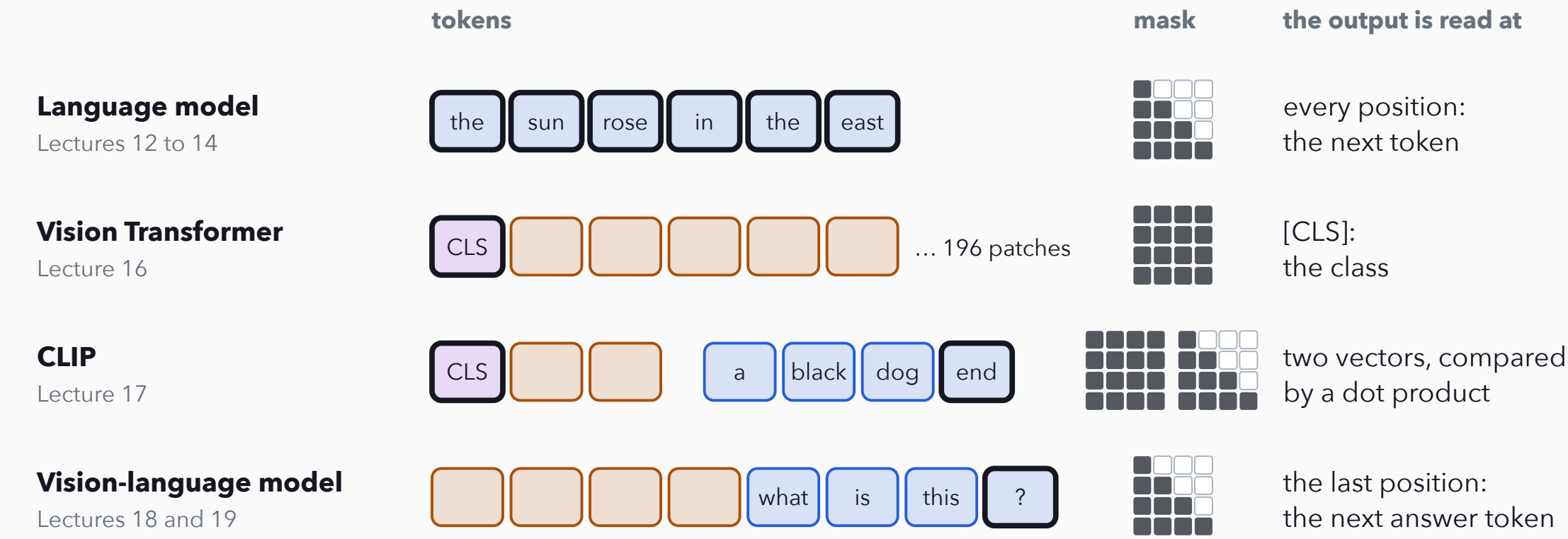


a word piece an image patch [CLS] · dark outline: the output is read here · filled mask cell: may attend

> CLIP runs two encoders, one for the photo and one for the caption, and compares the two vectors they return.

Parts 4 and 5 of this deck; Lectures 12 to 19.

The same block, in four models



a word piece an image patch [CLS] · dark outline: the output is read here · filled mask cell: may attend

> A vision-language model puts the 196 patch rows in front of the question, in one sequence. The answer comes out at the last position, one token at a time.

Parts 4 and 5 of this deck; Lectures 12 to 19.

Seven parts, six rows

Part	Lectures	Data	Model	Output	Loss	Update	At test time
1 · Foundations	1 to 4		▬	▬	▬		
2 · Gradients	5 to 9					▬	
3 · Generalization	10	▬			▬		▬
4 · Attention	11 to 15	▬	▬	▬			▬
5 · Vision	16 to 22	▬	▬	▬	▬		▬
6 · Generative	23 to 25	▬	▬	▬	▬		▬
7 · Detection	26	▬		▬	▬		▬
Parts that changed it		**5**	**4**	**5**	**5**	**1**	**5**

> A bar marks a row of the card that the part changed. Read down the Update column: why is it almost empty?

Seven parts, six rows

Part	Lectures	Data	Model	Output	Loss	Update	At test time
1 · Foundations	1 to 4		▬	▬	▬		
2 · Gradients	5 to 9					▬	
3 · Generalization	10	▬			▬		▬
4 · Attention	11 to 15	▬	▬	▬			▬
5 · Vision	16 to 22	▬	▬	▬	▬		▬
6 · Generative	23 to 25	▬	▬	▬	▬		▬
7 · Detection	26	▬		▬	▬		▬
Parts that changed it		**5**	**4**	**5**	**5**	**1**	**5**

Part 2 settled it. Backprop and an optimizer work for any differentiable loss, so the later parts changed the loss and left the step alone.

> This is why one training loop could serve every model of the course.

Ten models, one card each

MLP classifier · Lecture 4

Data	pairs (x, y): an input and its class
Model	an MLP gives scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

> Pick a model. Its card shows the six rows, and the tinted rows are the ones that differ from the MLP classifier's. How many models tint the Update row?

Ten models, one card each

MLP classifier · Lecture 4

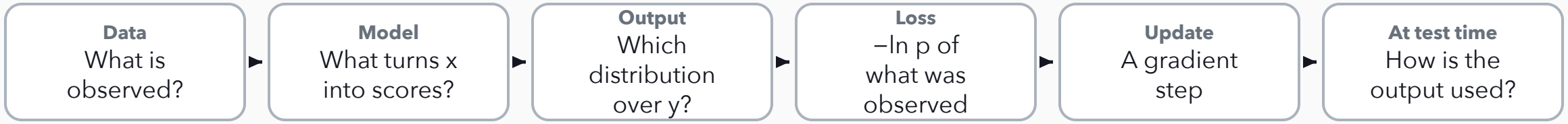
Data	pairs (x, y): an input and its class
Model	an MLP gives scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

Two: the Bayesian neural network and the variational autoencoder. Both draw a sample inside the model, and the gradient has to pass through it.

> Even there it is a gradient step. Everything else that changed is data, model, output, loss or use.

The recipe, for a problem of your own

Six questions



The card as a flowchart. Each box is a question to answer before you write any code.

Lectures 1 to 3 for the Gaussian likelihood; Lecture 9 for the positive spread.

The recipe, for a problem of your own

Six questions

Data	Model	Output	Loss	Update	At test time
What is observed?	What turns x into scores?	Which distribution over y?	−ln p of what was observed	A gradient step	How is the output used?

What the course put in each box

Data	Model	Output	Loss	Update	At test time
labels, tokens, captions, boxes	MLP, attention blocks, encoders	softmax, Gaussian, a list of boxes	cross-entropy, squared error, −ELBO	SGD, momentum, Adam	pick, sample, correct, filter

> These are the answers the course gave. Now a new problem: forecast tomorrow's PM2.5 and say how sure you are. What goes in each box?

Lectures 1 to 3 for the Gaussian likelihood; Lecture 9 for the positive spread.

The recipe, for a problem of your own

Six questions

Data	Model	Output	Loss	Update	At test time
What is observed?	What turns x into scores?	Which distribution over y?	−ln p of what was observed	A gradient step	How is the output used?

What the course put in each box

Data	Model	Output	Loss	Update	At test time
labels, tokens, captions, boxes	MLP, attention blocks, encoders	softmax, Gaussian, a list of boxes	cross-entropy, squared error, −ELBO	SGD, momentum, Adam	pick, sample, correct, filter

Tomorrow's PM2.5, with an error bar

Data	Model	Output	Loss	Update	At test time
today's weather, tomorrow's PM2.5	an MLP	a Gaussian: mean and spread	−ln of its density at the measured value	Adam, with a schedule	mean ± two spreads

> One good answer. The Gaussian output carries the error bar, and its loss is squared error divided by the predicted spread, plus a term for the spread.

Lectures 1 to 3 for the Gaussian likelihood; Lecture 9 for the positive spread.

The course on one slide

One loss

- The model outputs a probability distribution.
- The loss is $-\ln p$ of what was observed.

One update

- Backprop gives the gradient for every parameter.
- SGD, momentum or Adam takes the step.

Ten models

- Attention reads characters, word pieces and image patches.
- Each model fills the same six rows of the card.

> Twenty-six lectures as one method. For your own problem, which row of the card comes first?

Recipe card: MLP classifier

MLP classifier · Lecture 4

Data	pairs (x, y): an input and its class
Model	an MLP gives scores $z = f_\theta(x)$
Output	probabilities $p = \mathrm{softmax}(z)$
Loss	$-\ln p_y$, the right class
Update	$\theta \leftarrow \theta - \eta \nabla_\theta \,\mathrm{loss}$
At test time	the class with the largest p

Which rows differ from the MLP classifier's card?

Recipe card: character language model

Character language model · Lecture 11

Data	names: three characters in, the next one as the target
Model	three embeddings, joined, then an MLP
Output	p over the 27 characters
Loss	$-\ln p(\text{next character})$
Update	a gradient step, as before
At test time	sample a character, slide along, repeat

Which rows differ from the MLP classifier's card?

Recipe card: transformer language model

Transformer language model · Lectures 12 to 15

Data	text as tokens; the target at each position is the next token
Model	embeddings, then blocks of masked self-attention
Output	p over the vocabulary, at every position
Loss	$-\ln p(\text{next token})$, averaged over positions
Update	a gradient step, as before
At test time	sample a token, append it, repeat

Which rows differ from the MLP classifier's card?

Recipe card: Vision Transformer

Vision Transformer · Lecture 16

Data	photos with a class; each photo is cut into patches
Model	patch embeddings and [CLS], then attention blocks
Output	p over the classes, read from [CLS]
Loss	$-\ln p_y$, the right class
Update	a gradient step, as before
At test time	the class with the largest p

> Which rows differ from the MLP classifier's card?

Recipe card: CLIP

CLIP · Lecture 17

Data	photos with their captions; no class labels
Model	an image encoder and a text encoder, one shared space
Output	for each photo, p over the captions in the batch
Loss	$-\ln p(\text{its own caption})$, and the same from caption to photo
Update	a gradient step, as before
At test time	write each class as a caption, pick the closest

> Which rows differ from the MLP classifier's card?

Recipe card: vision-language model

Vision-language model · Lectures 18 and 19

Data	a photo, a question and the answer in words
Model	an image encoder, a projector, then a language model
Output	p over the vocabulary, for each answer token
Loss	$-\ln p(\text{next token})$, on the answer
Update	a gradient step, as before
At test time	generate the answer one token at a time

Which rows differ from the MLP classifier's card?

Recipe card: concept bottleneck model

Concept bottleneck model · Lectures 20 and 21	
Data	photos with concept labels c and a class y
Model	photo, then concepts $\hat{c}$, then the class $\hat{y}$
Output	a probability per concept, then p over the classes
Loss	a concept loss plus a class loss, each with a weight
Update	a gradient step, as before
At test time	predict the concepts, then the class; a person can correct a concept

> Which rows differ from the MLP classifier's card?

Recipe card: Bayesian neural network

Bayesian neural network · Lectures 23 and 24

Data	a few pairs (x, y)
Model	a network whose weights are drawn from $q(\theta)$
Output	$p(y \mid x, \theta)$ for each sampled θ
Loss	$-\text{ELBO} = -\mathbb{E}_q[\ln p(\mathcal{D} \mid \theta)] + \text{KL}(q \,\|\, \text{prior})$
Update	a gradient step on μ, σ, through $\theta = \mu + \sigma\epsilon$
At test time	average over samples of θ; their spread is the uncertainty

Which rows differ from the MLP classifier's card?

Recipe card: variational autoencoder

Variational autoencoder · Lecture 25

Data	images alone, no labels
Model	an encoder $q(z \mid x)$ and a decoder $p(x \mid z)$
Output	a Gaussian over the code z, then over the image
Loss	$-\mathrm{ELBO} = \text{reconstruction error} + \mathrm{KL}(q \,\|\, \mathcal{N}(0, I))$
Update	a gradient step, through $z = \mu + \sigma\epsilon$
At test time	draw $z \sim \mathcal{N}(0, I)$ and decode it

> Which rows differ from the MLP classifier's card?

Recipe card: object detector

Object detector · Lecture 26	
Data	photos, each with its boxes and their classes
Model	a network with one output record per grid cell
Output	per cell: a box, an objectness score and class scores
Loss	box, objectness and class errors, summed over the cells
Update	a gradient step, as before
At test time	keep the confident boxes, drop duplicates (NMS)

Which rows differ from the MLP classifier's card?

Lectures 1 to 5, with their material

	Lecture	Material
1	Where Deep Learning Losses Come From I	Notes Slides Cheat sheet
2	Where Deep Learning Losses Come From II	Notes Slides Cheat sheet
3	Likelihood to Loss in Practice	Notes Slides Cheat sheet Likelihood notebook Robust regression
4	From Linear Models to Neural Networks	Notes Slides Cheat sheet Linear gradient descent XOR
5	Optimization I: Gradient Estimates & Step Sizes	Notes Slides Cheat sheet Full recording

> Which of these would you reread first?

Lectures 6 to 10, with their material

	Lecture	Material
6	Backpropagation & Autograd from Scratch	Notes Slides Cheat sheet Full recording Colab notebook
7	Optimization II: Momentum, RMSProp & Adam	Notes Slides Cheat sheet Full recording 1 Full recording 2
8	Vector, Matrix & Affine Autograd	Calculus notes Autograd notes Calculus slides Autograd slides Cheat sheet Colab notebook
9	Valid Outputs by Design & Learning-Rate Schedules	Valid outputs notes LR schedule notes Valid outputs slides LR schedule slides Cheat sheet
10	Generalization & Regularization	Notes Slides Cheat sheet Learning curves Dropout and regularization

> Which of these would you reread first?

Lectures 11 to 15, with their material

	Lecture	Material
11	From characters to next-token prediction	Notes Slides Cheat sheet Full recording
12	Self-attention, from first principles	Notes Slides Cheat sheet Full recording
13	Self-attention, continued	Notes Slides Cheat sheet
14	Self-attention and multi-head attention	Self-attention notes Multi-head attention notes Self-attention slides Multi-head attention slides
15	From Attention to Applications	Notes PDF Slides Full recording

> Which of these would you reread first?

Lectures 16 to 20, with their material

	Lecture	Material
16	**Vision Transformer for image classification**	Notes Slides PDF · 55 pp. Full recording
17	**CLIP: From Fixed Labels to Language-Defined Vision**	Notes Slides PDF · 168 pp. Addendum slides Addendum PDF Full recording Applications Colab
18	**From CLIP to Vision-Language Models**	Notes Slides PDF · 178 pp. Full recording Three connections Colab
19	**Vision-Language Models, Part 2**	Notes Slides PDF · 178 pp. Full recording Three connections Colab
20	**Concept Bottleneck Models: I**	Notes Slides PDF · 262 pp. Full recording

> Which of these would you reread first?

Lectures 21 to 26, with their material

	Lecture	Material
21	Concept Bottleneck Models: II	Notes Slides PDF · 262 pp. Full recording
22	Learning ViTs with AI Tools	Full recording
23, 24	KL Divergence and Variational Inference	Notes Slides PDF · 154 pp. Notebook
25	Autoencoder and VAE	Notes Slides PDF · 85 pp. Notebook
26	Object Detection: From One Label to a Set of Objects	Notes Slides PDF · 154 pp.

> Which of these would you reread first?